

the complete **spectroscope** user guide

One headless core, many faces. Every feature of the full build, from the terminal, the web UI, the desktop shell, the Lab, subagents, skills, MCP, voice and vision to the fleet on the bus, explained with real screenshots, and every protocol, event and endpoint documented down to the wire.

7

MODULES ON ONE STREAM

18

RUNEVENT TYPES

22+

TOOLS (+ MCP)

3

LLM BACKENDS (5 PRESETS)

678

JUNIT TESTS

383

VITEST TESTS

3

DESIGNS (DARK · BRIGHT · WHITE)

7

BUILT-IN SCENARIOS

How to read this book

A NOTE ON THE SCREENSHOTS

Every screenshot is a real capture of the running app, taken by the repository's own capture pipeline (`capture_screens.mjs`) and reshot per release. The two editions of this guide carry matching sets: the dark pages show **spectro dark**, the light pages **spectro bright**.

spectroscope is the reference harness of the workshop “build an agent harness from scratch” — and the full build in the `spectroscope/` folder is its final form: everything the ten core stages and six bonus stages construct, plus the conveniences that never fit a single stage. This guide documents that full build. Not the journey — the machine.

The book has five parts, ordered from “I just want to use it” to “I want to know exactly what crosses which wire”:

- **Part I — Meet spectroscope:** what it is, how to start it, what it can do.
- **Part II — The faces:** the terminal, the web UI with all its tabs and panels, the Lab, session management, the design system, the desktop app.
- **Part III — The agent's powers:** tools, permissions, hooks, subagents, skills, providers, vision, voice, image generation, MCP and scheduling.
- **Part IV — The file system:** where everything lives on disk and what each file looks like inside.
- **Part V — Under the hood:** the complete technical reference — the architecture dossier, every event, every socket frame, every REST endpoint, every configuration key.

HONESTY RULES OF THIS DOCUMENT

Every screenshot in this guide is a real capture of the running application (English chrome, taken 2026-07-20 with the reproducible script in `docs/guide-assets/capture_screens.mjs`). Every terminal block is real command output. Every name, constant, wire string and default was read from the migration phase-6 source tree (2026-07-20) — not from memory. Where behaviour is a deliberate limitation, the guide says so.

Conventions

- `monospace` marks anything that exists literally in code, config or on the wire: tool names, event types, JSON keys, commands.
- Chips classify tools: `free` needs no permission, `gated` asks first, `gated + prefix rule` asks first and remembers approvals scoped to a prefix.
- German UI labels appear in parentheses where the chrome ships bilingual — the screenshots use the English chrome.
- File paths are relative to the `spectroscope/` root unless they start with `~`.

CONTENTS

The map of this book

0 How to read this book

Conventions

PART I Meet spectroscope

1 What is spectroscope?

One core, five faces
Additive forever
Tool inputs are model output
The stack, in one breath

2 Quick start

The launcher
The thirty-second start
What doctor tells you
The provider matrix

3 The capability map

PART II The faces

4 The command line

Commands and global flags
The REPL
Slash commands
The twelve doctor checks
The tour

5 The web face: Chat

The anatomy of the screen
The conversation
The composer
The permission dialog
Live indicators

6 The web face: tabs & panels

The Graph tab
The Trace tab
The Text tab
The right panel
The image gallery

6b the spectrum and the new lenses

the spectrum tab
the reasoning lens
the causal chain and the replay scrubber
the gate surface
the explain panel
spectro doctor, the web face

7 The Lab

The dam
The transport
The Flow map (React Flow)
The keymap
Under the hood: the Petri marking
Scenarios: deterministic demo runs

8 Sessions: replay, resume, delete, import

The archive
Resume — with the re-upload made visible
Delete — the two-step danger button
Import
New chat semantics

9 The design system

The token base
Three designs
The effects layer

10 The desktop face

The supervision sequence
A self-contained run kit
Tray, notifications, quitting

11 Two languages

PART III The agent's powers

12 The tool belt

- The shared rules
- Files & search
- Mutation & shell
- Metadata & delegation
- Who gets which tools

13 Permissions, allowlist & hooks

- The guard pipeline
- Permission modes
- The allowlist
- Hooks
- The audit trail

14 Subagents & the A2A protocol

- Two roles
- The A2A-lite lifecycle
- The development tools

15 Skills

- The package format
- Progressive disclosure
- The four shipped skills

15b the fleet: many agents, one spectrum

- what rides the bus
- spectro node: the process fleet
- code fleets on the hub
- entering a fleet
- the machine room
- the control plane

16 Providers & models

- The backends
- First run — choosing a backend
- Putting a key in the .env
- Switching mid-session
- Retry — a loop that doesn't tip over
- Prompt caching (Anthropic)
- The wrapper chain

17 Seeing, hearing, speaking, painting

- Vision — image input
- Voice input — local push-to-talk
- Voice output — spoken answers
- Image generation

18 MCP — external tool servers

- Configuration
- Call semantics: at-most-once
- The example server: spectro-mcp-notes

19 Scheduling

PART IV The file system

20 Where everything lives

- Inside a session file
- Blobs and generated images
- The workspace: where the agent actually works
- Lifecycle: create, resume, delete
- The jq cookbook

PART V Under the hood

21 The architecture dossier

- The big picture
- The protocol layer
- The model side
- The loop and the belt
- Delegation
- The web face
- Runtime and integration
- The codebase itself

22 The RunEvent protocol

- The wire rules
- The catalog
- The cross-edition proof
- Resume: lines become a conversation

23 Streams & cancellation

- EventStream
- MergedEventStream
- CancelSignal

24 The provider wire

The port
Backend mappings

25 The WebSocket protocol

Client → server
Server → client
Threading model

26 The REST API

27 The agent loop, hop by hop

The turn, precisely
The guard pipeline (per tool call)
The headless variant

28 Context: compaction, caching, introspection

Compaction
Prompt caching
Introspection

29 MCP internals

JSON-RPC framing
Client semantics
The reference server

30 Configuration reference

The layers
Every key
Environment-only variables
SPECTRO.md

31 Build & test inventory

Modules
Version pins
The test suite

32 Releases & working with the source

What a release ships
The libraries on Maven Central
The desktop run kit
Working with the source
The release ritual
The brand mark

33 Saving a key, safely

Where the key goes
Why it works without a restart
The two fences on the write
What an attacker can and cannot do
You never have to use it

A Troubleshooting

B Reproducing this guide

Credits and colophon



PART I

Meet spectroscope

What the harness is, the one idea everything hangs on, how to start it in thirty seconds, and a map of everything it can do.

- 1 What is spectroscope?
- 2 Quick start
- 3 The capability map

What is spectroscope?

spectroscope is an agent harness: the machinery around a large language model that turns “a model that can talk” into “an agent that can work” — read files, run commands, ask for permission, delegate to subagents, remember sessions, and show you every step it takes while doing so.

It was built from scratch across a sixteen-stage workshop (Java 21, Gradle, Spring only where it earns its place), and the `spectroscope/` full build is the sum of all of it. Three properties define the design:

One core, five faces

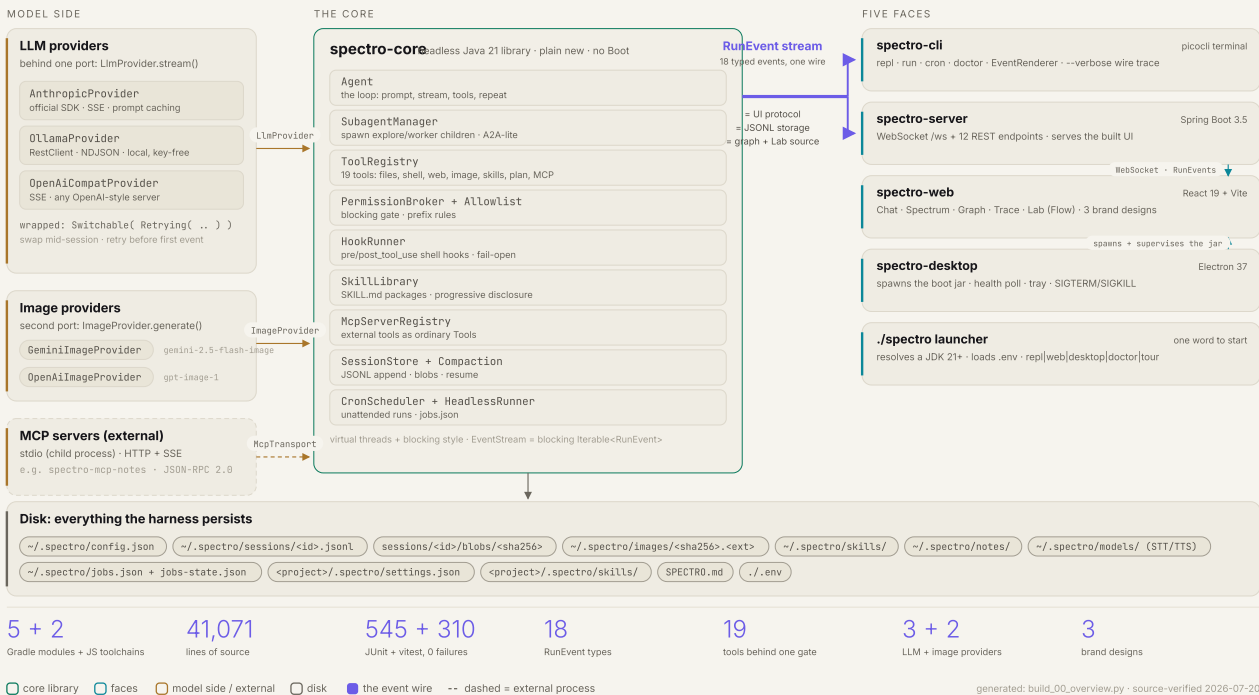
The heart of spectroscope is `spectro-core` — a headless Java library with no container, no UI and no I/O of its own. Everything in it is constructible with plain `new`. When the agent runs, the core produces exactly one thing: a typed stream of **RunEvents**. Everything else in the project is a *renderer* of that stream:

- the **CLI** (`spectro-cli`) renders it as ANSI text with tool cards and a spinner,
- the **web backend** (`spectro-server`) pushes it over a WebSocket,
- the **browser UI** (`spectro-web`) folds it into chat, graph, trace, Lab and panels,
- the **desktop shell** (`spectro-desktop`) wraps that same web UI in a supervised window,
- the **session store** writes it line by line into a JSONL file.

That last renderer is the trick: the same event stream is simultaneously the **UI protocol**, the **storage format** and the **data source for every visualisation**. Replaying an old session is not a separate code path — it is the same array of events, read from a file instead of a socket.

One core, five faces.

spectro-core is a container-free Java 21 library that emits one typed RunEvent stream. Every face is a renderer of that stream; the stream is also the storage format and the graph source.



The big picture: the container-free core, the RunEvent stream as the one wire, the five faces, the provider ports and the disk.

Additive forever

The JSONL wire format is shared byte-for-byte with the sibling TypeScript edition of the workshop. That makes the format a contract: new event types and new optional fields may be added, but nothing is ever renamed, removed or reinterpreted, and every consumer silently skips what it does not know. Sessions recorded in 2026 replay forever; sessions recorded by the other edition replay here (proven by a byte-for-byte round-trip test).

Tool inputs are model output

The course maxim that shapes every security decision in this book: whatever a tool receives as input was written by the model, and the model writes what it wants. So every file path passes a sandbox, every mutation passes a permission gate, network egress asks first, hooks can veto calls before the gate, and even the “always allow” button remembers its approval scoped to a prefix instead of a blanket. You will meet this maxim in almost every chapter.

The stack, in one breath

LANGUAGE	Java 21 — virtual threads + blocking style; no reactive frameworks. The stream type is a blocking <code>Iterable<RunEvent></code> , cancellation is a cooperative <code>CancelSignal</code> .
BUILD	Gradle 9.6.1, Kotlin DSL, version catalog, five JVM modules + two npm toolchains (Vite/React web UI, Electron shell).
LLM ACCESS	One narrow <code>LlmProvider</code> port; behind it Anthropic (official SDK, SSE), Ollama (NDJSON) and any OpenAI-compatible server (SSE) — switchable mid-session.
FRAMEWORKS	Spring <i>Framework</i> as a library in the core (RestClient + declarative HTTP interfaces); Spring <i>Boot</i> 3.5.3 only in <code>spectro-server</code> ; picocli in the CLI; React 19 + React Flow in the browser.

TESTS

678 JUnit + 383 vitest, all key-free: fake providers, scripted HTTP servers, process seams.

`./gradlew build` needs no API key and no network model.

Quick start

One executable starts everything. The `./spectro` launcher resolves a JDK, loads your `.env`, and dispatches to the face you ask for — you never have to touch the Gradle task zoo.

The launcher

```

$ ./spectro

◆ spectroscope

repl      interactive agent REPL — /voice push-to-talk, --speak reads aloud
          ./gradlew -q --console=plain :spectro-cli:run --args="..."
run       headless run: ./spectro run -p "task" [--json] [--image path] [--speak]
          ./gradlew -q --console=plain :spectro-cli:run --args="run -p ..."
cron      scheduler: list / status / run <id> (~/.spectro/jobs.json)
          ./gradlew -q --console=plain :spectro-cli:run --args="cron list"
sessions  list the stored JSONL sessions (~/.spectro/sessions)
          ./gradlew -q --console=plain :spectro-cli:run --args="sessions"
resume    continue a session by id: ./spectro resume <id>
          ./gradlew -q --console=plain :spectro-cli:run --args="--resume <id>"
doctor    check Java, config layers, provider reachability
          ./gradlew -q --console=plain :spectro-cli:run --args="doctor"
tour      guided feature tour — menu, tips, settings
          ./gradlew -q --console=plain tour
web       web face — Spring Boot server + browser UI on :8080
          ./gradlew -q --console=plain :spectro-server:bootRun
desktop   desktop face — Electron shell + managed Spring Boot process
          ./gradlew :spectro-server:bootJar && (cd spectro-desktop && npm start)
mcp-notes build the example MCP server (stdio) into a runnable dist
          ./gradlew -q --console=plain :spectro-mcp-notes:installDist

Extra arguments pass through: ./spectro repl --verbose · ./spectro run -p 'Say OK.' --json
Vision needs a vision model (attach via --image or the web composer); voice: bash scripts/setup-stt.sh
(in) and scripts/setup-tts.sh (out).
MCP: configure external servers in .spectro/settings.json ('mcpServers'); ./spectro mcp-notes builds the
example; in the REPL, /mcp lists them.

```

Before dispatching, the launcher fixes the two classic host problems for you:

- **JDK auto-resolution.** The modules compile with `--release 21`, but many machines default to an older `java`. If `JAVA_HOME` is not already a JDK 21+, the launcher probes `/usr/libexec/java_home`, the Homebrew `openjdk@21` keg (preferred: a stable versioned keg survives brew upgrades), newer kegs up to 25, and the Cellar globs — and exports the first hit. Verified up to JDK 26; a miss prints a note and continues.
- **.env loading for every face.** The gitignored `spectroscope/.env` holds API keys and provider switches. The launcher parses it (comments skipped, one quote layer stripped, empty values ignored so `KEY=` can never blank a shell export) and exports the pairs — deliberately including the `desktop` face, which spawns the server through Electron and would otherwise bypass Gradle's own injection.

The thirty-second start

```
cd spectroscope

# 1 - pick a model. Local (no key at all):
#   .env: SPECTRO_PROVIDER=ollama · SPECTRO_MODEL=qwen3 · SPECTRO_BASE_URL=http://localhost:11434
#   cloud: ANTHROPIC_API_KEY=sk-ant-...      (model defaults to claude-opus-4-8)

# 2 - check the environment
./spectro doctor

# 3 - go
./spectro web      # browser UI on http://localhost:8080
./spectro repl     # terminal REPL
./spectro run -p "Say OK." # headless one-shot
./spectro desktop  # the Electron app
```

What doctor tells you

`./spectro doctor` is the honest pre-flight: it probes what your configuration actually selects — it loads the config hierarchy with your flags, pings the provider you chose, connects to every configured MCP server, and write-tests the sessions directory. This is real output from the machine this guide was built on:

```
$ ./spectro doctor

◆ spectro doctor
  ✓ Java 25.0.2
  ✓ config: provider=ollama model=qwen3.5:27b permissionMode=ask autoApprove=2 rule(s)
    layers: user config absent · project settings present
  ✓ ollama at http://localhost:11434 (version 0.24.0)
    images: gemini - GEMINI_API_KEY not set; generate_image will return a readable error
  ✓ skills: 4 installed (brainstorming, test-driven-development, verification, writing-plans)
  ✓ hooks: 0 configured
  ✓ mcp: notes reachable at /Users/you/spectroscope/spectro/spectro-mcp-notes/build/install/spectro-mcp-
notes/bin/spectro-mcp-notes (2 tools)
    vision: local provider - attach images only with a vision model (e.g. ollama pull qwen3-vl); a text-
only model fails fast
  ✓ voice input: whisper-cli + ggml-small.bin ready (/voice)
    voice output: piper missing · voice missing - run bash scripts/setup-tts.sh to enable --speak
  ✓ sessions dir writable: /Users/you/.spectro/sessions (7 session(s))
  ✓ jobs.json: 0 job(s)

Everything looks good.
```

Green checks fail the exit code when broken; the indented dim lines are information, not failures — a missing image-generation key, for example, is deliberately not unhealthy (the tool will return a readable error instead). Chapter 4 documents all twelve checks.

The provider matrix

PROVIDER	TRANSPORT	NEEDS	DEFAULT MODEL
<code>anthropic</code>	official SDK, SSE streaming	<code>ANTHROPIC_API_KEY</code>	<code>claude-opus-4-8</code>
<code>ollama</code>	Spring RestClient, NDJSON	local Ollama on :11434	<code>qwen3</code>

openai

Spring RestClient, SSE

any OpenAI-compatible server (LM Studio, llama.cpp, vLLM);
key optional

local-model

Selection rides the settings hierarchy (chapter 30 has every key): defaults < environment

(SPECTRO_PROVIDER / SPECTRO_MODEL / SPECTRO_BASE_URL , usually from `./.env` — the BASE layer) <

`~/.spectro/settings.json` (user) < launch-dir `.spectro/settings.json` (deprecated) < workspace

`.spectro/settings.json` + `settings.local.json` < CLI flags — any settings file outranks the env. “Local Ollama for this checkout” is three uncommented lines in `.env` as long as no settings file overrides them — and in the web UI you can switch provider and model mid-session from the header.



Configuration precedence, low to high. Each layer only fills what the higher layers left unset.

The capability map

Everything the full build can do, on two pages. Each row names the chapter where the details live.

CAPABILITY	IN ONE SENTENCE	CH.
Terminal REPL	Interactive agent with tool cards, spinner, slash commands, voice input and spoken answers.	4
Headless runs	<code>spectro run -p "..."</code> — one prompt, exit codes for CI, NDJSON event output with <code>--json</code> .	4
Cron scheduling	Jobs in <code>~/.spectro/jobs.json</code> run unattended with desktop notifications and audit sessions.	19
Web chat	Streaming chat with markdown answers, collapsible reasoning, tool cards with gate chips, image attachments, push-to-talk.	5
Graph view	Every run as a BPMN-style flow overview or a dagre DAG — live and for any archived session.	6
Trace view	The Wireshark of the harness: every wire frame both directions, filters, Δt , LLM-direction column, three detail modes.	6
Right panel	Agents roster, the agent's live plan, the exact system context per agent, and a sandboxed file browser.	6
The Lab	A step-through debugger for agent runs: dam up the event stream, advance it click by click across three synchronized views.	7
Scenarios	Seven deterministic, compiled demo runs — no LLM, no key — for teaching every mechanism.	7
Sessions	Every run is a JSONL file: list, replay, resume live (with the context re-upload made visible), delete with a two-step guard.	8
Import	Replay foreign files: raw spectroscope JSONL verbatim, Claude Code transcripts through an adapter.	8
Design system	Three brand designs — spectro dark, spectro bright, spectro white — switchable in place, particle/scroll effects, one-step persistence.	9
Desktop app	Electron shell that spawns and supervises the server JVM: health checks, tray, native cron notifications.	10
DE/EN chrome	The whole UI chrome switches between German and English live; content keeps its language.	11
Tool belt	22 built-in tools — files, shell, grep/glob, web fetch + search, page browse, image + document view, plan, skills, spawns — plus every connected MCP tool.	12
Permission system	ask/auto/readonly modes, a blocking broker, prefix-scoped allowlist, “always allow” with optional persistence.	13
Hooks	Config-driven shell guards around every tool call: <code>pre_tool_use</code> can block before the gate, <code>post_tool_use</code> observes.	13
Subagents	Explore/worker children with their own context, visible A2A task/status/result protocol, four development role tools.	14
Skills	<code>SKILL.md</code> packages with progressive disclosure — catalog in the prompt, body on demand.	15

Provider switch	Change LLM backend and model mid-session from the header; history intact; refused cleanly without a key.	16
Resilience	Transient-error retry with backoff (spectroscope owns it, SDK retry off) and Anthropic prompt caching with an honest compaction trigger.	16
Vision	Attach images in the composer or <code>--image</code> ; blobs stay next to the session file, prompts stay small.	17
Voice in/out	Local whisper.cpp push-to-talk (editable transcript) and piper speech output that reads answers while they stream.	17
Image generation	<code>generate_image</code> behind its own provider port (Gemini / OpenAI), content-addressed store, gallery panel.	17
MCP client	External tool servers over stdio or HTTP/SSE, Claude-Desktop-shaped config, at-most-once calls, example notes server included.	18
Context management	Automatic compaction at 100k tokens, forced <code>/compact</code> , live context ring with per-part introspection.	28
Wire tracing	<code>--verbose</code> mirrors the full agent↔provider protocol to stderr; the Trace tab does it in the browser.	4, 6



PART II

The faces

Five renderers of one event stream: the terminal, the web UI with its tabs and panels, the step-through Lab, the session archive, the design system, and the desktop shell.

- 4 The command line
- 5 The web face: Chat
- 6 The web face: tabs & panels
- 7 The Lab
- 8 Sessions: replay, resume, delete, import
- 9 The design system
- 10 The desktop face
- 11 Two languages

The command line

The CLI is the first face the workshop builds and the most direct one: a picocli application whose root command is an interactive REPL, with subcommands for headless runs, scheduling and diagnostics. Everything it prints is a rendering of the same RunEvent stream that the web UI folds and the session store writes.

Commands and global flags

INVOCATION	WHAT IT DOES
spectroscope	the interactive REPL (root command, no subcommand)
spectro run -p "..."	headless single prompt (see below)
spectroscope cron [list status --once id]	the scheduler — chapter 19
spectro sessions	list stored sessions (a positional, not a subcommand)
spectroscope --resume <id>	continue a stored session in the REPL
spectro doctor	the environment check
./spectro tour	the interactive menu tour of all faces

Global flags on the root command apply to every entry point (subcommands reach them via picocli's @ParentCommand — so flags > env holds on run and doctor too):

FLAG	MEANING
--provider / --model / --base-url	override the configured backend for this invocation
--compaction-threshold <n>	compaction trigger in input tokens (default 100 000)
--resume <id>	rebuild the conversation from a session file and continue it
--verbose	mirror the agent↔provider wire protocol to stderr in cyan
--speak	read answers aloud while they stream (voice output, chapter 17)

The REPL

Startup prints a banner naming the provider and model (for Ollama it live-probes the server version), the session id and file path, and the essential key bindings. The prompt is a coral `>`. Each turn drives three consumers in order — the JSONL store, the speech renderer, the ANSI renderer — the CLI-side proof of “one stream, many renderers”.

```
◆ spectroscope full build
ollama · gpt-oss:20b · ollama 0.24.0 · images: gemini · skills: 4 · allowlist active
session 20260716-091500-3fa4b2c1 · ~/.spectro/sessions/20260716-091500-3fa4b2c1.jsonl
/help for commands · /mcp servers · /voice push-to-talk · /speak on|off reads answers aloud · Ctrl+C
aborts a run
```

Rendering rules worth knowing (all in `EventRenderer`):

- **Thinking is dimmed.** A reasoning model's `thinking_delta` stream prints dim under a `· thinking` label, visually apart from the answer.
- **Tool cards.** Each call prints `✖ name` (coral hammer) with a 100-char input preview; the result line shows `✓/✖ <ms> · output preview`.
- **Permissions inline.** A gated call prints a sand `run <tool>? [y/N]` prompt; only `y` approves. Allowlisted calls print `✓ auto-approved by allowlist` instead — the decision is still recorded in the stream.
- **Subagents in brackets.** Child events render as `[worker-1] ...` lines with line-buffered text so parallel children stay readable; A2A task/status/result messages appear as one dim line each.
- **The plan pretty-print.** An `update_plan` call renders `◇ plan (N steps)` with `[x]` done, coral `[~]` running, `[ ]` open.
- **Degradation.** Colors and the braille spinner appear only on a real TTY (and never with `NO_COLOR` or `TERM=dumb`); piped output is plain text.

Slash commands

Handled entirely in the CLI — none of these ever reaches the model:

COMMAND	BEHAVIOUR
<code>/help</code>	lists all twelve commands with one-liners
<code>/cost</code>	cumulative session tokens: <code>Session usage: X in / Y out</code>
<code>/model</code>	current provider · model (+ base URL for local providers)
<code>/sessions</code>	the stored-session listing, same as <code>spectro sessions</code>
<code>/skills</code>	installed skills with descriptions (chapter 15)
<code>/mcp</code>	connected MCP servers, reachability, and every <code>mcp_...</code> tool (chapter 18)
<code>/think on off</code>	toggles reasoning visibility; rebuilds the agent and reloads the session history so nothing is lost
<code>/voice</code>	push-to-talk: record → transcribe → editable confirmation (chapter 17)
<code>/speak on off</code>	voice output at runtime; <code>off</code> also stops the current sentence
<code>/compact</code>	force a context compaction now (<code>Agent.compactNow()</code>); prints the result or “Nothing to compact”
<code>/clear</code>	fresh agent, fresh session file: <code>New session: <id></code>
<code>/exit</code>	leave (empty line or Ctrl-D work too)

Headless runs: `spectro run`

The automation face. One prompt, one process, honest exit codes:

```
# plain: prints only the final answer text
./spectro run -p "Count the Java files under spectro-core."

# NDJSON: every RunEvent on stdout, one per line – byte-identical to the JSONL format
./spectro run -p "Say OK." --json | jq -c '{type}'

# guard rails
./spectro run -p "..." --max-turns 5 --permissions readonly
```

FLAG	MEANING
<code>-p, --prompt</code>	required task text
<code>--json</code>	NDJSON events on stdout; ALL diagnostics go to stderr, so <code> jq</code> pipelines stay clean
<code>--max-turns <n></code>	cancel when a turn exceeds the limit (stop reason <code>max_turns</code>)
<code>--permissions readonly auto</code>	the headless policy. Default readonly — “capability before convenience”: unattended runs deny mutations unless you opt in
<code>--image <path></code>	attach an image (repeatable; jpg/jpeg/png/webp/gif) — vision, chapter 17
<code>--verbose</code> / <code>--speak</code>	as on the REPL

Exit code contract: 0 only when the run ended with a regular `end_turn` . Anything else (aborted, max turns, error) exits 1 and names the stop reason and the session id on stderr — the session file is always there for post-mortem inspection.

The twelve doctor checks

#	CHECK	WHAT IT VERIFIES
1	Java runtime	runtime major version $\geq$ 21
2	Config hierarchy	the layered config loads; shows provider/model/permissionMode/allowlist count and which layers exist. A broken config stops doctor immediately — nothing below would make sense
3	Provider reachability	anthropic: key present · ollama: live version probe · openai: GET <code>/v1/models</code> (2 s/3 s timeouts)
4	Image provider	key for gemini/openai; a missing key is informational, not unhealthy
5	Skills	count + names of installed skills
6	Hooks	count + <code>event:matcher</code> pairs
7	MCP servers	connects each configured server: reachable + tool count, or a red UNREACHABLE line (never a crash)
8	Vision	informational hint matching your provider (vision model needed locally)
9	Voice input	<code>whisper-cli</code> on PATH + the ggml model file present
10	Voice output	piper binary + voice model present
11	Sessions dir	creates the dir, writes and deletes a probe file, counts sessions
12	jobs.json	the cron file parses (loudly invalid otherwise)

Wire tracing with `--verbose`

A decorator around the provider mirrors the whole protocol to **stderr** in cyan: the full outgoing request before each call (message count, tool names, clipped system prompt, every content block) and each incoming provider event as it streams (`← text_delta "..."` , `← tool_call name {...}` ,

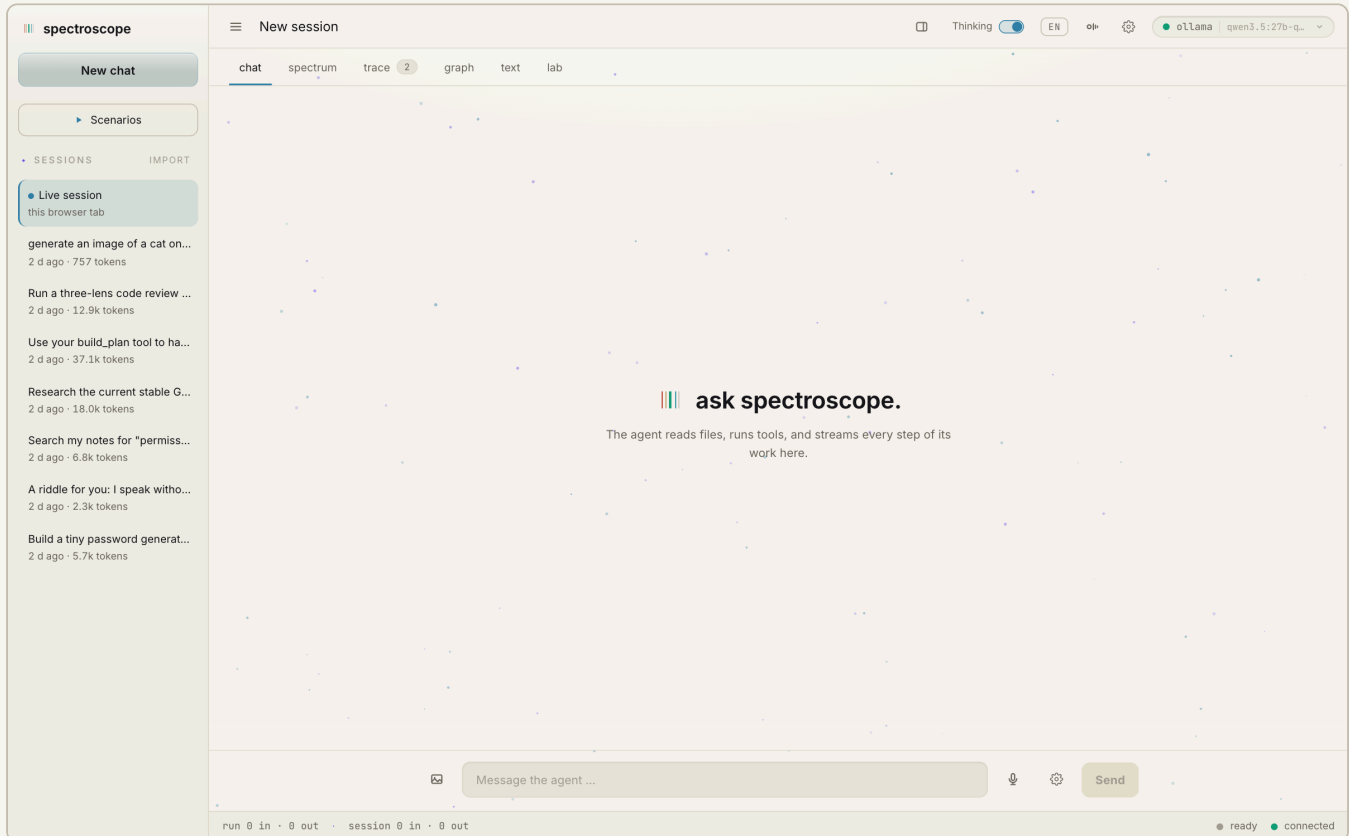
`← usage 123 in / 45 out , ← stop end_turn`). Because it writes to stderr, `spectro run --json | jq` keeps a clean stdout. In the REPL the traced instance is shared with subagents — you see the children's requests too.

The tour

`./spectro tour` is a menu-driven guide around all faces: launch the REPL, list or resume sessions, do a headless run, drive cron, start the web jar, run doctor — plus a settings screen that captures a provider choice and API key for the session (hidden input, optional save to `./.env` with mode 600) and stage-appropriate “try this” tips. The same file ships in every workshop stage and detects at runtime which features exist; in the full build it shows everything.

The web face: Chat

One Spring Boot jar serves everything: the built React UI, a handful of REST endpoints, and a single WebSocket that carries RunEvents in one direction and a small set of client frames in the other. Open `http://localhost:8080` after `./spectro web` and you are looking at the chat.



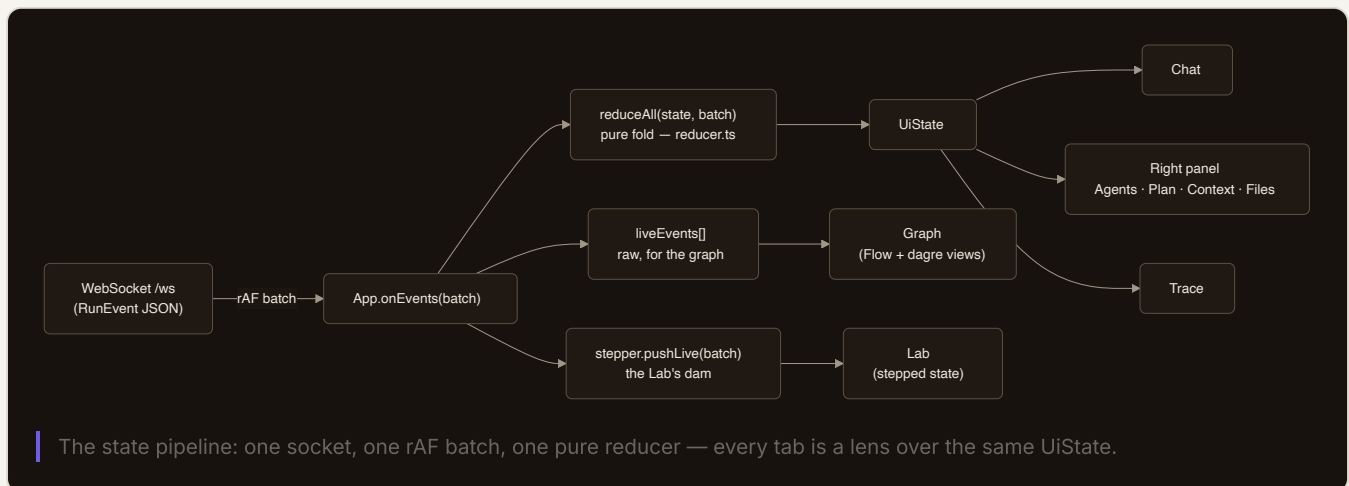
The empty app: sidebar with the session archive, header with thinking toggle · language toggle · design drawer · provider chip, the six tabs, and the composer waiting. “Ask spectroscope.”

The anatomy of the screen

- **Sidebar** (left, resizable, collapsible): “New chat”, the **Scenarios** button (chapter 7), the session archive with an Import button (chapter 8). The first row is always the live session of this browser tab.
- **Header**: session title (the first prompt), image-gallery toggle (appears once images exist), right-panel toggle, the **Thinking** switch, the **DE/EN** toggle, the design drawer, the **provider picker** chip, the **context ring** (appears once tokens flow) and the Stop button while a run is active.
- **Tab bar**: chat · spectrum · trace · graph · text · lab — six lowercase lenses over the same state (the labels are wire vocabulary; trace carries a live frame count).
- **Footer**: run and session token totals in tabular numerals, a run status dot, and the connection dot.

Behind the scenes, every incoming socket frame is batched once per animation frame (a `text_delta` flood costs one React render per frame, not per token) and folded by one pure reducer into an immutable `UiState`. Live stream and archived replay produce the same state shape from the same code — which is

why every view in this chapter works identically for both.



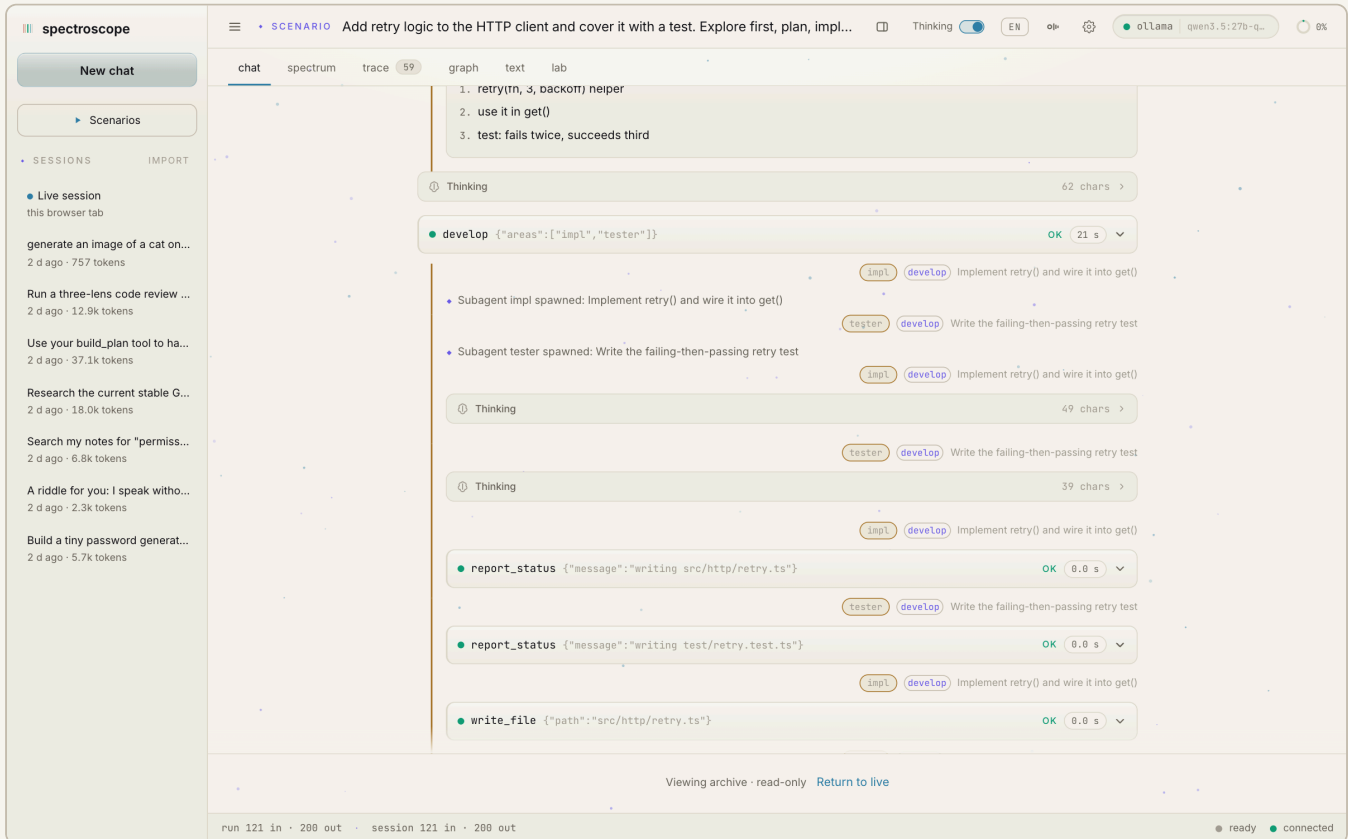
The conversation

The screenshot shows the Spectroscope application interface. On the left is a sidebar with a "New chat" button and a list of sessions. The main area displays a conversation with an AI assistant. The user prompt is: "Add retry logic to the HTTP client and cover it with a test. Explore first, plan, impl...". The assistant's response includes a "Thinking" disclosure, followed by two "read_file" tool calls, another "Thinking" disclosure, and a "build_plan" tool call. The "build_plan" tool call returns a plan with three steps: 1. retry(fn, 3, backoff) helper, 2. use it in get(), and 3. test: fails twice, succeeds third. The interface also shows a "Right panel" with tabs for Agents, Plan, Context, and Files. At the bottom, there is a status bar indicating "run 121 in · 200 out" and "session 121 in · 200 out".

A full conversation (here: the deterministic “coding” scenario): user prompt, collapsible Thinking disclosure, tool cards with gate chips, markdown answer cards.

- **Markdown answers.** Assistant turns render through a dependency-free markdown parser (headings, streaming-safe fenced code with language chip and copy button, nested lists, GFM tables, quotes). Nothing ever becomes raw HTML, and link protocols are vetted — `javascript:` renders as plain text. Model output is untrusted end to end.
- **Thinking, apart from the answer.** While a reasoning model thinks, a pulsing “thinking ...” dot is the live indicator; the finished reasoning collapses into a disclosure above the answer showing its size. The header switch turns the stream off for the next run (`set_thinking`).

- **Tool cards.** One collapsed card per call: status dot + name + input preview + **gate chip** + duration. The gate chip is the didactic point — *Gate: allowed / denied / waiting* appears only for permission-gated calls, so the three control paths (free tool, gated tool, hook-blocked call) read differently at a glance. Expanding shows full input and output JSON with copy buttons.
- **Subagent threads.** Consecutive turns of one child nest under a pastel-edged thread block (header: agent id, role label, task). Interleaving splits threads honestly — the Trace keeps the flat truth. More in chapter 14.
- **Info and error lines.** Spawns and compactions appear as quiet info lines (localized live); a failed run gets an error card with a “Send again” link.



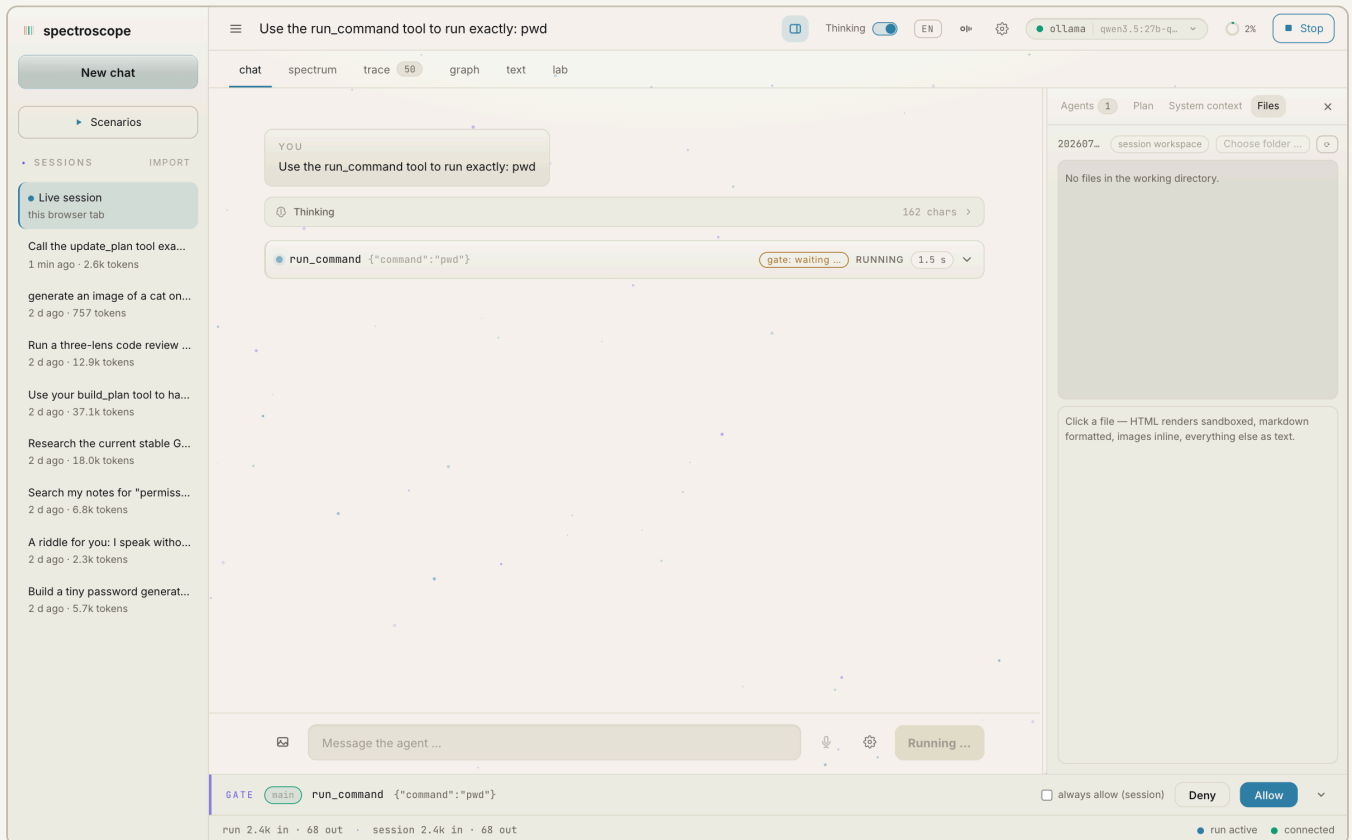
Subagent threads: two parallel workers from a fan-out, each with its own pastel-edged thread, status lines and tool cards; the A2A result closes each thread.

The composer

- **Enter sends, Shift+Enter breaks the line** — the only shortcut. The textarea autosizes up to 150 px; the Send button reads “Running ...” while a run is active (one run at a time per connection).
- **Image attachments.** The image button opens a file picker (jpeg/png/webp/gif, multiple); dragging files anywhere onto the chat works too. Every image is downscaled in the browser before sending (longest edge 1568 px, JPEG re-encode when useful; GIFs pass through to keep animation), previewed as removable chips, and never leaves the page until you hit Send. Attachments cross the socket as base64, land in the session's blob store, and the events carry only references.
- **Push-to-talk.** The mic button records (coral pulsing dot + `m:ss` timer), a second press stops; the audio is transcribed server-side by local whisper.cpp and the text lands *in the input field* — an STT error is reviewable text, never an agent instruction. If STT is not installed the button disables itself with a setup hint. Chapter 17 has the setup.

The permission dialog

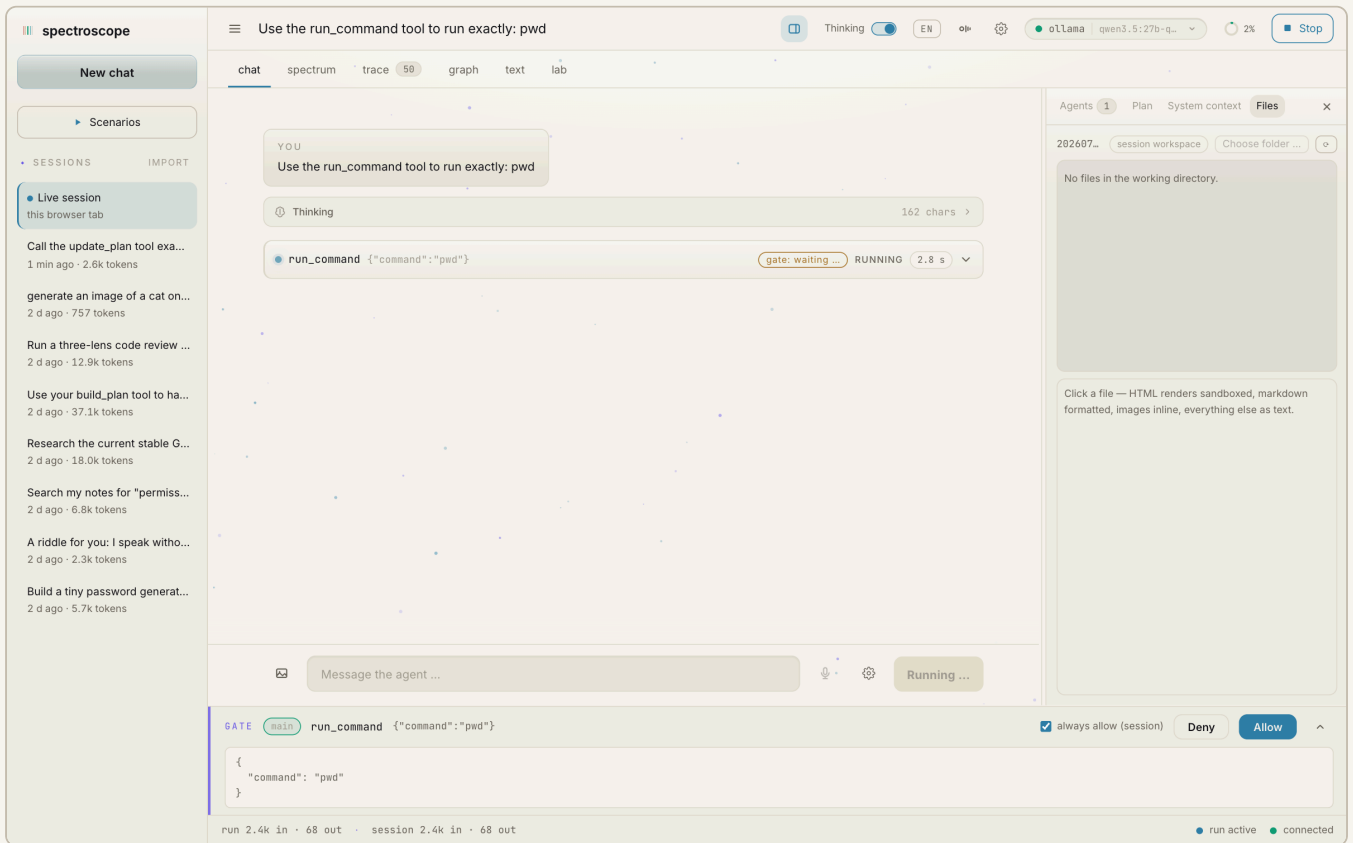
When the agent wants to run a gated tool and no allowlist rule covers it, the run genuinely pauses — the agent's virtual thread parks on a future server-side — and the dialog takes over:



The serious moment: run_command wants to execute `uname -a`. The input is shown in full — you approve what you see. Deny is the focused default; Escape denies.

- The input is shown **in full** as pretty JSON — you approve what you see, never a summary.
- **Deny is the safe default:** it holds focus, Escape denies, Tab is trapped, and clicking the backdrop does nothing — a decision must be deliberate.
- When a subagent asks, the dialog names it ("requested by worker-1").
- Multiple pending requests queue with a counter ("1 of 3").

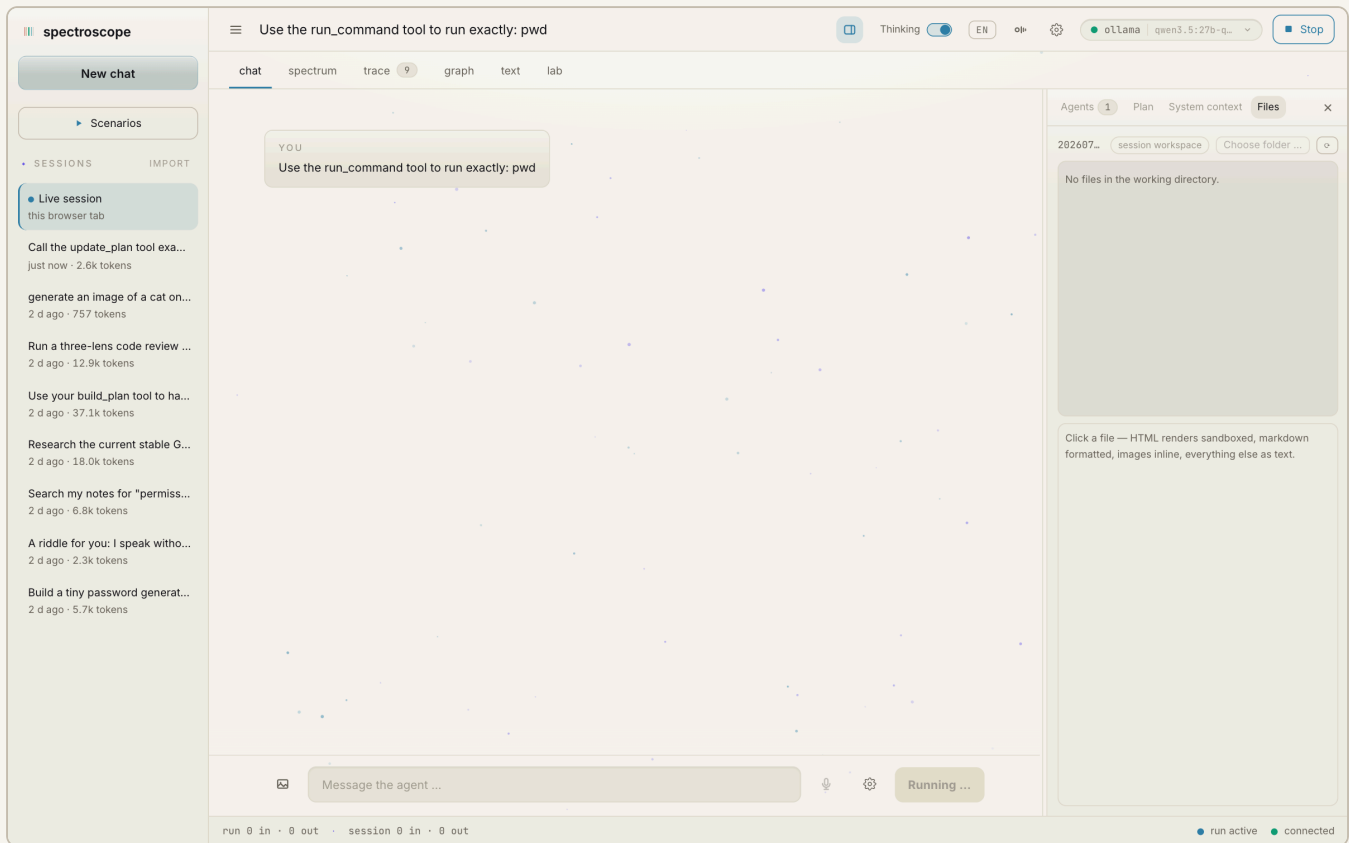
The two checkboxes appear once you tick the first:



“Always allow: run_command (this session)” — and, gated behind it, “Persist (project settings, .spectro/settings.json)”. Only Allow carries the flags.

Always allow (this session) remembers the approval for this browser tab only. **Persist** appends the rule to the project's `.spectro/settings.json` — the same `autoApprove` list the CLI reads, so a web decision is honored by the terminal on its next run. Remembered rules are **prefix-scoped** for risky tools (a `git status` approval becomes `run_command:git*`, never “all commands”) — chapter 13 has the exact rules. A denied call returns `ERROR: the user denied the execution.` to the model, which reads it and adapts.

Live indicators



A live run against local Ollama: the model's reasoning streams into the pulsing Thinking disclosure while the header shows the provider chip and the context ring.

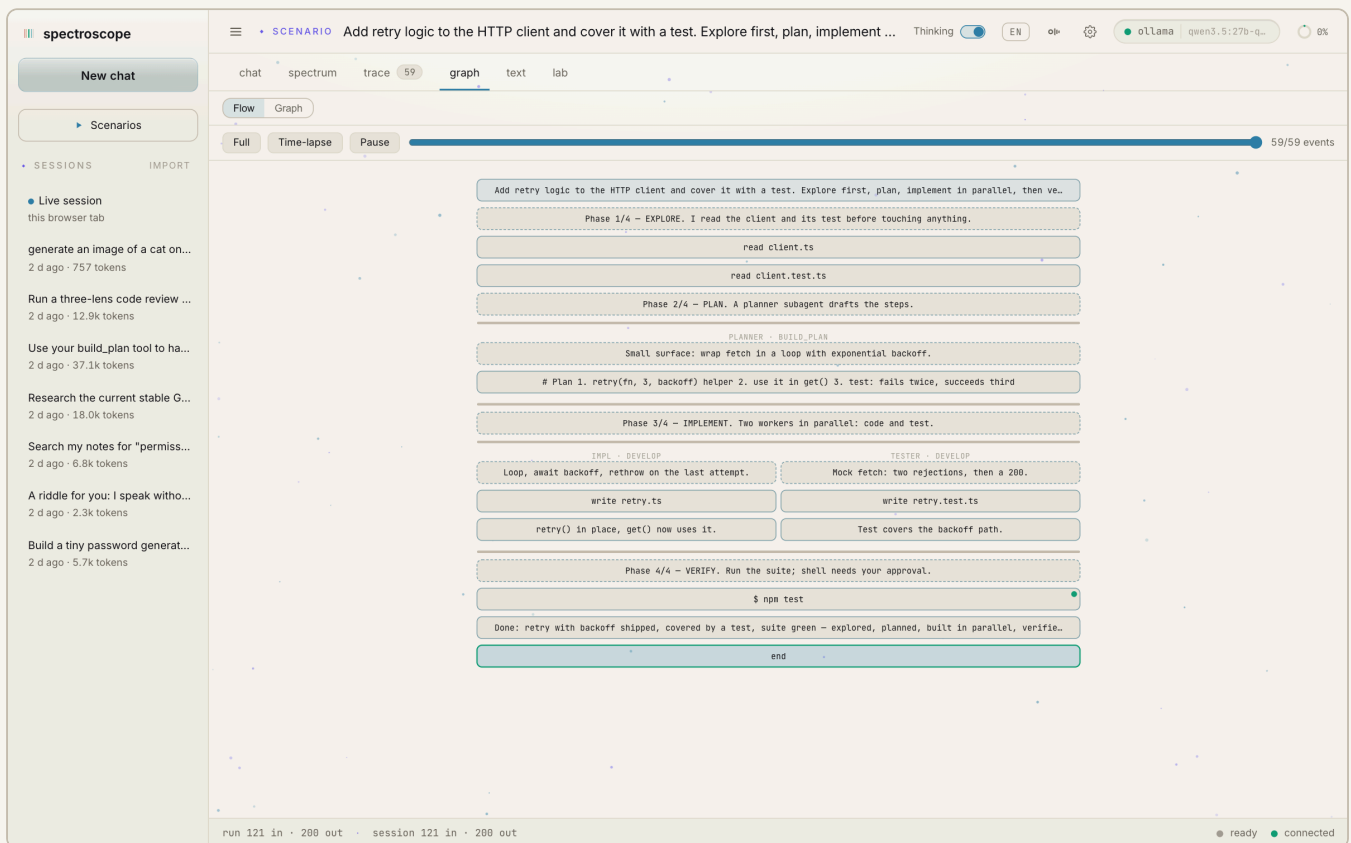
- The **context ring** in the header fills with the last reported input tokens against the compaction threshold (green < 70%, amber ≤ 90%, red above); clicking it opens the introspection popover — chapter 28.
- The **usage footer** counts run and session tokens live in tabular numerals; the status dot switches “run active → ready” and names any non-regular stop reason.
- The **connection banner** appears under the header when the socket drops, with a reconnect countdown (exponential backoff 1→15 s) and a “Reconnect now” action. Sessions survive: the JSONL file is the state, the socket is just transport.
- The **Stop button** sends `abort`, which fires the run's CancelSignal — the run ends with `stopReason: "aborted"`, tools and children included.

The web face: tabs & panels

Chat is one lens. The Graph tab shows the same run as a process diagram, the Trace tab as wire frames, and the right-docked panel adds the agents roster, the live plan, the exact system context and a sandboxed file browser.

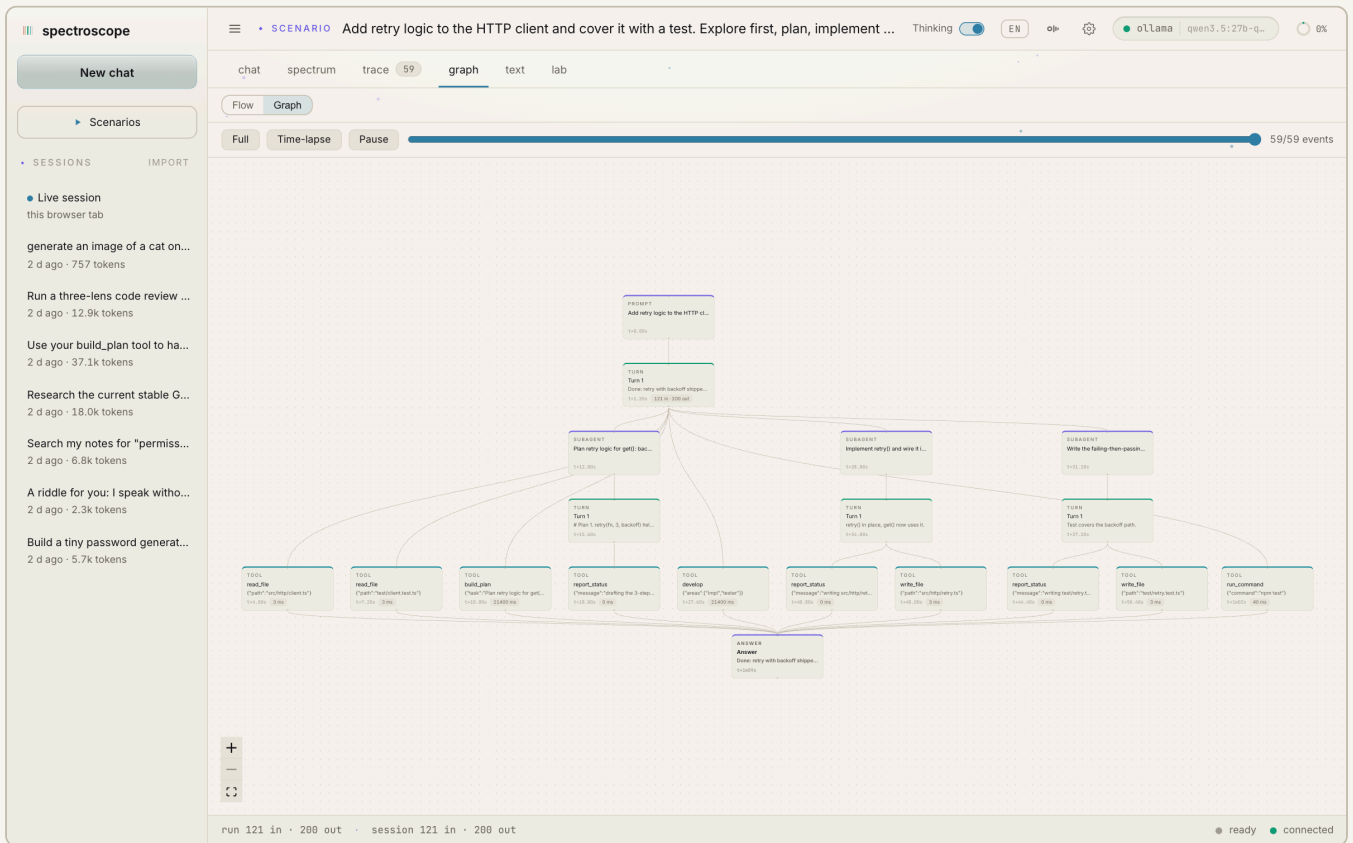
The Graph tab

Two views behind a *Flow* / *Graph* toggle (sticky per browser). For archived sessions a replay bar adds *Full*, *Time-lapse* (advances along the real timestamp gaps, accelerated), *Pause* and a scrubber.



Flow view: the whole session as a BPMN-style overview — activities on the main spine, one parallel split/join block per subagent wave, gate outcomes as dots.

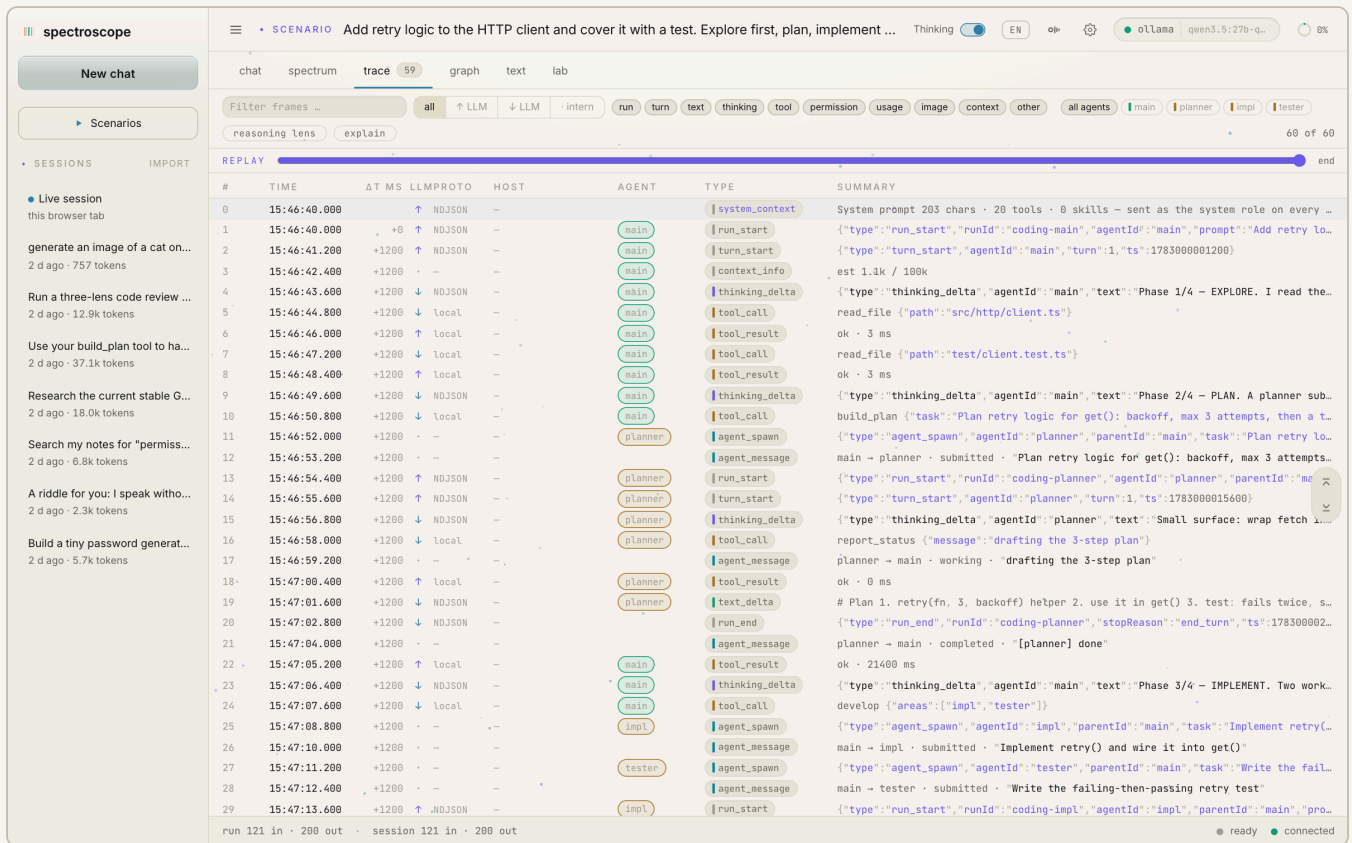
Flow is the minimap for humans: rounded activities (*user* / *think* / *tool* / *say* / *compact* / *end*) stack on the main spine; each subagent wave becomes a split block with one column per child and a join bar. Think and say activities accumulate their streamed text, so real runs get sentence labels instead of “token one”. Tool activities label themselves honestly — `read <file>`, `$ <command>`, `fetch <url>`, `server · tool` for MCP. Gate outcomes render as small dots (pending / allowed / denied). In a replay, clicking an activity seeks the timeline to its first event.



Graph view: the dagre DAG — prompt, turns, tools, subagent lanes and the answer; every node carries timing, token badges and status.

Graph is the classic DAG (React Flow + dagre): one node per prompt, turn, tool call, subagent and answer, with **t+** relative times, duration and token badges, error badges, and a pulsing outline for running nodes. Sequentially spawned subagent subtrees get their own side-by-side lanes. Clicking a node opens a detail panel with its timing line and its *raw evidence* — exactly the events that formed the node, each as a collapsible JSON tree. The right mouse button pans; the left button only clicks — on both React Flow canvases in the app.

The Trace tab



The wire view: every frame in both directions with time, Δt, LLM direction, wire protocol, agent, type and a token-highlighted summary. The tab badge counts frames live; the slim rail on the right jumps to the first or the newest frame.

“What Wireshark is to packets, this is to the harness protocol.” Every frame lands here — RunEvents inbound, client messages outbound — before the reducer even looks at it, so unknown event types are visible too.

- **Columns:** sequence · wall-clock time with milliseconds · Δt since the previous visible row · **LLM direction** (↑ going to the model with the next request, ↓ produced by the model, · internal) · **proto** · **host** · agent · type · summary.
- **The proto column** names the wire each payload actually rides — the protocol breakdown as a column: `SSE` for the cloud LLM streams (Claude and OpenAI-compatible), `NDJSON` for Ollama, `JSON-RPC` for `mcp_*` tool rows (a result resolves its tool through the callId), `HTTP` for `web_fetch` and image generation, `local` for the file tools, and `-` for everything harness-internal (the gate, the plan, introspection, A2A) that never leaves the process.
- **The host column** names the network counterpart: LLM rows show where the request really goes (`api.anthropic.com`, `localhost:11434` ...) — live from the socket-only `provider_info` frame the server sends on connect and after every provider switch, so a switch is a visible trace row, never a silent client-side swap. MCP rows name their server, `web_fetch` the fetched URL's host, image events their backend endpoint; replays only know the provider, so anthropic maps to its fixed endpoint and local backends show an honest `-`.
- **Filters:** free-text (matches type, agent and payload), an LLM-direction segment, and toggleable category chips (run, turn, text, tool, permission, usage, image, context, other).
- **Summaries with token highlighting:** the model's own words in sand, punctuation faint, numbers tabular, the literal `ERROR` red — lossless, the full payload is one click away (collapsible JSON tree + copy).

- **The synthetic `system_context` row:** the system prompt rides with every request but is not a wire event — the view prepends one synthetic ↑ row (from `GET /api/context`) naming prompt size, tools, skills and MCP servers. Display-only: never in the reducer, never in the JSONL.
- **Reading flow:** the tab **opens at the first frame** — the story reads from the beginning. While a frame is open, **Space steps to the next visible one** (it opens, takes focus and centres itself; Enter still toggles a focused row), so the trace taps through like the Lab stepper. A slim **jump rail** on the right edge scrolls to the first or the newest frame.
- **Auto-follow** engages once you scroll down to the live end; while you are reading further up, arrivals accumulate into a "{n} new ↓" pill instead of yanking the view.
- **Three detail modes** per row: *Insight* (the JSON tree), *Compact* (one token-highlighted line per real wire line, horizontally scrollable) and *Raw* (plain text). The `session_resume` marker's detail carries the entire re-uploaded history as its JSONL lines — chapter 8.

The Text tab

Another lens on the same stream — where the trace shows frames, this shows the *text*. Two views behind a *Text / JSONL* toggle (sticky per browser), one copy button for the whole thing:

- **Text** is the whole session as one readable feed, with the protocol made visible as literal markers: every reasoning run sits between `<think>` and `</think>` — exactly the tags Ollama really streams inline; Anthropic's thinking blocks render to the same boundaries — followed by the answer text in plain type. Tool use appears as honest indicators with nothing hidden: `[tool_call read_file {"path":"..."}]`, then `[tool_result read_file · 9ms]` with the *full* output underneath. Run boundaries (`[run_start ollama]`, `[run_end end_turn]`), the permission gate, spawns, A2A messages, compaction and errors all keep their place in reading order; subagent lines carry their agent id as a prefix. Prompts render in sand, thinking dimmed italic, markers faint — but it is all ONE selectable text, and the copy button hands it out as plain text.
- **JSONL** is the session exactly as the file on disk stores it: one compact JSON line per wire event, newline per event — the NDJSON view of the truth. Socket-only UI frames (`provider_info`, `workspace_info`) are deliberately absent: they never enter the file, and this view IS the file.

The right panel

The header's panel button docks a resizable panel into the Chat tab with four tabs.

Agents

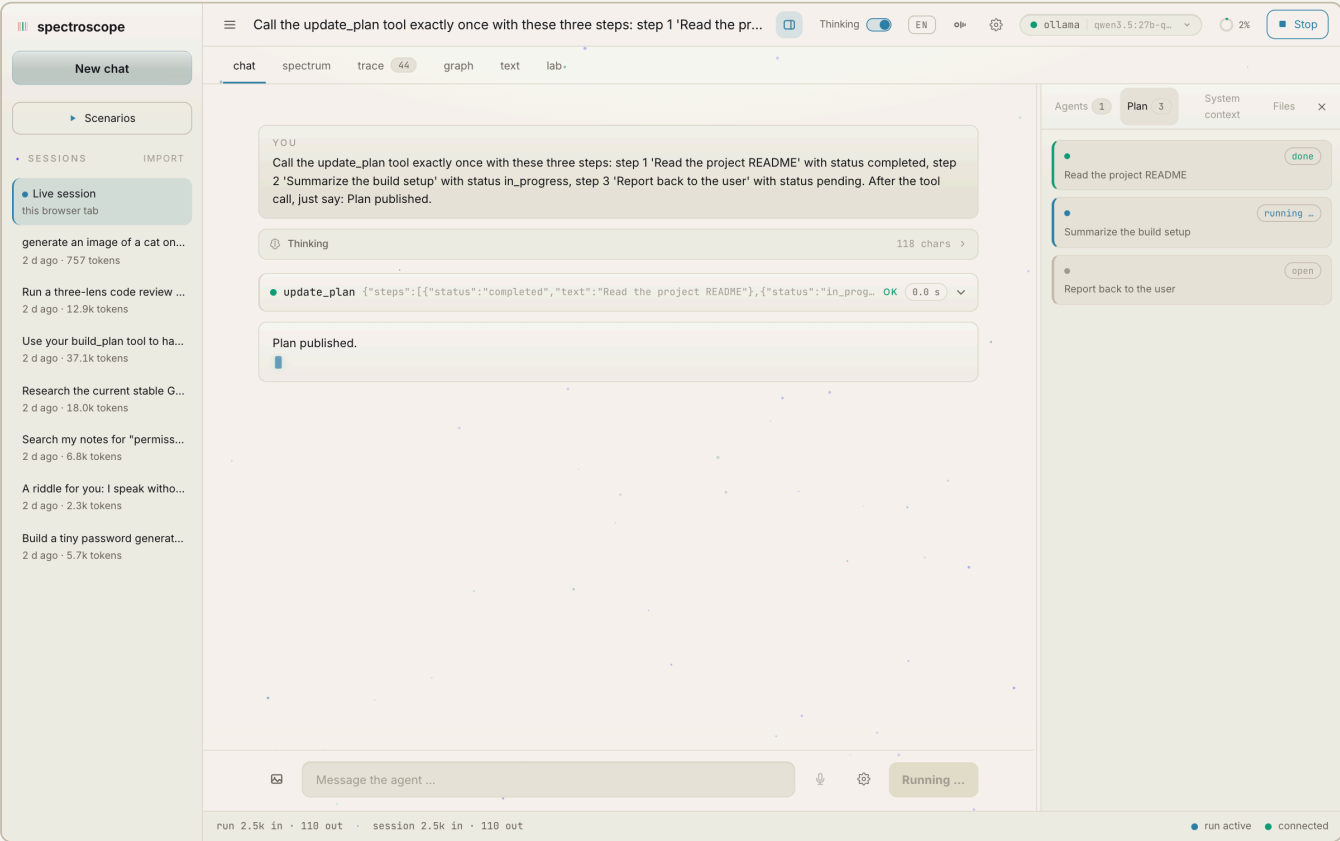
The screenshot displays the Spectroscope interface for a scenario titled "Add retry logic to the HTTP client and cover it with a test. Explore first, plan, impl...". The interface is divided into several sections:

- Left Sidebar:** Contains a "New chat" button, a "Scenarios" section, and a list of sessions with their respective token costs and timestamps.
- Top Bar:** Shows the scenario title, a "Thinking" status, and a language selector set to "EN".
- Central Panel:** Displays the execution flow of the scenario. It includes a sequence of tasks and their results, such as "report_status", "write_file", and "run_command". Each task is associated with a lifecycle badge (e.g., "impl", "develop", "tester") and a status (e.g., "OK", "gate: allowed").
- Right Panel:** Shows a list of agents involved in the scenario, including "main", "build_plan", "planner", "develop", "impl", and "tester". Each agent card displays its lifecycle badge, task, latest status line, and token cost.

The session-wide roster: main plus every subagent that ever ran, with lifecycle badge, task, latest status line and token cost. It survives run ends — only a new chat clears it.

One card per agent — main first, then every child in spawn order — with its state dot and lifecycle badge (*submitted / working / done / failed*), its task, the latest `report_status` line, and its token cost (`1.2k in · 0.3k out`). Selecting a card jumps to the System context tab for that agent.

Plan



The agent's published plan, live: three steps in the three states (done / running... / open). This capture is from a real local-model run — note the first update_plan attempt errored on a wrong field name and the model self-corrected on the second call.

The read-only, latest-wins snapshot of the agent's plan, fed by the permission-free `update_plan` tool: each step with a status dot and badge — *done*, coral *running ...*, *open*. The wire statuses stay canonical English (`pending` / `in_progress` / `completed` , enforced at the write boundary); only the labels localize. The tab badge counts steps; the CLI pretty-prints the same event.

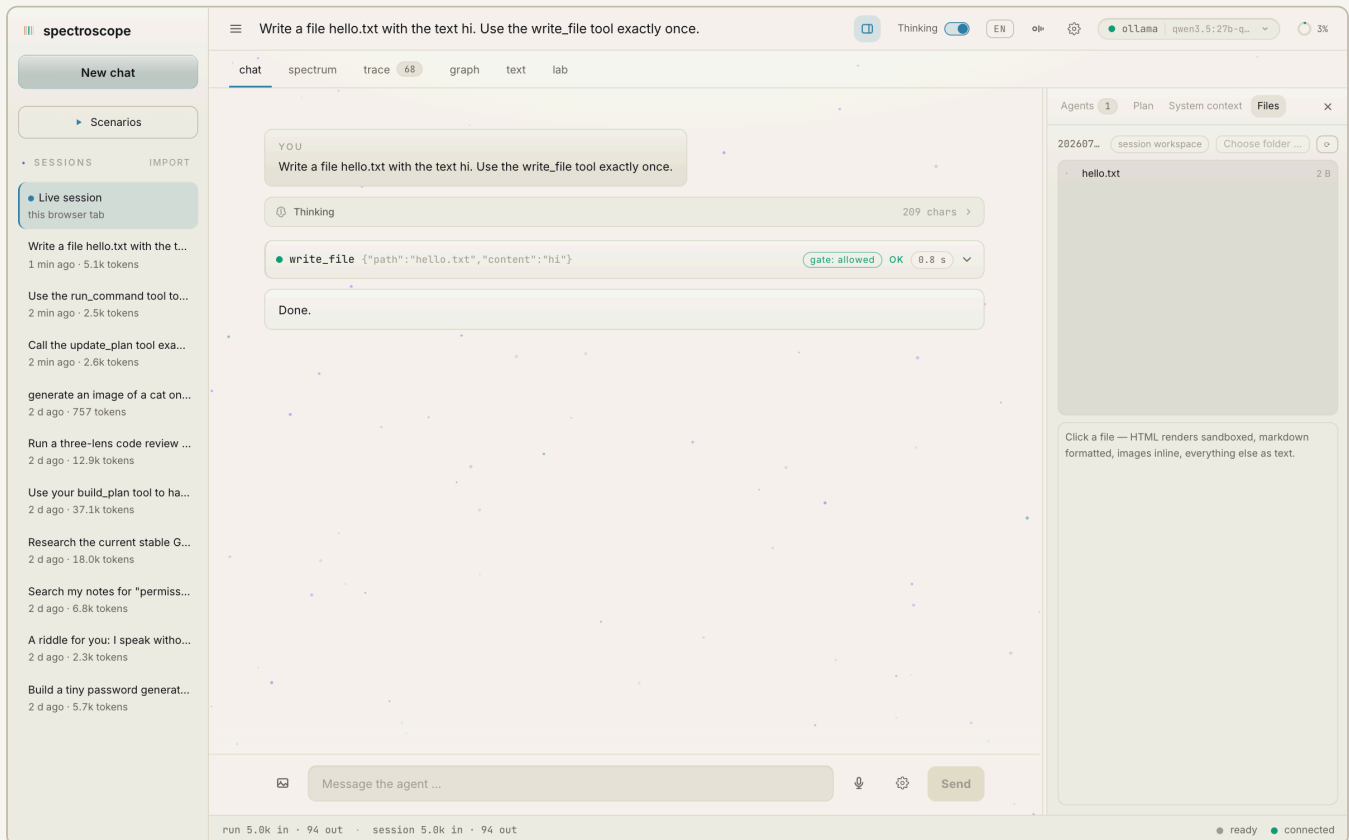
System context

The screenshot displays the Spectroscope AI interface. On the left, a sidebar shows a 'New chat' button and a list of sessions. The main chat area is titled 'SCENARIO Add retry logic to the HTTP client and cover it with a test. Explore first, plan, impl...'. It features a timeline of actions: 'report_status' (writing src/http/retry.ts), 'write_file' (src/http/retry.ts), 'write_file' (test/retry.test.ts), and 'run_command' (npm test). Each action is accompanied by a status indicator (OK) and a duration (0.0 s). The right-hand panel, titled 'System context', shows the system prompt, model information (ollama, qwen3.5:72b-q4_K_M), and a list of tools (list_dir, read_file, write_file, run_command, edit_file, glob, grep) with gate markers.

“What goes to the LLM before any user message”: the real system prompt (markdown-rendered), model + thinking state, all tools with gate markers, skills, MCP servers and the subagent role profiles.

The transparency panel: exactly what the LLM receives before any user message. For the **main agent**: the full system prompt (working directory, project `SPECTRO0.md` context, the skills catalog), provider/model/thinking, every tool in registration order (gate chip = needs your approval), skills, MCP servers, and the six subagent role profiles. For a **selected subagent**: its role prompt, its task, its reduced tool set — and the note that children never get the spawn tools (nesting ends at depth 1). The data comes from the stateless `GET /api/context`, assembled from the same constants the live tools use — what the panel shows is what the model gets. A fullscreen *Raw* modal shows the whole context untruncated with a character count and copy button.

Files



The Files tab is the agent's desk: the workspace this session runs in, refreshed live — hello.txt, which the agent wrote through an allowed gate seconds ago, is right there.

A read-only view of the agent's **workspace** — the same sandbox the file tools see. Every session gets its own folder (`<tmpdir>/spectroscope-ws/<session-id>` unless a `workspace` is configured — chapter 30), the server announces it over the socket (`workspace_info` , visible in the trace), and the tab follows it live: what the agent writes appears here, and the repo you started spectroscope in stays clean. The panel **opens itself** on the Files tab when the announcement arrives — the agent's desk appears where its files land. And you can **pick the desk per session**: the “Choose folder ...” button opens the *native* folder dialog on the spectroscope machine (`POST /api/pick-workspace` , macOS; a browser cannot hand out absolute paths, but spectroscope runs locally), and the picked path travels back as the additive `set_workspace` socket message. Only before the first run — once the agent ran, the sandbox and every subagent are anchored, the button locks and a new chat is the way to a different folder. The pin is shared server state, so the tree you browse and the sandbox the tools see are always the same folder, and a resume in the same server process finds the picked folder again. And the browser **remembers your pick**: every new chat lands on the same desk with zero clicks (a dashed “📁 folder” chip shows the memory; clicking it forgets — the next chat gets its per-session temp folder again. Resumes are never touched). Hidden entries and build folders (`.git` , `build` , `node_modules` , `target` , `dist` , `out`) are skipped, depth and entry count are capped honestly (“list truncated” note). Clicking a file previews it by type: HTML runs in a **CSP-sandboxed iframe** (scripts execute in an opaque origin — no cookies, no API, no parent access), markdown renders through the shared component, images inline, everything else as text. Binary answers 415, oversized 413 — and the defense in depth is real: the `.env` with your API key answers 404 *even by direct URL*, because every path segment is re-checked server-side against the hidden/ignored rule.

The image gallery

When the agent generates images (chapter 17), a gallery column opens next to the chat: cards newest-first with the prompt as caption, provider and model badges, full-size link, and a provider dropdown (`gemini` / `openai`) that switches the image backend for the next generation. Bytes never cross the socket — cards load from `GET /api/images/<sha256>.<ext>`, where the content address doubles as the traversal guard (64 hex chars or 400, before any filesystem access).

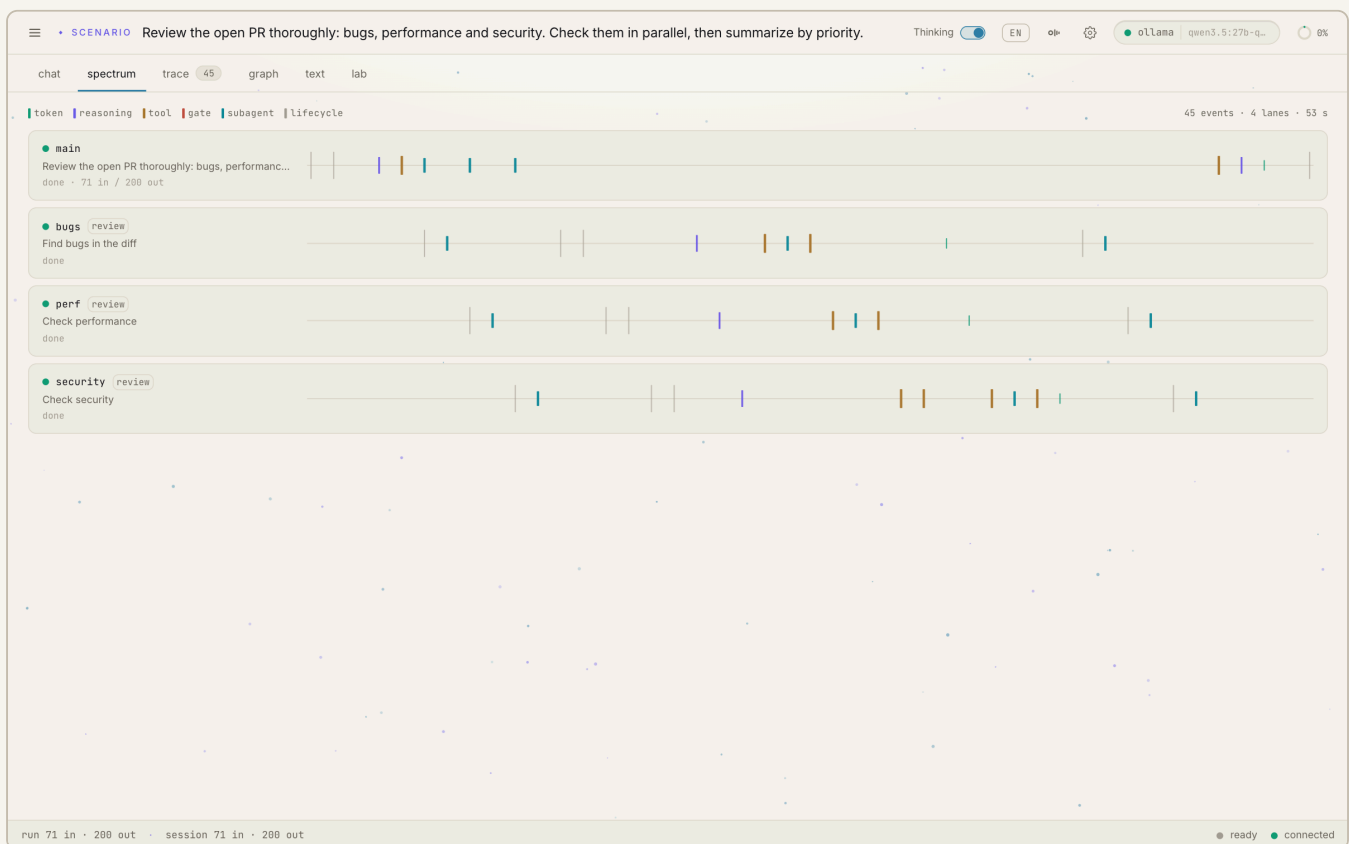
Each card also carries “→ **Workspace**”: it copies the image from the global store into *this session's workspace* — you pick the file name (keep the content-addressed original or type your own; a name without an extension inherits the original's), and the file appears in the Files tab, ready for the agent's tools. The endpoint keeps the house guards: only store names touch the disk, the target name is a single sanitized file name, and an existing file is never overwritten (409).

the spectrum and the new lenses

The design transplant (migration phase 4) brought the brand into the product and six new surfaces with it. All of them read the same event stream you already know; none of them adds a second source of truth.

the spectrum tab

The spectrum is the fleet on one screen. Every agent draws one horizontal lane; every event is a discrete mark on it, colored by type: token teal, reasoning violet, tool amber, gate red, subagent ocean, lifecycle faint. The lane rail carries the agent id, its task, its lifecycle state and its token totals. A pending permission turns the lane's gate mark violet and pulses until the decision lands.

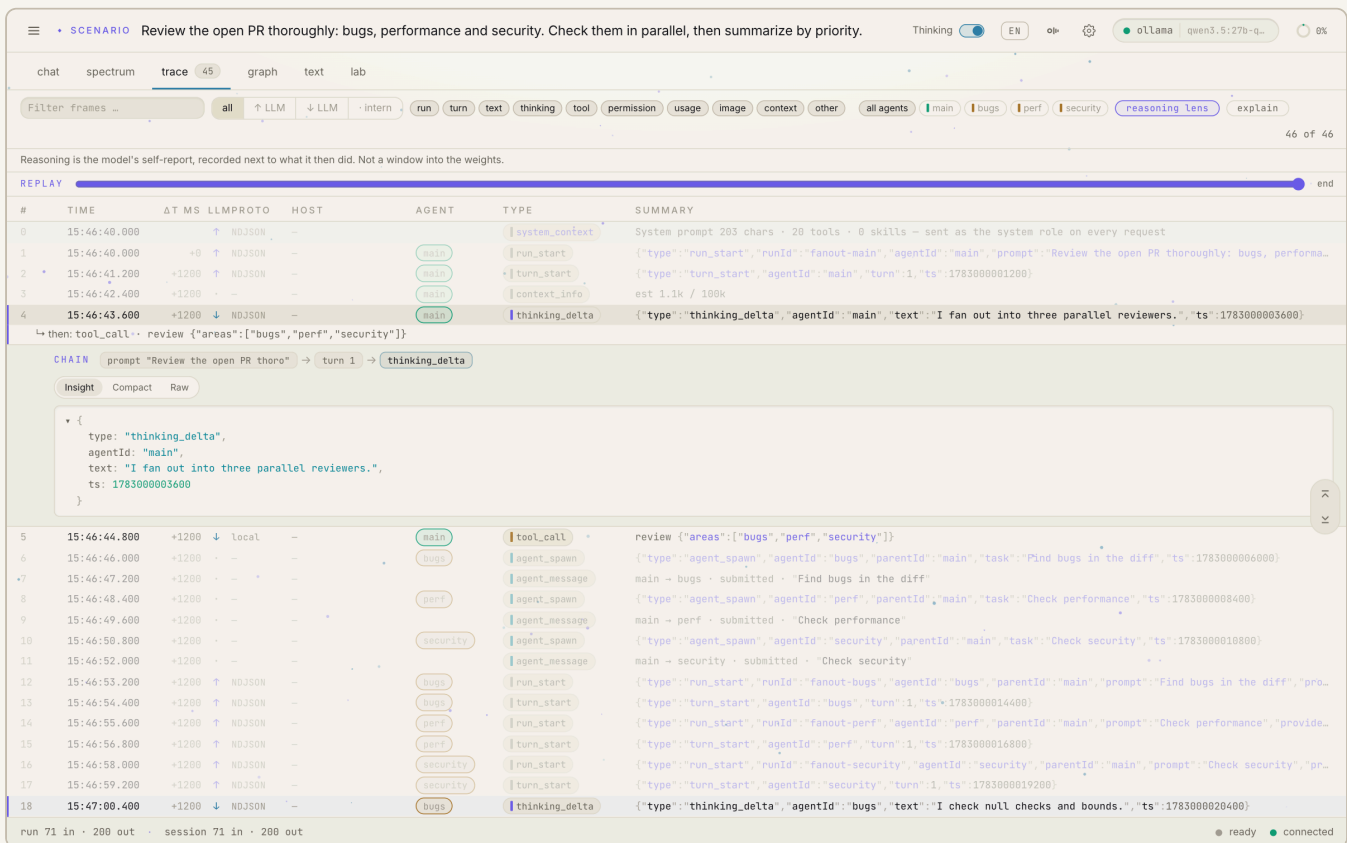


The spectrum tab: one lane per agent, every event a spectral mark. A real capture of a review fan-out with three subagents.

Clicking a lane opens the trace pinned to that agent: the new agent chip row filters the wire view to one lane while session-level frames (decisions, run ends) stay visible. Dense token streams are thinned for display; the counter names the drop, and the JSONL file keeps everything.

the reasoning lens

The reasoning lens is a trace mode for one question: what did the model say it was thinking, and what did it then do? Switched on, thinking frames come to the foreground in violet, everything else steps back, and tool calls and gate frames stay readable as anchors. Every block of thinking ends in a clickable "then:" line that jumps to the action that followed it.



The reasoning lens: thinking foregrounded in violet, the said-vs-did pairing line under the block, the causal chain in the open frame.

HONEST LABELING

Reasoning text is the model's self-report, recorded next to what it actually did. It is not a window into the weights, and the interface says so. With reasoning capture off, the lens tells you that too instead of showing an empty highlight.

The lens state persists with your design preferences, applies to live runs and replays alike, and combines with every other filter.

the causal chain and the replay scrubber

Every open trace frame now shows its walk-back: result, decision, request, call, turn, run, spawn, parent run. Each link is a chip; click it and the trace jumps there. The chain is computed from the recorded stream alone, with no model involved.

The replay scrubber caps the visible stream at any frame. Drag it back and you read the session exactly as far as it had happened, filters included.

the gate surface

Pending permissions are first-class now. Instead of a blocking modal, a gate bar docks above the footer on every tab: the violet line means the run waits on you. Approve or deny inline, keep the session rule checkbox at hand, and expand the bar for the full input and the recorded outcomes of earlier gates. The Lab keeps its own stepper-owned dialog; replays never ask.

the explain panel

The explain button in the trace toolbar docks the why layer next to the stream: a run summary (duration, turns, tool calls by name, gate tallies, tokens, stop reason) and one entry per gate that answers "why did the gate ask" from the permission mode at ask time. Stored sessions carry no mode frames; the panel says "not recorded" instead of guessing. Everything in the panel folds deterministically from the stream.

spectro doctor, the web face

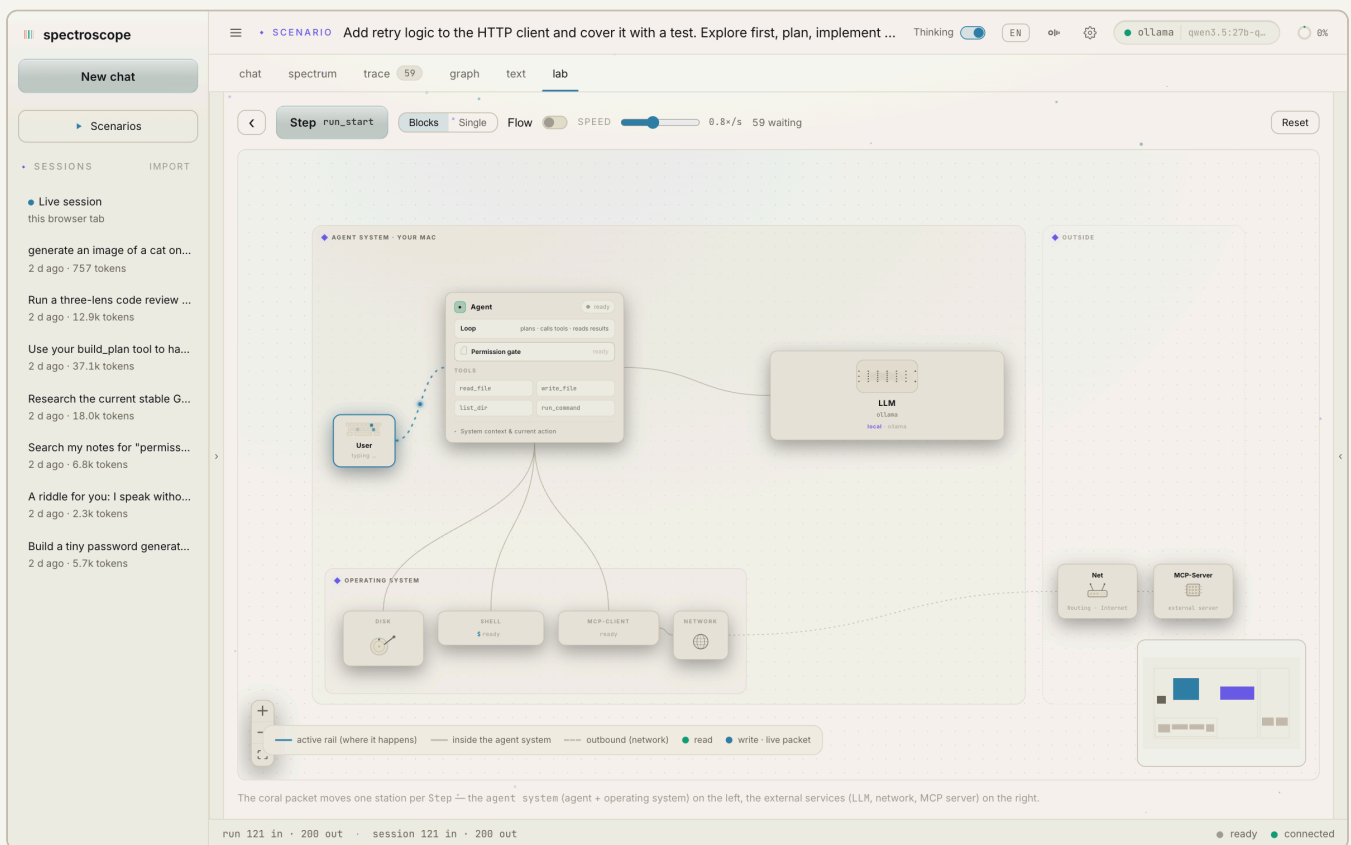
The reference-lamp button in the header opens the calibration page: one measured line per subsystem, from this browser's viewpoint. Server api, live socket, llm backend, session store, default workspace, log level, permission mode. The page re-measures on every open; the CLI's `spectro doctor` checks the machine side.

The Lab

The Lab is spectroscope's step-through debugger for agent runs, and its best classroom. Events do not flow through automatically: they queue behind a client-side **dam**, and you advance them one boundary at a time while chat, map and JSONL strip move in perfect lockstep. It works on live runs and on any archived replay.

The dam

Live events keep queueing even while the Lab is closed. Each step applies the next event(s) through the *same pure reducer* the chat uses, plus two pure folds underneath: the scene model the Flow map paints, and a strict Petri marking. The invariant is literal: the stepped UI state always equals “fold of everything applied so far”. Because the server genuinely waits on permission futures, a token sitting at the gate is the real system state, not a simulation.



A freshly loaded scenario: the dam holds every event (the “now” band reads “ready to run · 61 waiting”) and the Flow map shows the idle system, waiting for the first step.

The transport

Two things frame the map. Above it floats the **“now” band**: a plain-language line of what the run is doing at the current step (“the model is thinking”, “writing hello.txt to disk”, “the permission gate is deciding”, “orchestrating 3 workers”, “done · control is with you”), and next to it the size of the dam (“n waiting”, or

"waiting for server ..." when a live run has drained the queue). Before the first step it reads "ready to run". Because the band names the folded scene, a scrubbed replay always tells you where it is.

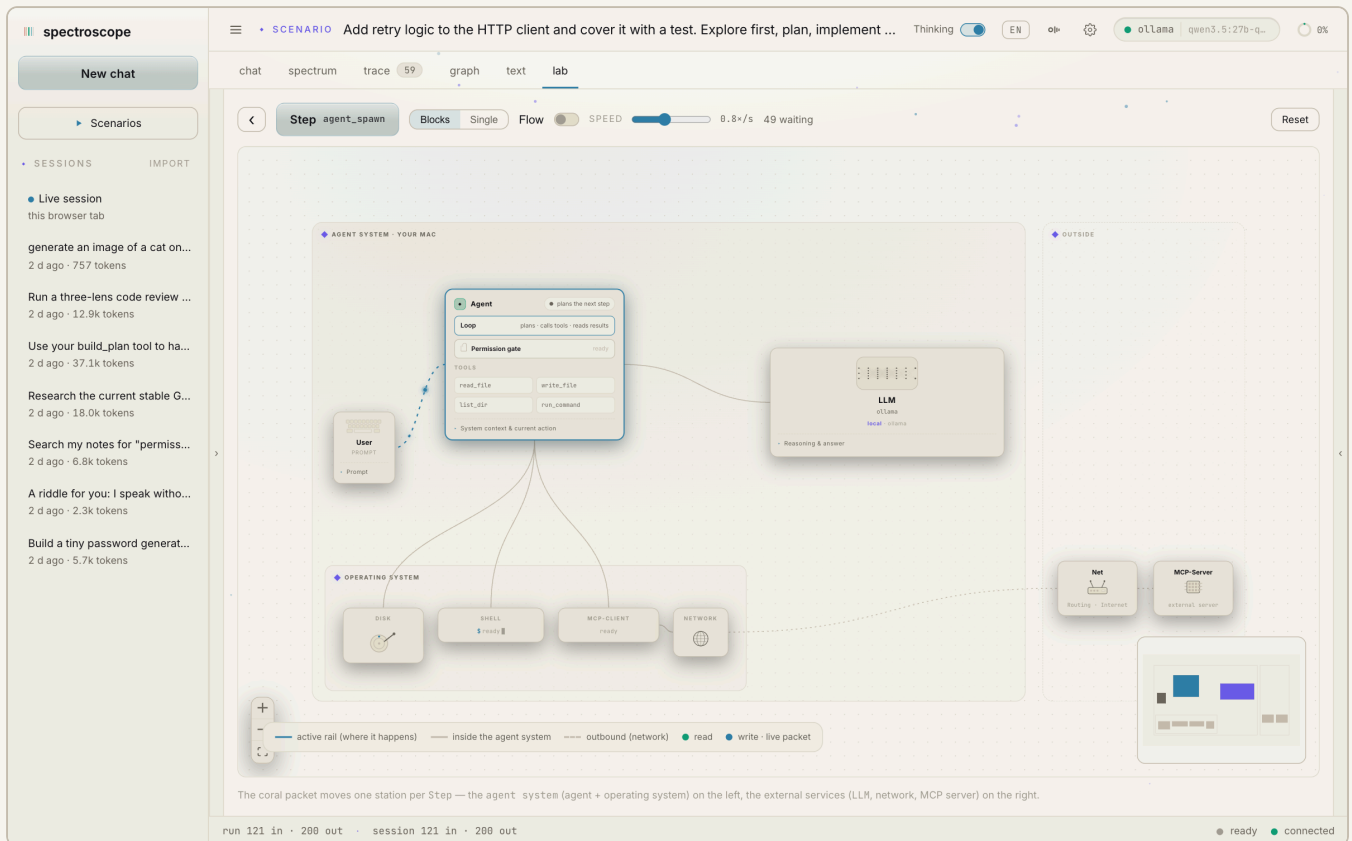
Below the map sits the **scrub bar**. Four buttons drive it: reset (↺), step back (⏮), play / pause (▶ / ⏸) and step forward (⏭). A slider walks the run's **coarse-step boundaries**, so a drag always lands on a whole step, never mid-block; a "step 4 / 27" counter reads out where you are. Scrubbing pauses auto-play. Grain and speed are one rung down, behind a small **more** disclosure that a live run rarely needs:

CONTROL	BEHAVIOUR
▶ Step forward	applies the next block (or one JSONL line in Single grain); the "now" band, chat, map and strip all move together
⏮ Step back	undoes the last step group, re-folding chat, map and strip backwards from scratch
▶ Play / ⏸ Pause	opens the dam into a paced auto-play: a timer steps at the chosen speed instead of teleporting to the end
↺ Reset	re-queues everything applied and returns to the start
Blocks Single (more)	the grain: one click = one meaningful block (a whole thinking run, a whole answer run, or one event), or exactly one JSONL line
Speed (more)	the auto-play slider, roughly 0.5×–16×/s (default 0.8/s)

Every button has a keyboard twin, guarded so it never eats a keystroke while you type in a field: space or → steps forward, ← steps back, f toggles flow (auto-play), and r resets. Press ? at any time for the full sheet (see [the keymap](#)).

The panes left and right of the map, the fully functional chat and the JSONL strip, are drag-resizable and collapsed by default, so the map gets the room. The strip shows applied lines, the just-fired line highlighted, the dam divider ("■■ dam · n waiting"), then the dimmed queue; every row expands to the full event, and an "All" toggle drops the render window for very long sessions.

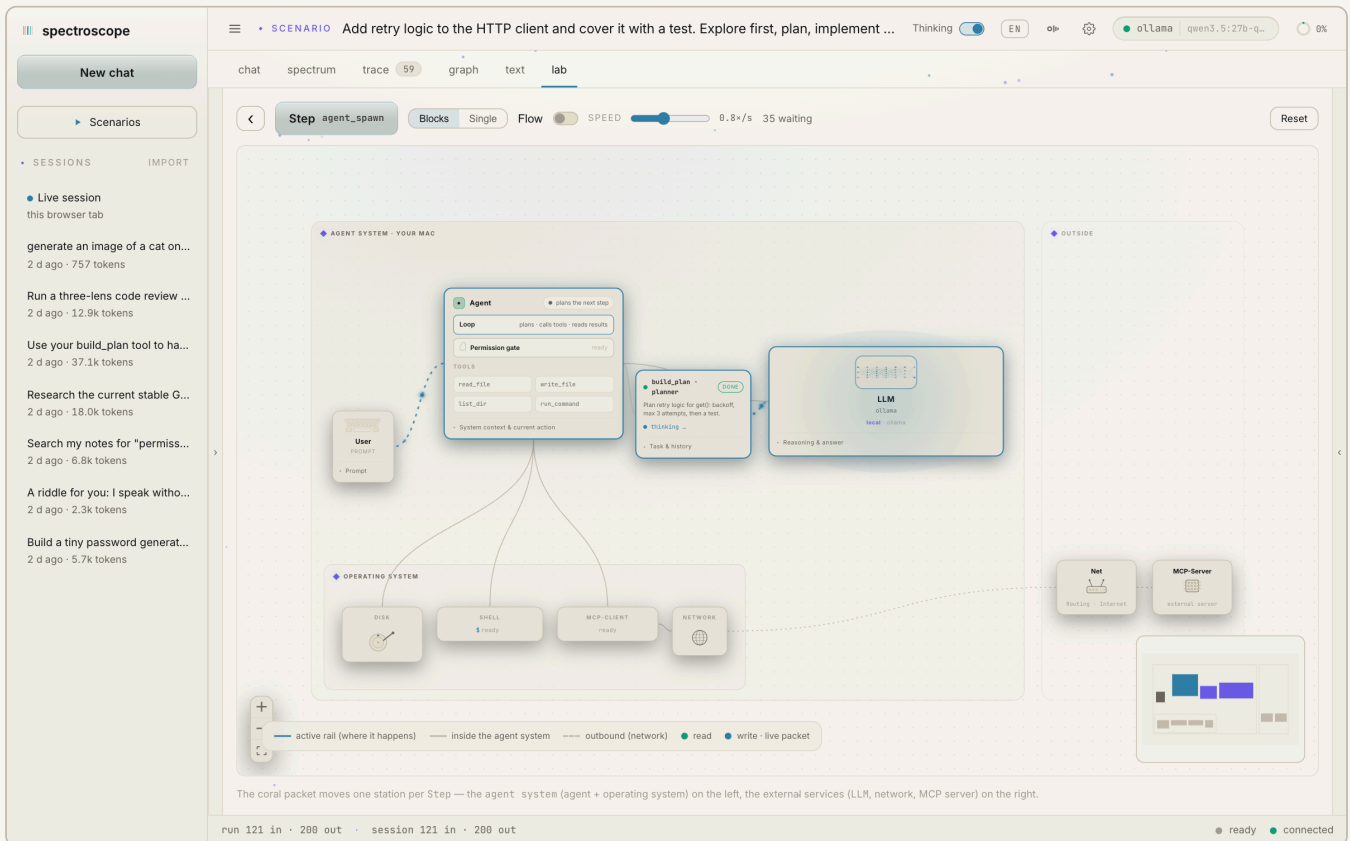
The Flow map (React Flow)



Mid-run: the coral packet sits on the Agent hub, its tool belt below the Loop and Permission-gate rows, the OS band under that, the LLM zone to the right, with live disclosures on every node.

The centre is a literal systems map. On the left, the **AGENT SYSTEM · YOUR MAC** zone: the user, the **Agent hub** (live activity line, the Loop row, the **Permission gate** row that colors with the gate state, and the tool belt) and the **OPERATING SYSTEM** band, disk (animated platter, the touched file as a pill), shell (the running command), MCP client and network stack. On the right, **OUTSIDE**: the LLM (neural-net animation while thinking, model name, reasoning and answer stream slices per agent), the network and the MCP server.

One coral packet rides the rails to wherever the current event happens. The map is **honest about locality**: with a local provider (Ollama) the LLM node sits *inside* your machine and nothing crosses the boundary; switching to a cloud provider re-replaces it beyond a dashed **network boundary**, live, because the layout follows the selected provider.



Parallel subagents: each child gets its own loop card inside the machine zone with its own coral packet, two dots flying to the shared LLM, real parallelism made visible.

Every node carries disclosures with the real data. The Agent hub now shows a **tool belt**: the chips `read_file`, `write_file`, `list_dir`, `run_command`, `use_skill`, `call_mcp` and `generate_image`, with the active one lit. Its *system context* disclosure holds the main agent's system prompt, the live context budget (estimated tokens against the compaction threshold, with a bar per segment) and the current tool call as a JSON tree, or, when the active tool is `generate_image`, a small generated-image thumbnail with its prompt caption. The LLM node reveals per-agent reasoning and answer streams; the disk shows its file, the shell its command, the MCP client its call. Subagents pop in as their own loop cards with lifecycle badges and disappear when the root run ends.

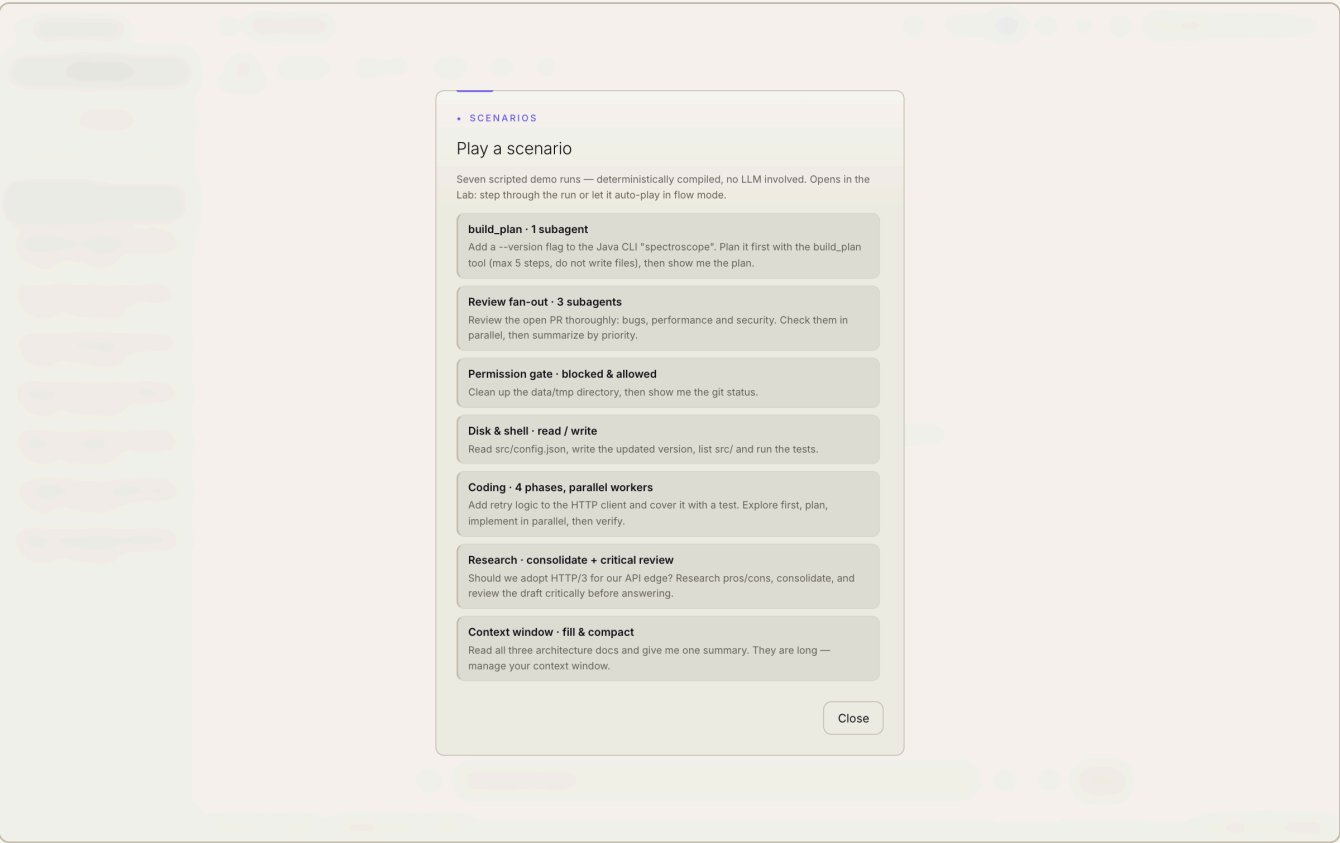
The keymap

Press `?` anywhere (or the header's keyboard button) to open the **keymap**: a game-style shortcut sheet that pairs each keycap with a small icon of what it does. It documents the Lab's real keys, `space / →` step forward, `←` step back, `f` flow, and `r` reset, alongside `?` to reopen the sheet and `esc` to close it. Close it with `Esc`, the `x`, or a backdrop click. The footer states the one rule worth knowing: shortcuts pause while you type in a field.

Under the hood: the Petri marking

The stepper also folds every event into a strict Petri net, for the formally minded: places are the stations a request passes through (*User*, *LLM*, *Tool*, *Gate*), transitions are the RunEvents, and one token moves. A **Log** place accumulates a token per fired transition, so its marking literally counts the JSONL lines, which is the invariant the strip's highlight rides on. Guards keep the net consistent: a move only fires when the source place holds the token, otherwise it degrades to a pulse (parallel tool calls, odd foreign replays). The marking has no view of its own; it is the Lab's bookkeeping, not its face.

Scenarios: deterministic demo runs



The scenario picker: seven scripted demo runs, compiled deterministically, no LLM, no API key. Each opens in the Lab with the dam closed.

The sidebar's **Scenarios** button opens seven built-in demo runs. Each is authored once in a bilingual DSL and *compiled* to a deterministic RunEvent stream: real wire format, fixed timestamps, no model involved. Picking one lands in the Lab with the stepper at event zero; it rides the same replay path as any stored session, so every tab renders it.

SCENARIO	WHAT IT DEMONSTRATES
build_plan · 1 subagent	dev-tool delegation: a planner child with status updates, verification by main, a gated test run, a <i>denied</i> MCP call and graceful continuation
Review fan-out · 3 subagents	parallel children (bugs / performance / security) running round-robin, then a prioritized consolidation
Permission gate · blocked & allowed	the red path: <code>rm -rf data/tmp</code> denied, the agent backs off to read-only listing, then an allowed <code>git status</code>
Disk & shell · read / write	the disk/shell stations: read → write → list → a gated <code>npm test</code>
Coding · 4 phases, parallel workers	a realistic dev run: explore → plan (subagent) → implement (two parallel workers writing files) → verify (gated test run)
Research · consolidate + critical review	parallel source sweep with MCP searches, a consolidation draft, an adversarial critic child that finds a contradiction, the corrected answer
Context window · fill & compact	no subagents: three big reads fill the context ring (ok → warn → high), then a compaction drops the meter and the run continues

WHY SCENARIOS EXIST

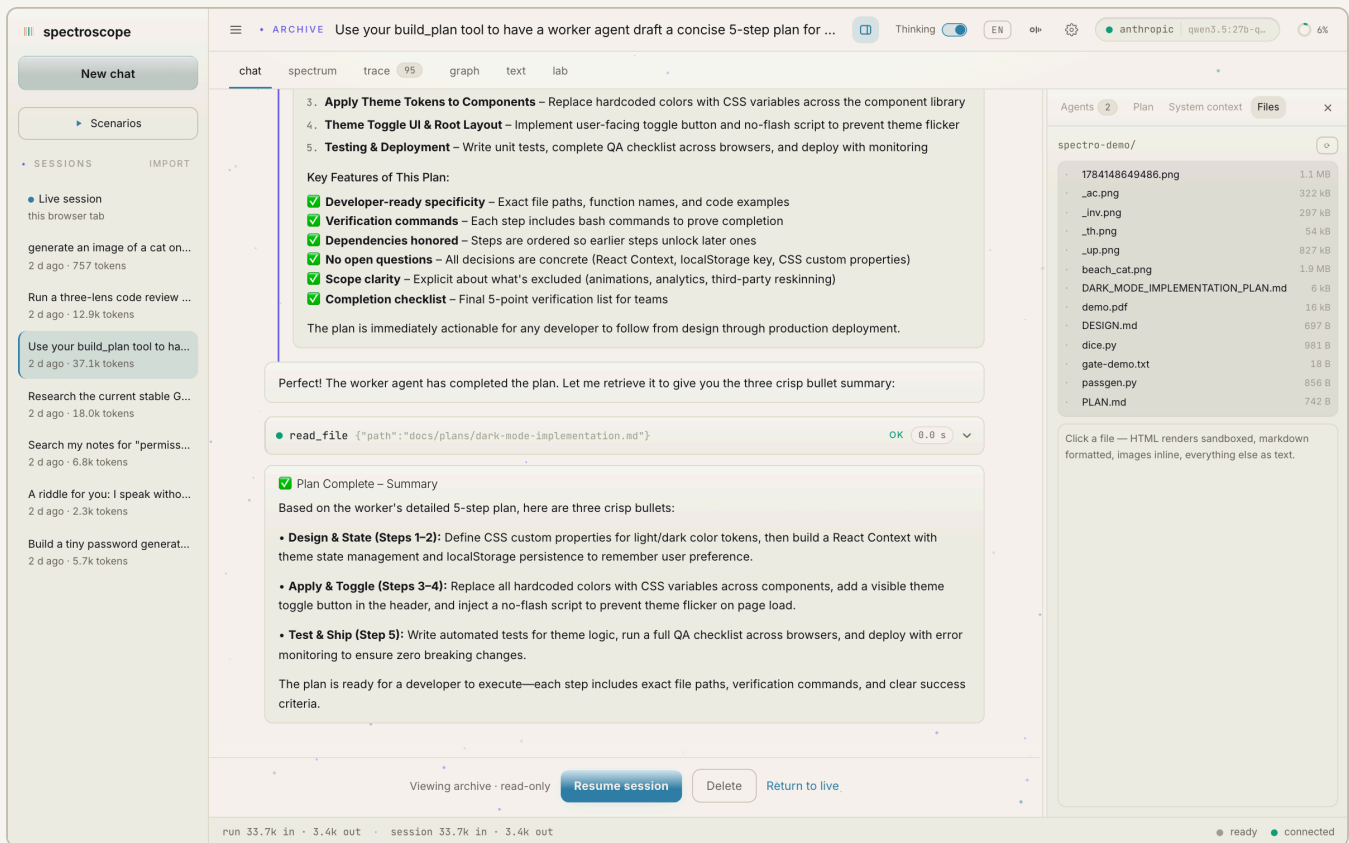
Live model behaviour is non-deterministic: a workshop demo that depends on the model calling exactly the right tool at the right moment will betray you. Scenarios make every mechanism of the harness (gates, subagents, MCP, compaction) steppable and repeatable, offline, in either language. (Most screenshots in Part II use the “coding” scenario for exactly this reason.)

Sessions: replay, resume, delete, import

Every run writes a JSONL session file as it happens — there is no “save” step, the file *is* the state. This chapter is about what you can do with those files from the UI; Part IV shows them on disk, and chapter 22 documents every line.

The archive

The sidebar lists every stored session newest-first: first prompt, relative age, token total. Opening one is a **replay** — the same events, folded by the same reducer, rendered read-only. Chat, Graph, Trace and Lab all work on it; the composer is replaced by the archive bar.



An archived session: the header eyebrow says Archive, the bar offers “Resume session”, the two-step Delete, and “Return to live”. All four tabs work on the replay.

Resume — with the re-upload made visible

Resume session turns an archive back into the live session: the UI seeds itself from the stored events, the socket reconnects with `?resume=<id>`, the server reconstructs the provider history from the file (main agent only, orphaned tool calls dropped — chapter 22), and new events *append to the same file*. The header eyebrow flips to *Resumed*.

spectroscope

Use your build_plan tool to have a worker agent draft a concise 5-step plan for addin... Thinking 6%

chat spectrum trace 99 graph text lab

Filter frames ... all LLM LLM intern run turn text thinking tool permission usage image context other all agents main worker-1

reasoning Lens explain

100 of 100

REPLAY

#	TIME	Δ T MS	LLMPROTO	HOST	AGENT	TYPE	SUMMARY
0	14:47:32.841		↑ SSE	api.anthropic.com		system_context	System prompt 203 chars · 20 tools · 0 skills — sent as the system role on every ...
1	14:47:32.841	+0	↑ SSE	api.anthropic.com	main	run_start	{ "type": "run_start", "runId": "3233799b-2e4a-471a-92b0-9036b75551f4", "agentId": "mai...
2	14:47:32.841	+0	↑ SSE	api.anthropic.com	main	turn_start	{ "type": "turn_start", "agentId": "main", "turn": 1, "ts": 1784292452841 }
3	14:47:32.841	+0	-	-	main	context_info	est 2.1k / 100k
4	14:47:33.950	+1109	↓ SSE	api.anthropic.com	main	thinking_delta	{ "type": "thinking_delta", "agentId": "main", "text": "The user wants me to:", "ts": 178...
5	14:47:34.150	+200	↓ SSE	api.anthropic.com	main	thinking_delta	{ "type": "thinking_delta", "agentId": "main", "text": "\n1. Use the build_plan tool to...
6	14:47:34.352	+202	↓ SSE	api.anthropic.com	main	thinking_delta	{ "type": "thinking_delta", "agentId": "main", "text": "'s working\n3. After it complet...
7	14:47:34.614	+262	↓ SSE	api.anthropic.com	main	thinking_delta	{ "type": "thinking_delta", "agentId": "main", "text": " with a clear, self-contained t...
8	14:47:35.297	+683	↓ local	-	main	tool_call	build_plan { "task": "Create a concise 5-step implementation plan for adding dark m...
9	14:47:35.297	+0	↓ SSE	api.anthropic.com	main	usage	3312 in / 188 out
10	14:47:35.297	+0	-	-	worker-1	agent_spawn	{ "type": "agent_spawn", "agentId": "worker-1", "parentId": "main", "task": "You are acti...
11	14:47:35.297	+0	-	-	worker-1	agent_message	main → worker-1 · submitted · "Create a concise 5-step implementation plan for ad...
12	14:47:35.298	+1	↑ SSE	api.anthropic.com	worker-1	run_start	{ "type": "run_start", "runId": "f3ff543e-f5d7-4f01-87f7-2fcb292d6ef", "agentId": "wor...
13	14:47:35.298	+0	↑ SSE	api.anthropic.com	worker-1	turn_start	{ "type": "turn_start", "agentId": "worker-1", "turn": 1, "ts": 1784292455298 }
14	14:47:35.960	+662	↓ SSE	api.anthropic.com	worker-1	text_delta	I'll help
15	14:47:36.175	+215	↓ SSE	api.anthropic.com	worker-1	text_delta	you create a concrete implementation plan for adding dark mode to a web app. Let...
16	14:47:36.393	+218	↓ SSE	api.anthropic.com	worker-1	text_delta	plan.
17	14:47:36.432	+39	↓ local	-	worker-1	tool_call	.use_skill({ "name": "writing-plans" })
18	14:47:36.432	+0	↓ SSE	api.anthropic.com	worker-1	usage	1697 in / 93 out
19	14:47:36.432	+0	↑ local	-	worker-1	tool_result	ok · 0 ms
20	14:47:36.432	+0	↑ SSE	api.anthropic.com	worker-1	turn_start	{ "type": "turn_start", "agentId": "worker-1", "turn": 2, "ts": 1784292456432 }
21	14:47:37.107	+675	↓ SSE	api.anthropic.com	worker-1	text_delta	Good
22	14:47:37.347	+240	↓ SSE	api.anthropic.com	worker-1	text_delta	! Now let me create a concrete implementation plan following the skill's guidance...
23	14:47:37.539	+192	↓ SSE	api.anthropic.com	worker-1	text_delta	is a task without an existing codebase context, I'll create a comprehensive, deve...
24	14:47:37.886	+267	↓ SSE	api.anthropic.com	worker-1	text_delta	plan.
25	14:47:38.011	+205	↓ local	-	worker-1	tool_call	report_status { "message": "Analyzing task and creating concrete 5-step dark mode i...
26	14:47:38.011	+0	↓ SSE	api.anthropic.com	worker-1	usage	2241 in / 107 out
27	14:47:38.011	+0	-	-	worker-1	agent_message	worker-1 → main · working · "Analyzing task and creating concrete 5-step dark mod...
28	14:47:38.011	+0	↑ local	-	worker-1	tool_result	ok · 0 ms
29	14:47:38.011	+0	↑ SSE	api.anthropic.com	worker-1	turn_start	{ "type": "turn_start", "agentId": "worker-1", "turn": 3, "ts": 1784292458011 }

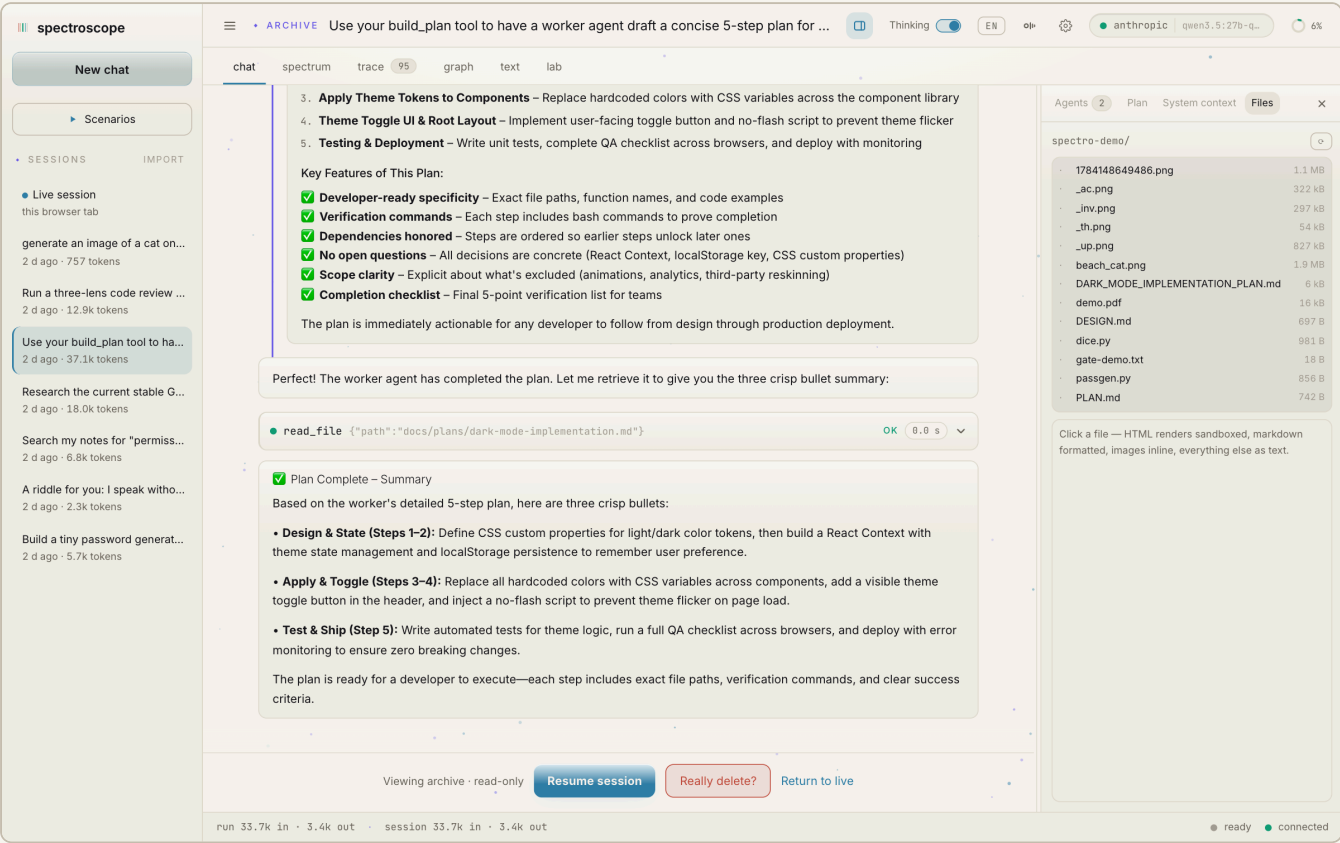
run 33.7k in · 3.4k out · session 33.7k in · 3.4k out

ready connected

After a resume, the trace shows the whole reconstructed history followed by the session_resume marker row: "n events loaded · ~t tokens of history ride along with the next request (plus system prompt & tool schemas on top)."

The didactic detail: resuming is not free. Your next prompt re-uploads the whole conversation to the model. The trace makes that visible with a UI-only **session_resume marker** between history and new traffic, summarizing how many events and roughly how many tokens now ride along. The estimate mirrors what the server actually reconstructs: main agent only (a subagent's inner steps never ride back up), thinking excluded (it never re-enters the provider history) — and the marker says explicitly that the system prompt and tool schemas come on top, so the next **usage** event's real jump lands where you expect it. The marker's detail view carries the entire re-uploaded history as its JSONL lines. Scenario and import replays offer no resume — there is no file to append to.

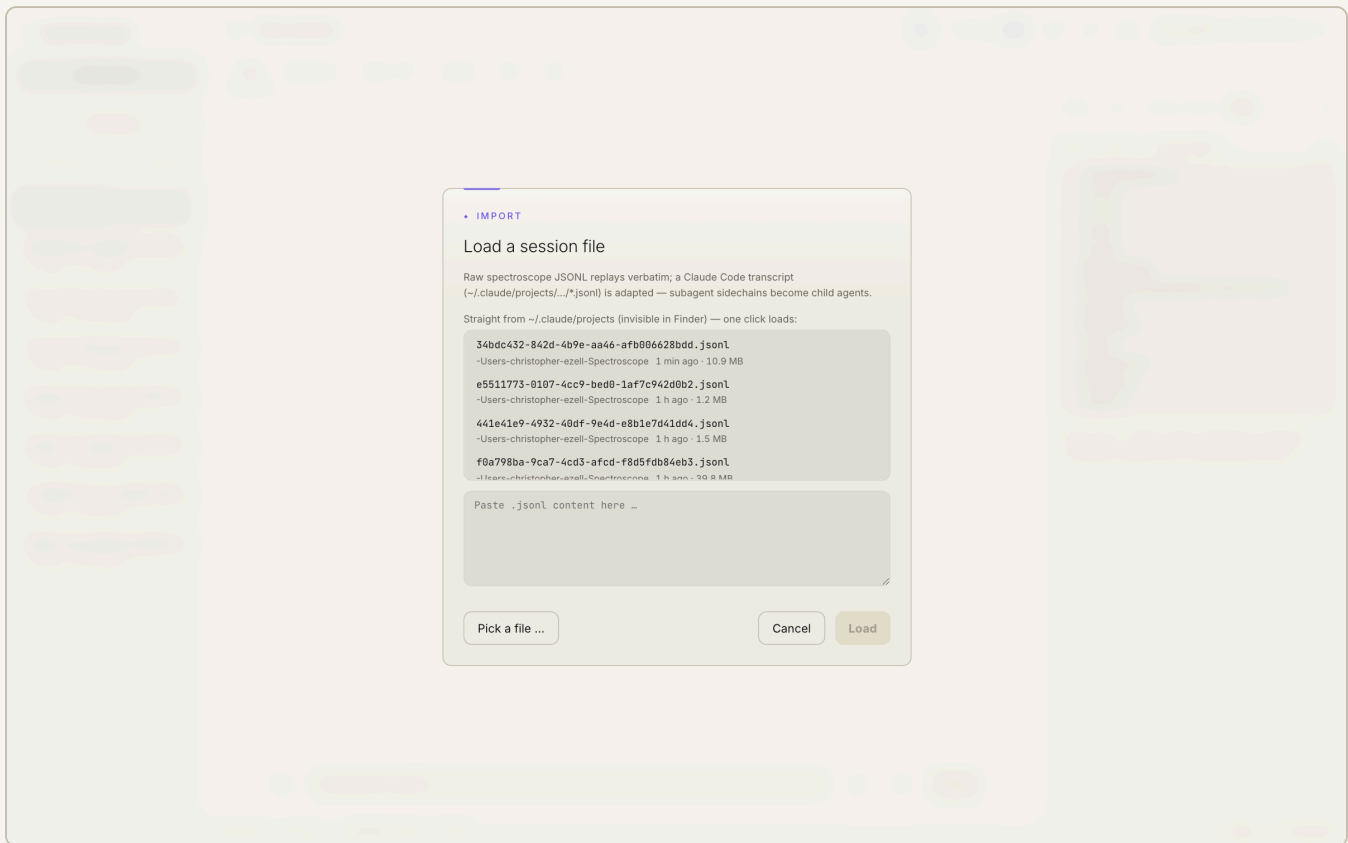
Delete — the two-step danger button



First click arms: “Really delete?” in error tint. A second click within four seconds deletes; standing still disarms. Nothing else in spectroscope destroys data.

Deleting a session is the first deliberately destructive surface in spectroscope, so it is defended in depth: the button must be clicked twice within four seconds (first click arms, idle disarms); the REST endpoint (`DELETE /api/sessions/{id}`) guards the id shape before touching anything; and the store itself refuses any id that does not resolve to a direct child of the sessions directory — traversal ids delete nothing. Success removes the JSONL *and* the session's blob folder, closes the replay and refreshes the sidebar. The session currently being resumed by the live tab never offers the button.

Import



Three ways in: one-click list of your local Claude Code transcripts (the ~/.claude store is Finder-invisible — a file picker cannot reach it), a file picker, or paste.

The Import dialog replays *foreign* files through the same pipeline:

- **Raw spectroscopy JSONL** — recognized by its event types, replayed verbatim. Files from the TypeScript edition work identically (the format is the contract).
- **Claude Code transcripts** — recognized by their `message` records and translated by an adapter: tool uses become `tool_call`s, `Task / Agent` spawns become `agent_spawn` + A2A messages, thinking becomes `thinking_delta`, sidechains fold under their owning task. You can watch a real Claude Code session step through the Lab's map.



Import detection: spectroscope JSONL replays verbatim; Claude Code transcripts run through the adapter; both feed the same replay path as stored sessions.

KNOWN LIMITATION

Newer Claude Code versions store subagent transcripts as separate files

(`.../subagents/agent-*.jsonl`). Importing a main transcript shows each spawn and its result, but not the child's inner steps — a multi-file merge is a future candidate.

New chat semantics

One socket connection is one server-side session — one agent, one JSONL file. “New chat” therefore closes and reopens the socket: fresh agent, fresh file, fresh state. Two browser tabs are two fully independent sessions (separate agents, files, remembered permission rules); only persisted rules in `.spectro/settings.json` are shared, which is why that writer is serialized.

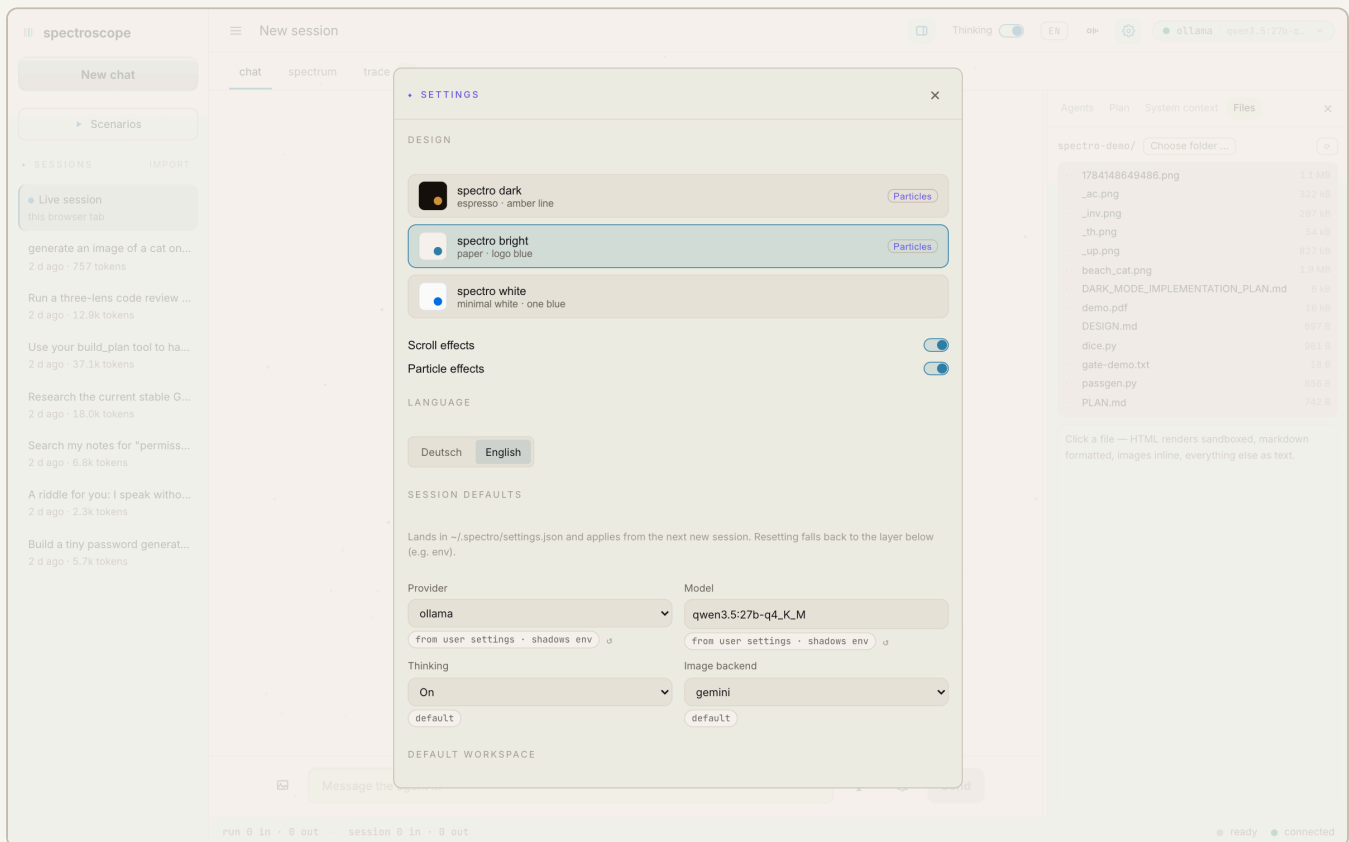
The design system

The whole UI reads its colours, fonts, radii and motion through CSS custom properties. That single indirection makes two things cheap: swapping the entire look (a *design*) and painting effects that always match.

The token base

The master vocabulary is brand-strict: espresso `#17120D` as the ground (never pure black), **the logo's amber line** `#CE9440` **exclusively for interaction and live elements** (links, buttons, focus, running indicators, the packet), **violet** `#8B7CF0` **as the editorial accent** (eyebrows, kickers — never clickable), desaturated status colours, and pastel agent accents that are decoration only. Focus is always visible and always the accent; reduced motion switches every animation off globally; numbers render tabular so nothing jitters mid-stream.

Three designs

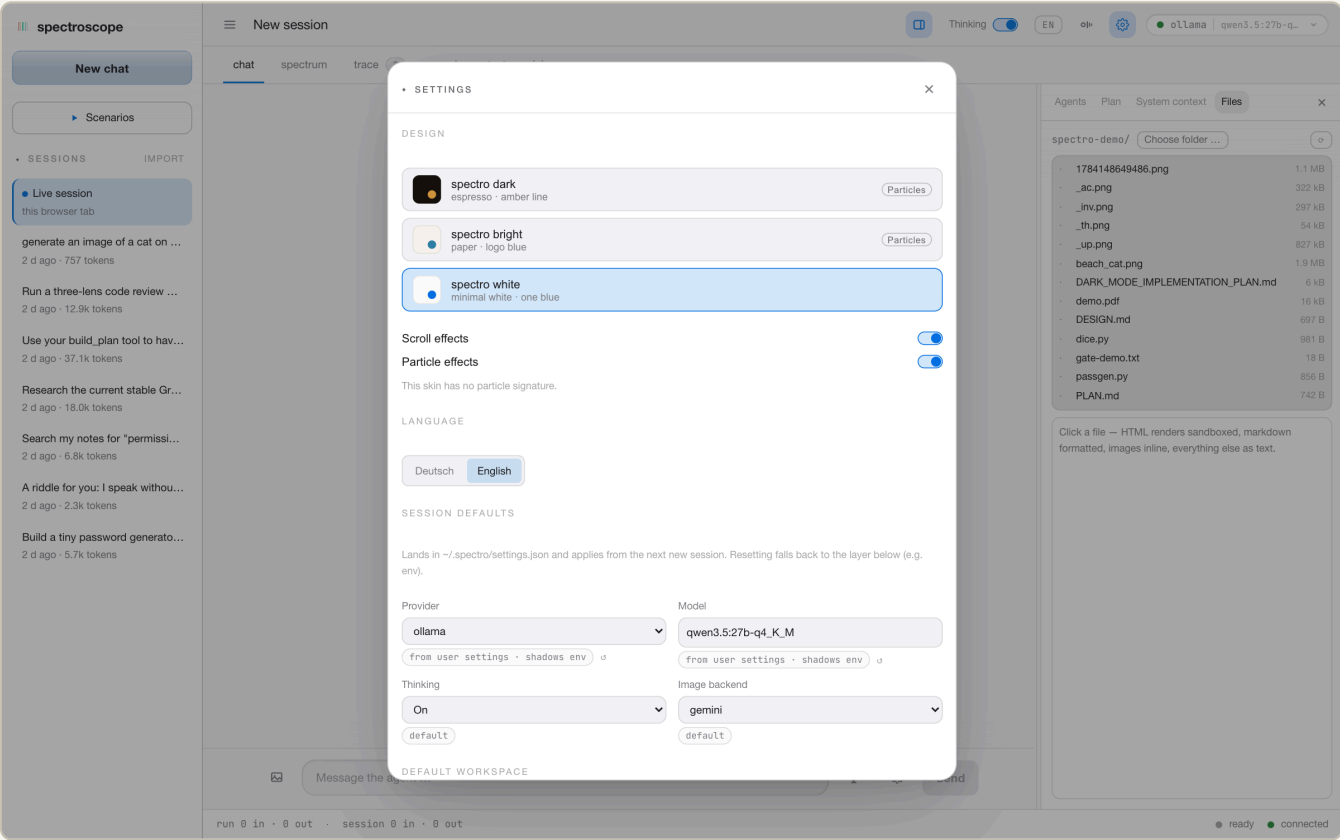


The design section of the settings page: the three designs with swatches, the effect switches beneath — a choice applies live and persists in one step.

A design is a token override set — nothing structural changes. Selecting one flips a `data-design` attribute on `<html>` and the UI re-expresses itself instantly, no reload; a FOUC guard applies the saved design before first paint. There used to be a shelf of extra skins; the picker was pruned to the three brand designs, and any retired id left in older browser state folds back to the default.

DESIGN	DATA-DESIGN	CHARACTER	GROUND / ACCENT	PARTICLES
spectro dark	spectroscope	espresso · amber line (the default)	#17120D / #CE9440	brand dust
spectro bright	paper	LIGHT — paper · the logo blue	#F6F4EE / #2E7EA6	ink dust
spectro white	still	LIGHT — minimal white, gray, one blue	#fbfbfd / #0071e3	— (deliberately still)

The two **light** designs flip `color-scheme` and re-derive the status colours, agent pastels and the whole shadow set for a light ground — proof that the token vocabulary carries a full polarity flip, not just a palette swap.



spectro white: the same UI in the quiet all-white design — every component reads `var(--token)`, so the flip is total, live behind the settings page.

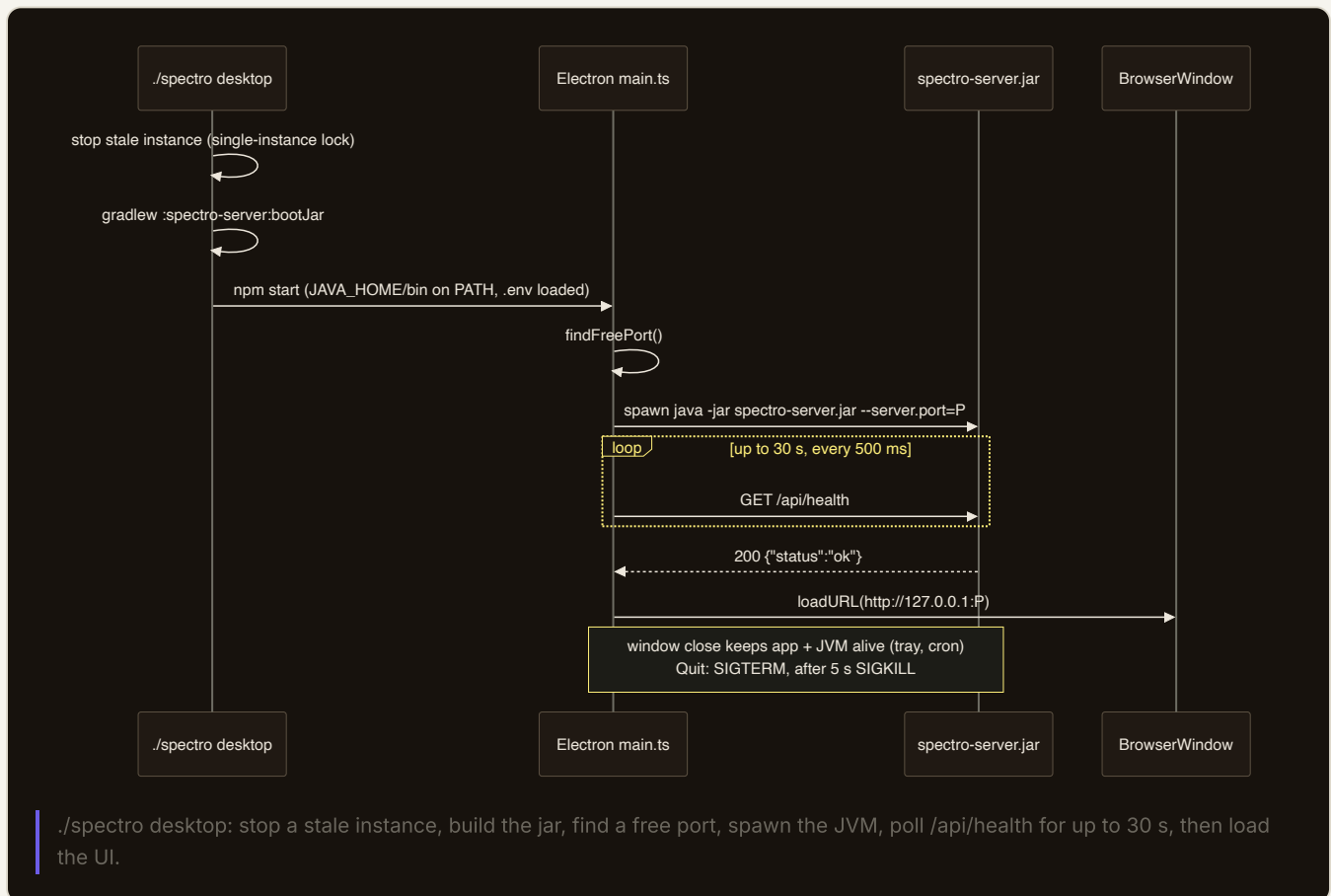
The effects layer

Two independently togglable effects, both reading the *active* design's colours live: the **particle field** (a canvas behind the UI with the brand dust drift — spectro white deliberately renders none) and the **scroll reveal** (panels fade/rise on entry). Both defer to reduced motion; particles pause in hidden tabs. A design choice from the settings page applies and persists in one step; the effect switches work the same way.

The desktop face

A JVM cannot live inside Electron's main process, so the desktop face is honest about it: a thin shell that *supervises* the real server as a child process and points a window at it. Packaged, it goes one step further and carries everything it supervises — the server jar and a whole Java runtime — so a fresh machine needs no system Java. Double-click, the server starts with it, the cockpit opens. The lesson of this face is process management, not IPC.

The supervision sequence



`./spectro desktop` prepares the ground (stops a stale instance — Electron's single-instance lock would otherwise hand a relaunch to the *old* build; builds the boot jar; clears the macOS quarantine flag from fresh Electron binaries on first run; puts the resolved JDK on the PATH). The Electron main process then finds a free port, spawns `java -jar spectro-server.jar --server.port=P`, polls `/api/health` every 500 ms for up to 30 s, and opens a 1200×800 window on `http://127.0.0.1:P` — the same React UI, talking over the same WebSocket. The renderer gets no Node API at all (`contextIsolation` on, `nodeIntegration` off): it is an ordinary web page.

A self-contained run kit

That dev flow expects a JDK on the machine. The packaged app does not.

`scripts/build-desktop-runkit.sh` bundles two things into the `.dmg` as Electron `extraResources`: the **server jar** and a **jlink'd Java runtime** (a trimmed JRE built straight from your JDK). Run packaged, the shell resolves both from inside its own bundle — the jar from the app resources, `java` from the bundled `jre/bin/java` — so a fresh machine needs no system Java at all. Double-click the app, the server boots on the bundled runtime, the cockpit window opens: the same React UI over the same WebSocket. (In dev the shell falls back to the `java` on your PATH and the jar from the Gradle build.) The app wears the brand — `productName` spectroscope, a brand app icon that the build script regenerates from `icon.svg` when `rsvg-convert` is present.

- **First launch is a right-click.** The default build is *ad-hoc signed* (free, no Apple account). That clears the “is damaged” gate, but on first download macOS still calls it an unidentified developer: right-click the app → *Open* once (or `xattr -cr`), and it launches normally ever after.
- **Zero-warning distribution is a paid path.** A plain double-click with no prompt needs an Apple Developer ID plus notarization. The bundled JRE is the one twist — its dylibs and JIT need signing and the right entitlements. The full recipe, end to end, is in `docs/DESKTOP-SIGNING.md`.
- **Built for the host.** The bundled runtime and the Electron binary are OS- and CPU-specific, so the script builds for the machine you run it on; a Windows, Linux, or Intel kit means running the build there.

Tray, notifications, quitting

- **Closing the window does not quit.** The tray (an ebony diamond) keeps shell and server alive so cron jobs keep running. Tray menu: New chat, Cron status, Quit.
- **Native cron notifications.** Every 30 s the shell polls `GET /api/jobs/state` and fires a desktop notification for each job whose status *changed* — click to open the window.
- **Quit is graceful, then firm.** `⌘Q` / tray-Quit sends the JVM a SIGTERM (Spring Boot closes sockets and finishes the current JSONL line); if the process still lives after five seconds, SIGKILL follows.
- **Startup failures clean up.** If health never turns green, the shell kills the JVM before showing the error — no headless server left behind. In dev a missing `java` shows “Java 21 is required”; the packaged run kit carries its own runtime, so it cannot miss.

Two languages

The UI chrome ships fully bilingual — English (the default) and German — switched live by the header toggle. This guide's screenshots use the English chrome.

- **Chrome only, content never.** Sidebar, header, dialogs, panels, tool cards, Lab toolbar, the in-map SVG texts, trace chrome, settings page, footer — all switch live (~460 dictionary entries). Chat content, tool payloads and the JSONL wire never run through translation: a session keeps its own language.
- **Wire values stay English.** Plan statuses (`pending` / `in_progress` / `completed`), A2A states and stop reasons are canonical English on the wire — only labels localize. Cross-edition compatibility depends on it.
- **Scenarios compile per language.** Picking a scenario compiles it in the current chrome language; the compiled session then keeps that language, like any other session.
- **Late-bound info lines.** Reducer-folded info lines (spawns, compactions) carry an additive key + variables, so lines already in the transcript re-render when you flip the language.



PART III

The agent's powers

Everything the model may do, and everything that keeps it honest: the tool belt, the permission system, hooks, subagents, skills, providers, the senses — vision, voice, images — MCP and the scheduler.

[12](#) The tool belt

[13](#) Permissions, allowlist & hooks

[14](#) Subagents & A2A

[15](#) Skills

[16](#) Providers & models

[17](#) Seeing, hearing, speaking, painting

[18](#) MCP — external tool servers

[19](#) Scheduling

The tool belt

Twenty-two built-in tools, plus one dynamic adapter per connected MCP tool. Every tool obeys one iron contract: **it never throws** — failures come back as a string starting `ERROR:` , which the model reads and self-corrects from. Every model-supplied path passes the sandbox. Every mutation passes the gate.

The shared rules

THE PATH SANDBOX

Every file tool resolves paths relative to the working directory and refuses anything that normalizes outside it
(`ERROR: path is outside the working directory`).
Tree-walking tools additionally verify each hit's *real* path (symlink-escape guard) and never enter `.git`, `build`, `node_modules`, `target`.

THE CAPS

Files: 50 kB read/edit ceiling. Output: 10 000 chars shared clamp (surrogate-safe truncation). Glob: 200 results. Shell: 10 s timeout, pipe drained on a background thread — a timeout means “genuinely hung”, never “output too large”. Web: 512 kB streamed body cap before the text clip.

Files & search

`read_file` free

input: { path }

Reads a UTF-8 text file (≤ 50 kB) relative to the working directory. *Try: “Read build.gradle.kts and tell me the dependencies.”*

`list_dir` free

input: { path }

Lists a directory, sorted, directories marked with a trailing slash; an empty directory answers `(empty)`.

`glob` free

input: { pattern, path? }

Finds files by glob pattern (`**/*.java`) under a directory — pure Java, pruned walk, symlink-guarded, results sorted and capped at 200. Also granted to explore subagents. *Try: “Find every *.test.ts file under spectro-web.”*

grep

free

input: { pattern, path?, glob? }

Searches file contents by Java regex, returning `path:line:text` hits — no shell involved, so it works even where `run_command` is denied. Skips oversized and binary files; invalid regex answers `ERROR: invalid regex: ...`. Try: *"Search for TODO across the Java sources."*

Mutation & shell

write_file

gated · remembers path

input: { path, content }

Writes a text file, creating parent directories. Success: `Wrote: <path> (<n> bytes)`. An "always allow" remembers the *full path*, never a blanket approval.

edit_file

gated · remembers path

input: { path, old_string, new_string, replace_all? }

Surgical exact-string replacement. `old_string` must be unique (`ERROR: old_string is not unique (n matches)` otherwise) unless `replace_all`; zero matches and empty `old_string` error readably. A denied edit writes nothing. Try: *"In README.md, change the title X to Y."*

run_command

gated · remembers first token

input: { command }

Runs a shell command in the working directory via `/bin/sh -c`: stderr merged, output drained concurrently and capped, 10 s timeout, killed on abort. Non-zero exit returns `ERROR: exit code <n>` *plus the output* — exit-code feedback is what lets the model fix its own mistakes. An "always allow" remembers only the first token (`run_command:git*`).

web_fetch

gated · remembers URL

input: { url }

Fetches an http/https page and returns readable text (scripts/styles dropped, tags stripped, entities decoded, whitespace collapsed, 10 k-char clip). Network egress on model-chosen input is a side effect — hence the gate. Streaming fetch with finite 15 s timeouts and a 512 kB body cap; non-2xx answers `ERROR: web_fetch got HTTP <status>`. Main agent only.

Metadata & delegation

update_plan

free · main only

input: { steps: [{text, status}] }

Publishes the agent's step list to the Plan panel (chapter 6) as the additive `plan` event. Latest wins — each call replaces the whole plan. The status enum (`pending / in_progress / completed`) is *enforced*: a model improvising `"done"` gets an error instead of polluting the wire. Deliberately not given to subagents — a worker's plan would clobber the display.

<code>use_skill</code> free	input: { name }
Returns the full instructions of a named skill (chapter 15). Registered only when skills are installed; children get it too, so dev-tool workers can load their role skill.	
<code>generate_image</code> gated	input: { prompt }
Generates an image via the configured image provider (chapter 17). Gated because it is a paid cloud call. The image lands content-addressed under <code>~/.spectro/images/</code> ; the model sees only a short confirmation — the user sees the gallery.	
<code>spawn_agent</code> / <code>spawn_agents</code> free · parent only	input: { type, task } / { agents: [...] }
Starts one subagent (or up to four in parallel) and waits for the result — chapter 14. Free of a gate because the <i>child's</i> tools ask for permission individually; delegation is not an escape hatch.	
<code>build_plan</code> · <code>write_spec</code> · <code>develop</code> · <code>test</code> free · parent only	input: { task }
The four development role tools — thin wrappers over a worker spawn, each pointing its child at a skill (chapter 14).	
<code>report_status</code> free · children only	input: { message }
The child's side of the A2A protocol: one short progress sentence per milestone, surfacing live in the parent's UI. Telling the parent what you do is never dangerous.	
<code>mcp__<server>__<tool></code> gated · dynamic	schema from the server
Every tool a connected MCP server advertises, wrapped as a first-class spectroscope tool (chapter 18). Always gated: its inputs are model output and its effects are external.	

Who gets which tools

AGENT	REGISTRY
<code>main</code>	all of the above (21 named + MCP; <code>use_skill</code> when skills exist)
<code>worker-N</code>	<code>list_dir</code> , <code>read_file</code> , <code>write_file</code> , <code>run_command</code> , <code>edit_file</code> , <code>glob</code> , <code>grep</code> , <code>view_image</code> , <code>view_file</code> , <code>use_skill*</code> , <code>report_status</code>
<code>explore-N</code>	<code>list_dir</code> , <code>read_file</code> , <code>glob</code> , <code>grep</code> , <code>report_status</code> — read-only <i>by construction</i> : everything else simply is not in its registry

* when skills are installed. Children never receive: `spawn/dev` tools (nesting depth 1 is structural), `generate_image`, `web_fetch`, `web_search`, `browse_page`, `update_plan`, MCP tools.

Permissions, allowlist & hooks

You stay in control through three layered mechanisms: the permission gate (a human decides), the allowlist (you decide once, scoped), and hooks (your scripts decide, before the gate even asks). All three leave an audit trail in the session file.

The guard pipeline

Every single tool call passes this pipeline, in this order — the order is load-bearing:

the 1990s, the number of people in the UK who are aged 65 and over has increased by 1.5 million (1990–2000) and is projected to increase by a further 1.5 million by 2020 (Office for National Statistics, 2000). The number of people aged 65 and over is projected to increase by 2.5 million by 2020 in the USA (U.S. Census Bureau, 2000).

There is a growing awareness of the need to develop strategies to meet the needs of the ageing population. The World Health Organization (WHO) has developed a 'Global Strategy on Ageing and Health' (WHO, 1999) which states that 'the challenge of ageing is to ensure that people are able to live longer, healthier, and more active lives'. The WHO also states that 'the goal of ageing is to ensure that people are able to live longer, healthier, and more active lives'. The WHO also states that 'the goal of ageing is to ensure that people are able to live longer, healthier, and more active lives'.

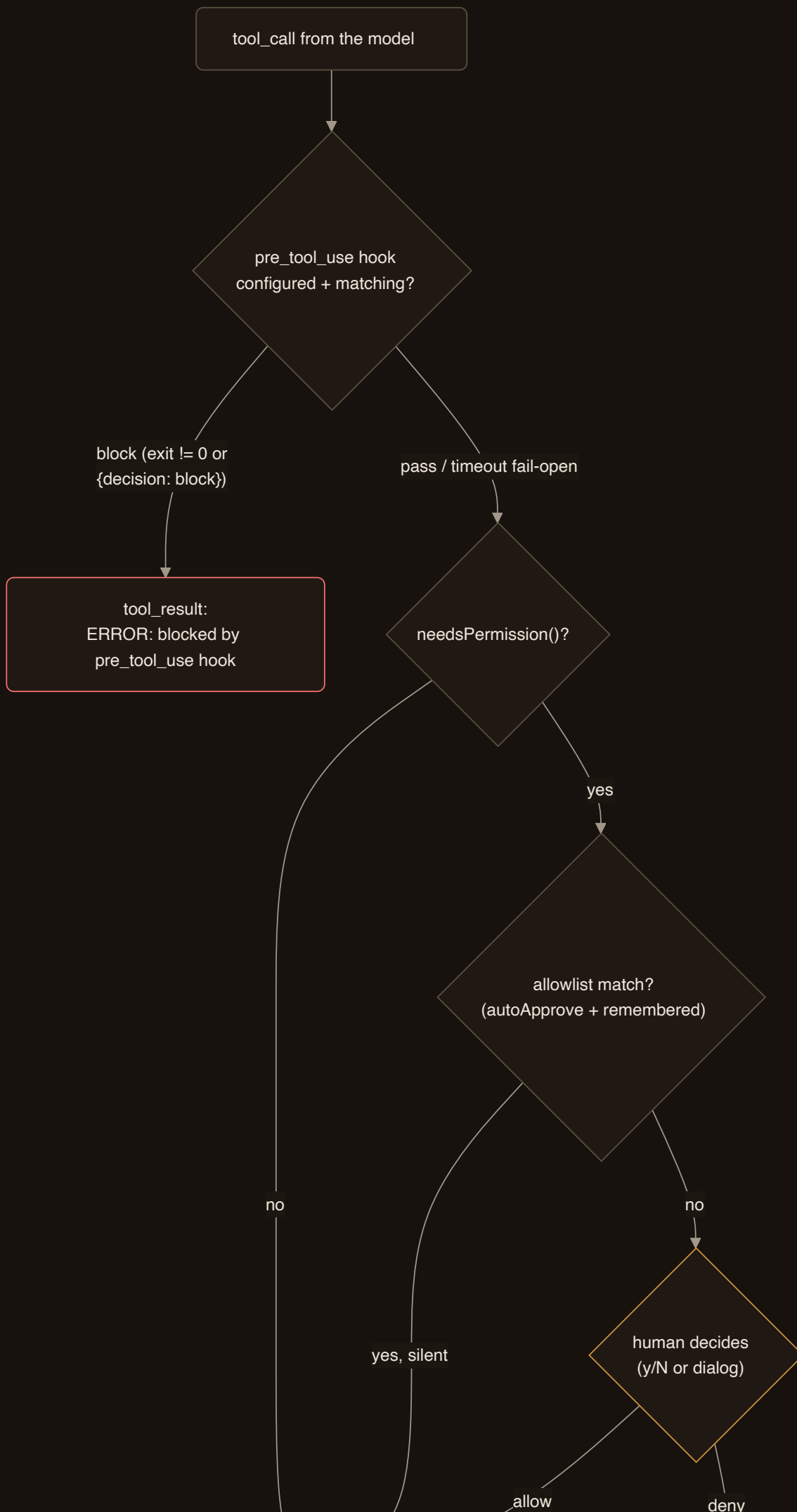
The WHO also states that 'the goal of ageing is to ensure that people are able to live longer, healthier, and more active lives'. The WHO also states that 'the goal of ageing is to ensure that people are able to live longer, healthier, and more active lives'. The WHO also states that 'the goal of ageing is to ensure that people are able to live longer, healthier, and more active lives'. The WHO also states that 'the goal of ageing is to ensure that people are able to live longer, healthier, and more active lives'.

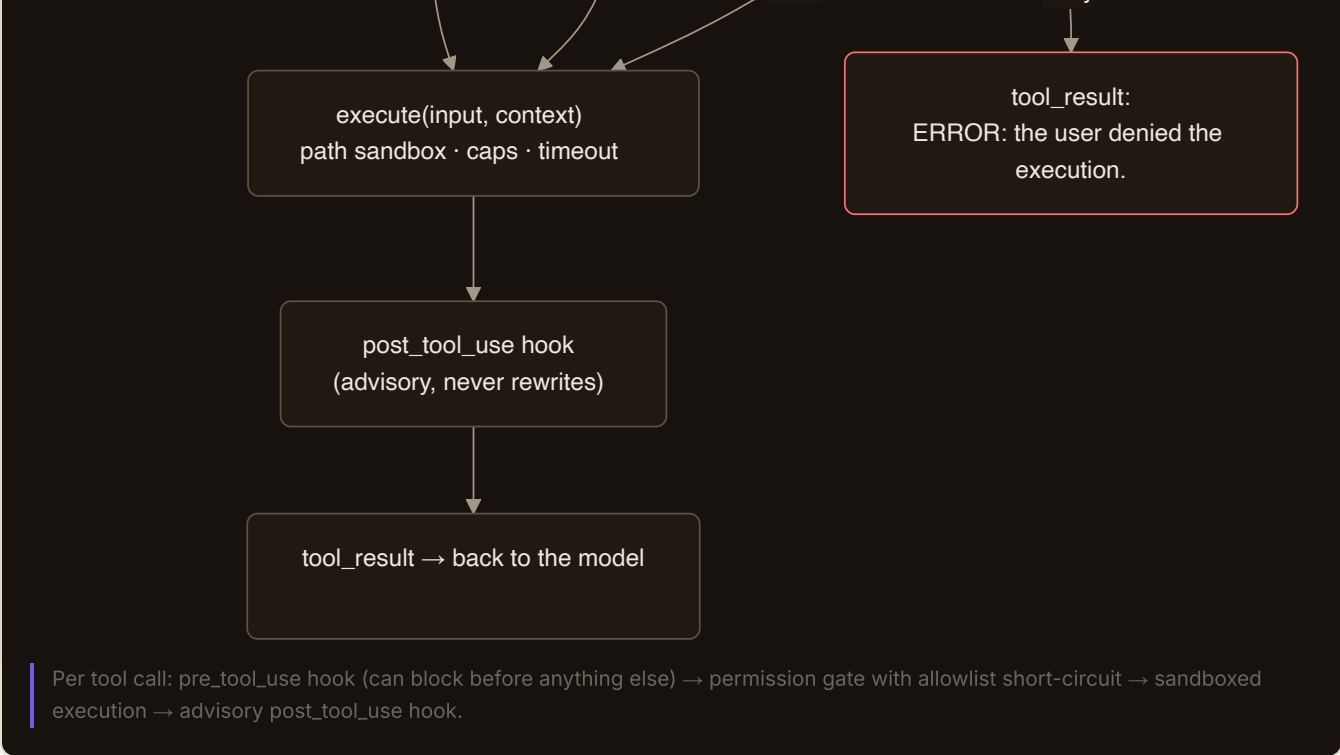
The WHO also states that 'the goal of ageing is to ensure that people are able to live longer, healthier, and more active lives'. The WHO also states that 'the goal of ageing is to ensure that people are able to live longer, healthier, and more active lives'. The WHO also states that 'the goal of ageing is to ensure that people are able to live longer, healthier, and more active lives'. The WHO also states that 'the goal of ageing is to ensure that people are able to live longer, healthier, and more active lives'.

The WHO also states that 'the goal of ageing is to ensure that people are able to live longer, healthier, and more active lives'. The WHO also states that 'the goal of ageing is to ensure that people are able to live longer, healthier, and more active lives'. The WHO also states that 'the goal of ageing is to ensure that people are able to live longer, healthier, and more active lives'. The WHO also states that 'the goal of ageing is to ensure that people are able to live longer, healthier, and more active lives'.

The WHO also states that 'the goal of ageing is to ensure that people are able to live longer, healthier, and more active lives'. The WHO also states that 'the goal of ageing is to ensure that people are able to live longer, healthier, and more active lives'. The WHO also states that 'the goal of ageing is to ensure that people are able to live longer, healthier, and more active lives'. The WHO also states that 'the goal of ageing is to ensure that people are able to live longer, healthier, and more active lives'.

The WHO also states that 'the goal of ageing is to ensure that people are able to live longer, healthier, and more active lives'. The WHO also states that 'the goal of ageing is to ensure that people are able to live longer, healthier, and more active lives'. The WHO also states that 'the goal of ageing is to ensure that people are able to live longer, healthier, and more active lives'. The WHO also states that 'the goal of ageing is to ensure that people are able to live longer, healthier, and more active lives'.





Permission modes

MODE	MEANING	WHERE
<code>ask</code>	gated tools pause the run until a human decides (terminal y/N, web dialog) — with the allowlist answering first	default; the interactive faces
<code>readonly</code>	every gated tool is denied automatically — the agent can look, not touch	headless default (<code>spectro run</code> , cron): “capability before convenience”
<code>auto</code>	every gated tool is approved automatically	opt-in for unattended runs: <code>--permissions auto</code> / per-job <code>"permissions": "auto"</code>

Even automatic policies emit `permission_request` / `permission_decision` events — the JSONL always shows who allowed what.

The allowlist

Rules live under `autoApprove` in any settings layer and are consulted before any human is asked:

```
{ "autoApprove": [  
  "grep",           // a bare name approves every call of that tool  
  "run_command:git status*", // a prefix rule approves matching inputs only  
  "write_file:docs/notes.md"  
] }
```

Matching reads each tool's **guarded field** — the one input that defines its blast radius. The same table drives what an “always allow” click remembers, so a remembered rule can never silently go dark on the matching side:

TOOL	GUARDED FIELD	AN “ALWAYS ALLOW” CLICK STORES
<code>run_command</code>	<code>command</code>	the first token: <code>run_command:git*</code>
<code>write_file</code> / <code>edit_file</code>	<code>path</code>	the full path: <code>edit_file:src/Main.java*</code>
<code>web_fetch</code>	<code>url</code>	the full URL: <code>web_fetch:https://example.com*</code>

Session-remembered rules live per connection (two tabs never share them). Ticking **Persist** appends the rule to the session's **workspace** project file (`<workspace>/spectro/settings.json`) through a single serialized writer that preserves every other key and dedupes — falling back to the deprecated `launch-dir/spectro/settings.json` only for a session with no real workspace yet. Because the CLI resolves the same workspace, it honours your web decision there on its next run.

Hooks

Hooks are config-driven shell commands around every tool call — your policy, in your scripting language, versioned with the project. They come *only* from config, never from tool input: a pre-hook is arbitrary shell running before the gate, so it must never be model-controlled.

```
{ "hooks": [
  { "event": "pre_tool_use",          // or post_tool_use - anything else fails LOUDLY
    "matcher": "run_command",        // tool-name glob: * (default), exact, or prefix*
    "command": "./guard.sh",         // any shell string
    "timeoutSeconds": 5 }             // optional, default 10
] }
```

ENVIRONMENT	the hook process sees <code>SPECTRO_TOOL_NAME</code> , <code>SPECTRO_TOOL_INPUT</code> (compact JSON) and — post only — <code>SPECTRO_TOOL_RESULT</code> ; cwd is the agent's working directory.
PRE_TOOL_USE	runs before the permission gate and blocks on <code>exit ≠ 0</code> or a stdout verdict <code>{"decision":"block","reason":"..."}</code> . A block short-circuits: no dialog, no execution — the model receives <code>ERROR: blocked by pre_tool_use hook: <reason></code> . First matching block wins.
POST_TOOL_USE	runs after execution, advisory only — exit code ignored, the result is never rewritten. For logging, notifications, formatters.
FAIL-OPEN TIMEOUT	a hanging hook must not wedge every tool call — on timeout the pipeline continues to the normal gate. Safe because the shared runner drains the pipe concurrently: a chatty hook cannot deadlock itself into the timeout.
SUBAGENTS INCLUDED	children run the <i>same</i> hook runner — a hook that blocks a tool also blocks it on a child, or delegation would be a bypass.
WHOLE-BLOCK MERGE	like <code>mcpServers</code> : a settings layer that defines <code>hooks</code> replaces the entire block below it.

EXAMPLE — VETO EVERY RM

In `.spectro/settings.json`:

```
{ "hooks": [ {
  "event": "pre_tool_use", "matcher": "run_command",
  "command": "case \"${SPECTRO_TOOL_INPUT}\" in *'rm '* ) echo '{\"decision\": \"block\", \"reason\": \"rm is off limits here\"}' ;; esac"
} ] }
```

Ask the agent to delete a file with `rm` and it instantly receives

`ERROR: blocked by pre_tool_use hook: rm is off limits here` — no permission dialog, no execution. `spectro doctor` shows `hooks: 1 configured (pre_tool_use:run_command)`. Verified live.

The audit trail

Every decision is an event: `permission_request` (with the full input) and `permission_decision` (allowed or not) land in the JSONL even for allowlist auto-approvals; hook blocks are visible as their error result. The Trace tab's permission category shows the whole history; the Lab's gate row turns green or red as you step onto it.

Subagents & the A2A protocol

A subagent is not a special construct — it is another `Agent` from the same core, with its own fresh context, its own reduced tool registry, and the same permission gate and hooks as its parent. What makes spectroscope's delegation special is that the protocol between the agents is *visible*: a task/status/result lifecycle you can watch in every view.

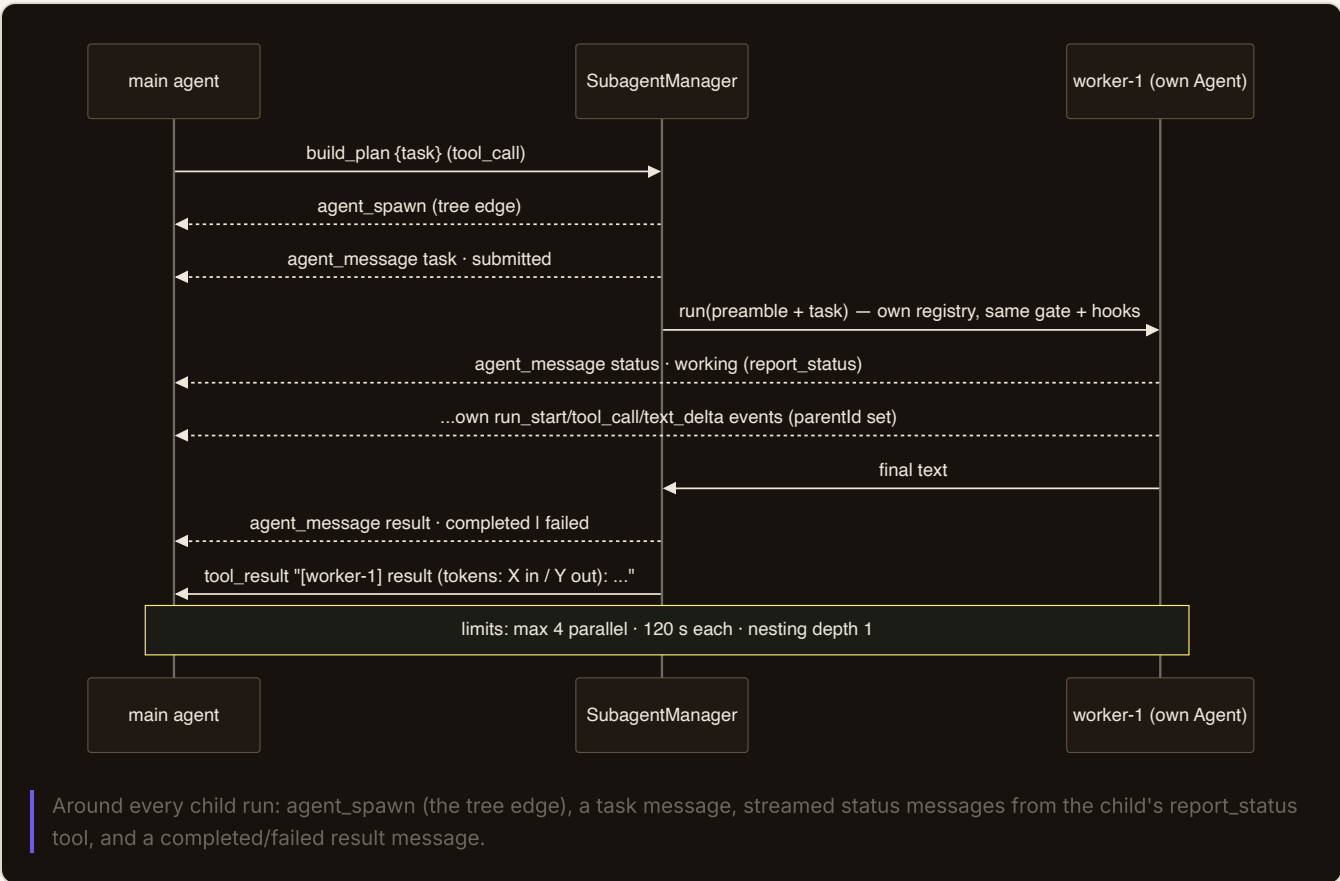
Two roles

ROLE	PURPOSE	TOOLS
<code>explorer</code>	read-only research: map a directory, find usages, summarize	<code>list_dir</code> , <code>read_file</code> , <code>glob</code> , <code>grep</code> + <code>report_status</code> — read-only by construction
<code>worker</code>	real subtasks that may change things	the full standard set + <code>use_skill</code> + <code>report_status</code>

Children see *only their task text* — no conversation history — which is the point: a fresh, focused context. Their final text is all the parent receives, wrapped with the token cost so delegation visibly costs something: `[worker-1] result (tokens: 2.1k in / 0.4k out): ...`

LIMITS	at most 4 parallel children (schema-enforced and runtime-checked); 120 s per child (then <code>ERROR: [worker-1] timeout after 120 s - cut the subtask smaller.)</code> ; nesting depth 1 — children simply do not have the spawn tools.
CANCELLATION	each child has its own <code>CancelSignal</code> ; the parent's cancel cascades — <code>Ctrl+C</code> or <code>Stop</code> ends the whole tree cleanly.
ISOLATION THAT ISN'T	same working-directory sandbox, same permission broker (the dialog names the asking child), same hooks. Delegation never widens what is allowed.

The A2A-lite lifecycle



Two additive events carry the protocol — the wire format needed no break. Every spawn emits `agent_spawn` (child id, parent id, task) followed by an `agent_message` with role `task`, state `submitted`. While working, the child's permission-free `report_status` tool publishes role-`status` messages ("working") that surface live in the Agents panel, the chat thread header and the Lab cards. At the end — on *every* path, including timeout and cancellation — a role-`result` message closes the child with state `completed` or `failed`.

Where you see it: chat nests child turns into pastel threads (chapter 5); the Graph tab draws split/join blocks per wave (chapter 6); the Lab gives every child its own loop card with its own packet (chapter 7); the Agents panel keeps the roster with token costs (chapter 6); the CLI renders `[worker-1]`-prefixed lines (chapter 4).

The development tools

Four role tools wrap a worker spawn with a persona and a skill — no new agent types; specialization is prompt + skill:

TOOL	PERSONA	LOADS SKILL	DELEGATES
<code>build_plan</code>	senior PLANNER	<code>writing-plans</code>	a feature request → a step-by-step implementation plan (a written document)
<code>write_spec</code>	requirements ANALYST	<code>brainstorming</code>	a rough idea → a design/spec with decisions and trade-offs
<code>develop</code>	IMPLEMENTER	<code>test-driven-development</code>	a development task, carried out in small verified steps

test

TESTER — “Do NOT fix anything.”

verification

runs and inspects, reports evidence, changes nothing

The child's prompt is the persona preamble + a standing instruction to report progress via `report_status` + your task text; the A2A task message shows *your* task with the tool name as its label — which is exactly how the UI badges the thread. Try: *“build_plan: draft a plan for adding CSV export to the sessions list.”*

Skills

Skills extend the agent with *data, not code*: markdown instruction packages the agent loads when a task matches. The loop is untouched, no new event type exists — a skill use is an ordinary tool call you can see in every view.

The package format

```
~/spectro/skills/<folder>/SKILL.md      (user scope)
<project>/spectro/skills/<folder>/SKILL.md  (project scope – wins by name)

---
name: test-driven-development
description: Red-green-refactor discipline for every change - write the failing
  test first, watch it fail for the right reason, make it pass minimally, then clean up.
---
# The skill body – full markdown instructions...
```

Frontmatter is hand-parsed (`name:` , `description:`); a file without it falls back to the folder name and the first body line. Broken skills are skipped with a warning — they never take the harness down.

Progressive disclosure

Only the *catalog* — one `name: description` bullet per skill — rides in the system prompt under a `## Skills` header, with the standing instruction to call `use_skill` before matching work. The body loads on demand. That keeps the context small however many skills you install, and makes skill usage visible: you can watch the agent fetch its instructions in the chat and trace.

The four shipped skills

SKILL	TEACHES
brainstorming	turn a vague idea into an agreed design before code: one question per turn, real alternatives, decisions recorded
test-driven-development	red → green → refactor with honest verification rules (no network in tests, one behavior per test)
writing-plans	plans a fresh engineer could execute: exact files, verifiable steps, no open questions
verification	evidence before claims — never state what you did not observe in this run; report VERIFIED / FAILED / UNVERIFIABLE per claim

`/skills` lists what is installed; `spectro doctor` counts them; the System-context panel shows the catalog exactly as the model sees it. The four development tools of chapter 14 point their children at these skills.

the fleet: many agents, one spectrum

Subagents fan out under one main agent. The fleet is the next level: several full agents, each with its own model, tools and workspace, run in parallel and merge into one spectrum. They can share a single JVM, or run as separate processes on separate machines that report to a hub. Either way you enter a fleet like a session and watch it on the lenses you already know. That is the `spectro-orchestrator` module (migration phase 5), and its facade follows the frozen five-lines style exactly:

```
var panel = Spectro.panel().model(Anthropic.opus());
panel.agent("bugs").task("Find bugs in the diff");
panel.agent("perf").task("Check the hot queries");

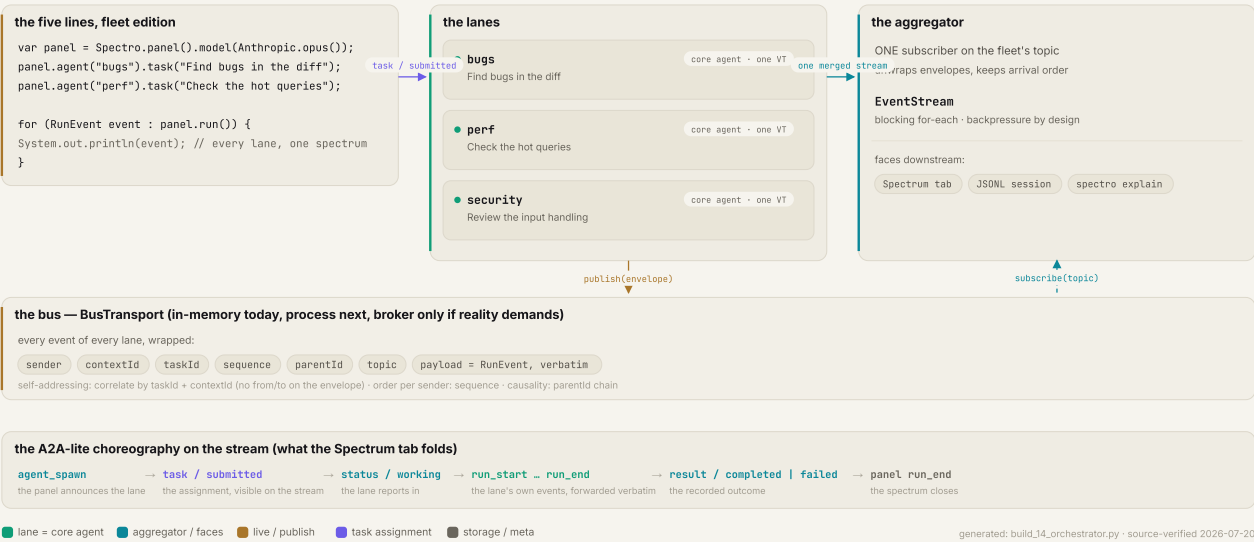
for (RunEvent event : panel.run()) {
    System.out.println(event);    // every lane, one spectrum
}
```

Every lane is a complete core agent on its own virtual thread. The panel announces each lane on the stream (`agent_spawn`, then a task message), the lane reports in with a working status, its whole event stream rides along verbatim, and a result message records completed or failed before the panel's own `run_end` closes the spectrum. One broken lane never kills the fleet; it fails loudly and alone.

SPECTRO-ORCHESTRATOR

the fleet: agents on a bus, one spectrum

`Spectro.panel()` runs every lane as a full core agent on its own virtual thread. Every event rides the bus as an envelope; ONE aggregator drains the fleet's topic into one `EventStream` — the same blocking for-loop as a single agent.



The code fleet: every lane a full core agent on its own virtual thread, every event wrapped in a bus envelope, one aggregator draining the topic into one `EventStream`, the same blocking for-loop as a single agent.

what rides the bus

Under the surface, every event of every lane is wrapped in a bus envelope: sender, contextId, taskId, a per-sender sequence, a causal parentId chain, a topic for session isolation, and the `RunEvent` itself, verbatim. Addressing is self-addressing: consumers correlate by taskId within a contextId instead of

from/to routing. The transport behind it is a seam. In-memory serves a fleet inside one JVM; a process transport (`ProcessBus` over a small TCP hub) carries a fleet spread across processes and machines. The envelopes and their consumers are identical either way, so nothing downstream can tell the two apart.

WHERE IT DOCKS

The bus publishes through the same tracing seam the JSONL sink uses: one registry in the core where the session file and the bus publisher hang side by side, durability first (JSONL before bus). A node's local session file is always the anchor; the bus is what makes it visible to the rest of the fleet.

spectro node: the process fleet

A fleet does not have to live in one program. The launcher runs one prompt as a fleet node whose whole event stream rides the bus to a hub:

```
spectro node -p "Review the input handling" \  
  --hub 127.0.0.1:7700 --context pr-42 --role security
```

The node runs headless, exactly like `spectro run`, and publishes as it goes. `--context` names the fleet to join (the bus topic derives from it) and is required; `--hub host:port` (or `$SPECTRO_HUB`) names the hub. Options fill in the rest: `--id` (default `node-<pid>`), `--role` (default `worker`), `--permissions` (`readonly` by default, `auto`, or `ask`), `--max-turns`, and `--linger`. The local session JSONL stays the durability anchor, so a dead or stalled hub never blocks the run: events past the buffers are dropped from the *bus view* only, counted and warned loudly, while at-least-once still holds for everything the outbox accepted. Each process start stamps a fresh wall-clock epoch, so a restarted node reads as a new stream on the bus, delivered rather than misread as a redelivery.

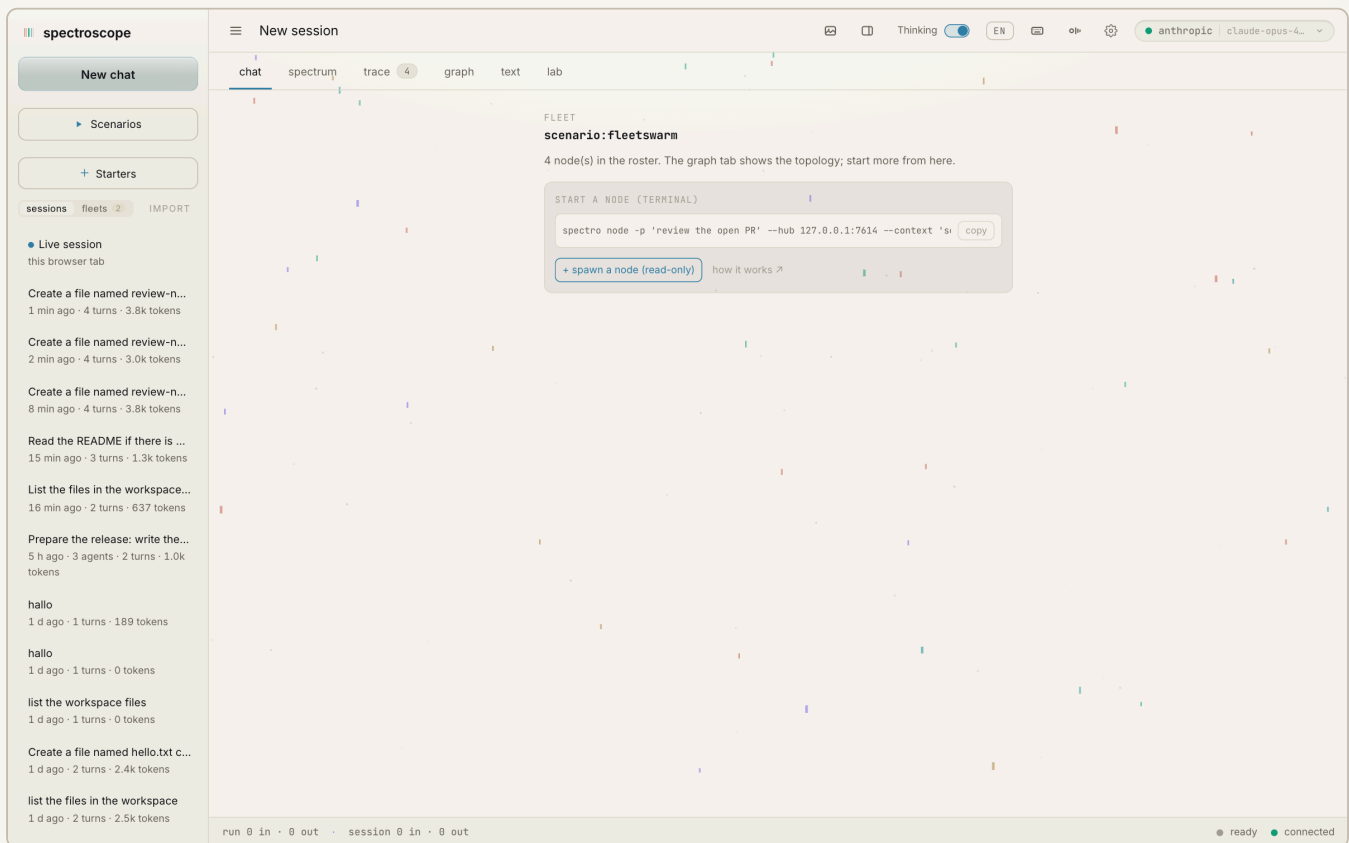
By default a node connects, runs once, and exits. `--linger` keeps it connected and controllable after the run, until a stop arrives over the hub (or, in production, `SIGTERM`). `--permissions ask` parks each gated tool for an operator to answer over the hub instead of a fixed yes or no (see the control plane below).

code fleets on the hub

A `Spectro.panel()` fleet runs in one JVM by default and returns one merged `EventStream`. Set `$SPECTRO_HUB` (or build the panel with a hub address) and each lane also mirrors its frames to the hub as a node, so a code fleet becomes live-visible in a running cockpit and the canvas draws its real spawn edges. The invariant holds where it matters: the returned `EventStream` stays byte-identical on the local in-memory spine, the hub is a purely additive mirror, and a dead hub never delays the run's close.

entering a fleet

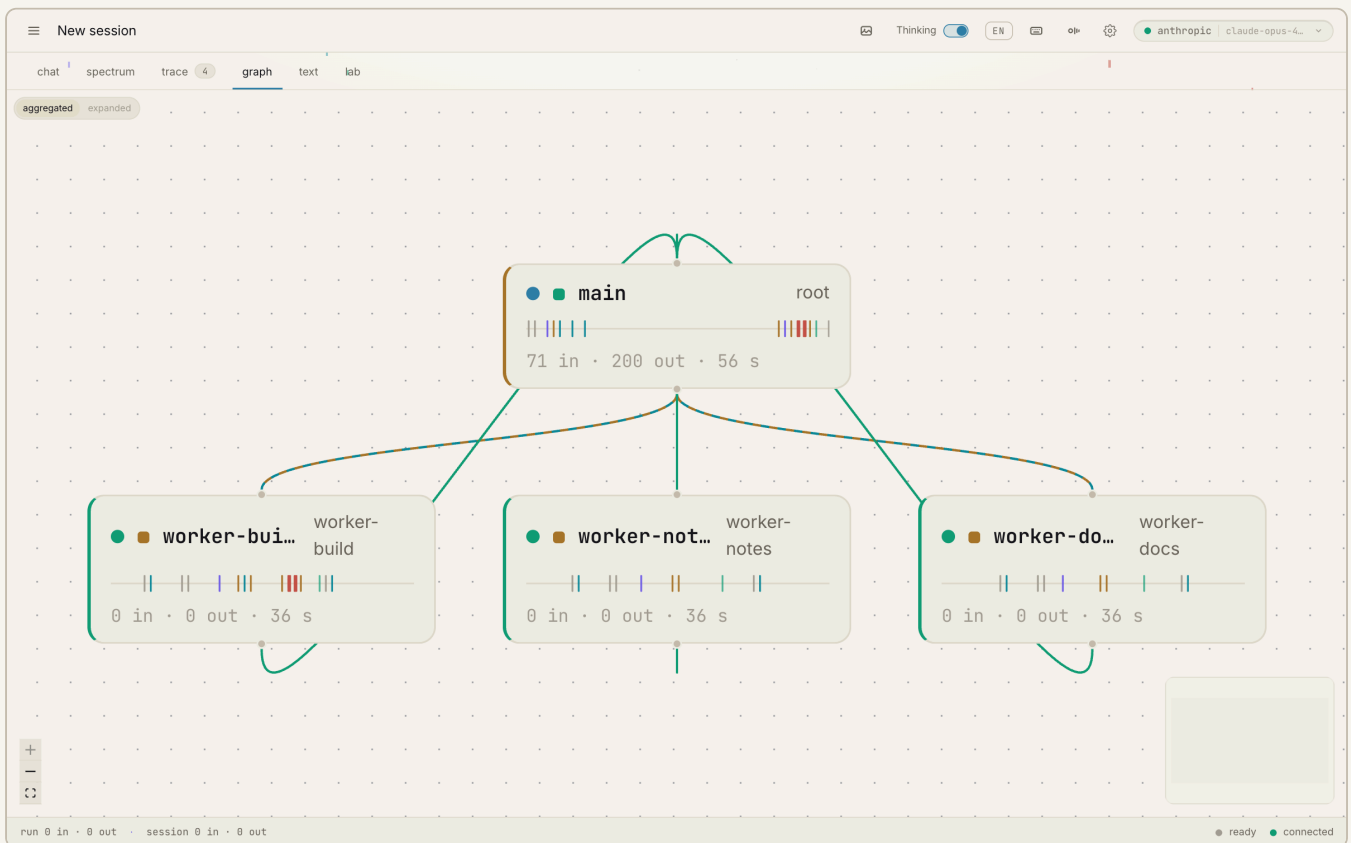
Fleets are first-class, like sessions. The sidebar has two segments, `sessions` and `fleets`; switch to fleets and every one the hub has seen is a row with its sigil, its name, and its agent count (online of total). A fleet with a parked gate floats to the top and wears a pulsing `gate open` chip. Click a row to enter the fleet: its events feed the same tabs. The chat tab becomes the fleet's home — a short getting-started panel with the spawn form, instead of a stale session chat — and the Spectrum shows every agent at once, one lane each. A browser opened after the fact is not blind to history: entering pulls each node's recent frames from the hub's replay ring, so a gate parked before you arrived is still there waiting. The ring is bounded and the view says so; the node's own JSONL stays the full record.



An entered fleet: the fleets segment in the sidebar, the fleet's sigil and counts, and the fleet home with its getting-started panel and spawn form.

No hub at hand? The scenario dialog has a **fleet** tab with scripted fleet runs — deterministic, no LLM, no key — that load as a replay fleet and light up every fleet view, including the canvas and the machine room below. That is the fastest way to see the whole surface before wiring a real hub.

The graph tab, for an entered fleet, becomes the fleet canvas: a spectral DAG. One node per agent (never one per event), its body a compact spectral band folded from that agent's slice of the stream, plus an activity span that answers "who was slow" at a glance; causal rails wire the topology, read from the event payloads (spawn ocean, task an amber comet in flight, result teal). Click an agent to open its own trace. A wide fan-out of same-role siblings folds into one group card (marked with a count) — an **aggregated · expanded** toggle in the corner switches between the folded reading and every card on its own — and the canvas shrinks node bodies as the fleet grows, so a large fleet stays readable rather than turning to hairball. A MiniMap and pan-zoom come with it. Export a fleet run to JSONL and the import dialog replays the whole thing, exactly as recorded.



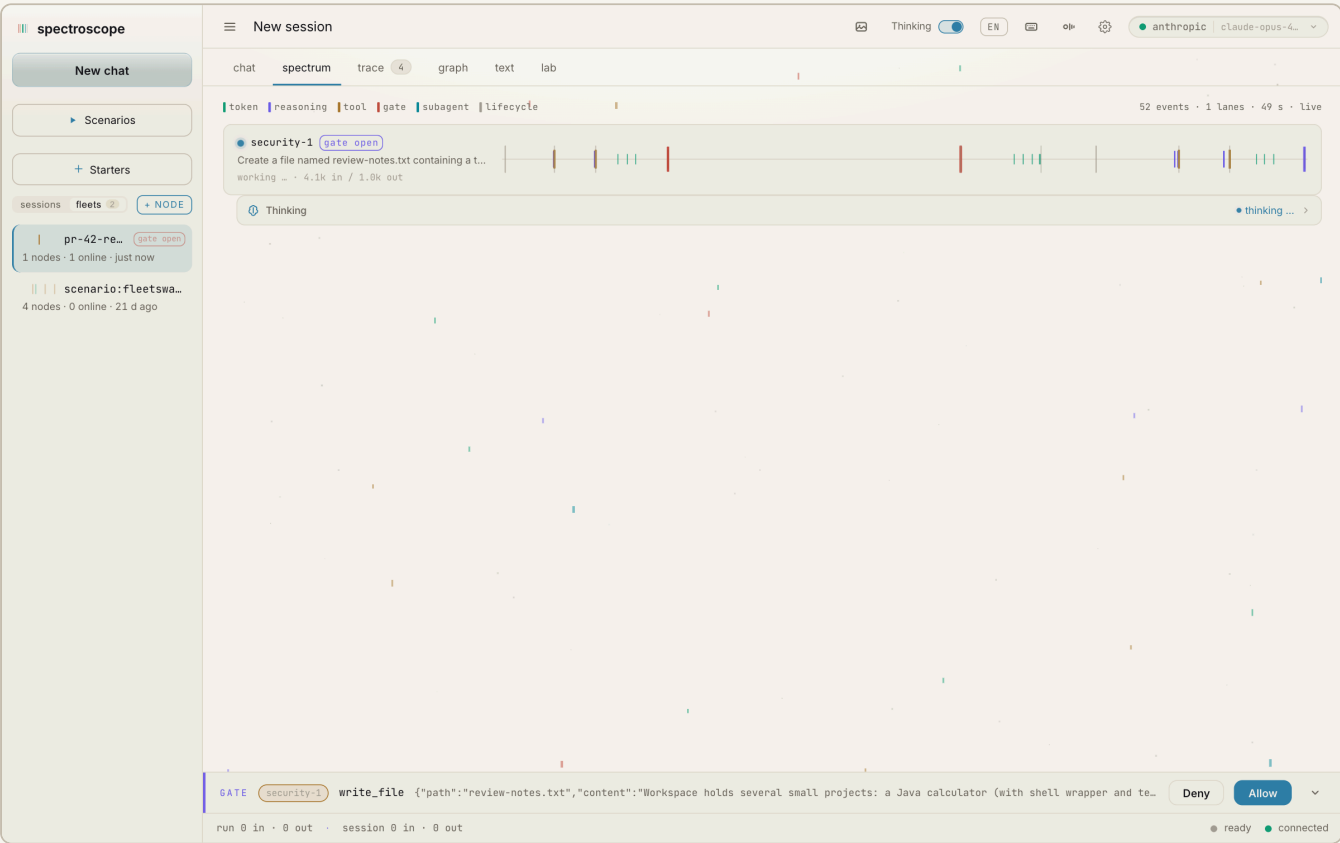
The fleet canvas: one spectral node per agent, spawn and task rails between them, the aggregated · expanded toggle, and the MiniMap.

the machine room

The lab tab of an entered fleet is the machine room: the whole fleet as ONE agent-system diagram, drawn with the same pieces as the single-run map. Every node is its own small agent loop — task, lifecycle state, gate color, and the exact thing it is doing right now — and they all share one machine: the OS band (disk, shell, MCP chain) lights up for whichever node is on it, and packets ride the rails exactly as in the single-run lab.

The LLM stations are honest about locality. Nodes on a remote provider rail across the network boundary to a station outside; a node on Ollama rides to a local station inside the machine. A mixed fleet shows both stations with each node wired to its own; a purely local fleet shows no remote box at all, because there is no traffic that would cross the boundary. A process fleet without a `main` node draws no user station either — nothing enters from a person there, so nothing pretends to.

inline and the answer travels to the node over the hub. There is no "remember" here: a remote node keeps its own allowlist, not yours. Like stop it is best-effort; if the node left first, its own close denies the orphaned gate, so the gate clears either way.



A real parked gate: the security node waits at its write_file, the lane and the sidebar row wear the gate-open chip, and the gate bar offers deny or allow — the answer rides the hub.

WHO CAN DRIVE

Every fleet write, and the roster read itself, is gated to a local origin: a loopback caller *and* a localhost Host header (the Host check is the DNS-rebinding defense). A remote or rebound page gets a 404 with no hint the endpoint exists. Spawn adds its opt-in flag and forces the node read-only on top. The bus carries prompts, reasoning and tool I/O, so its face is treated as sensitively as the ability to spawn.

Providers & models

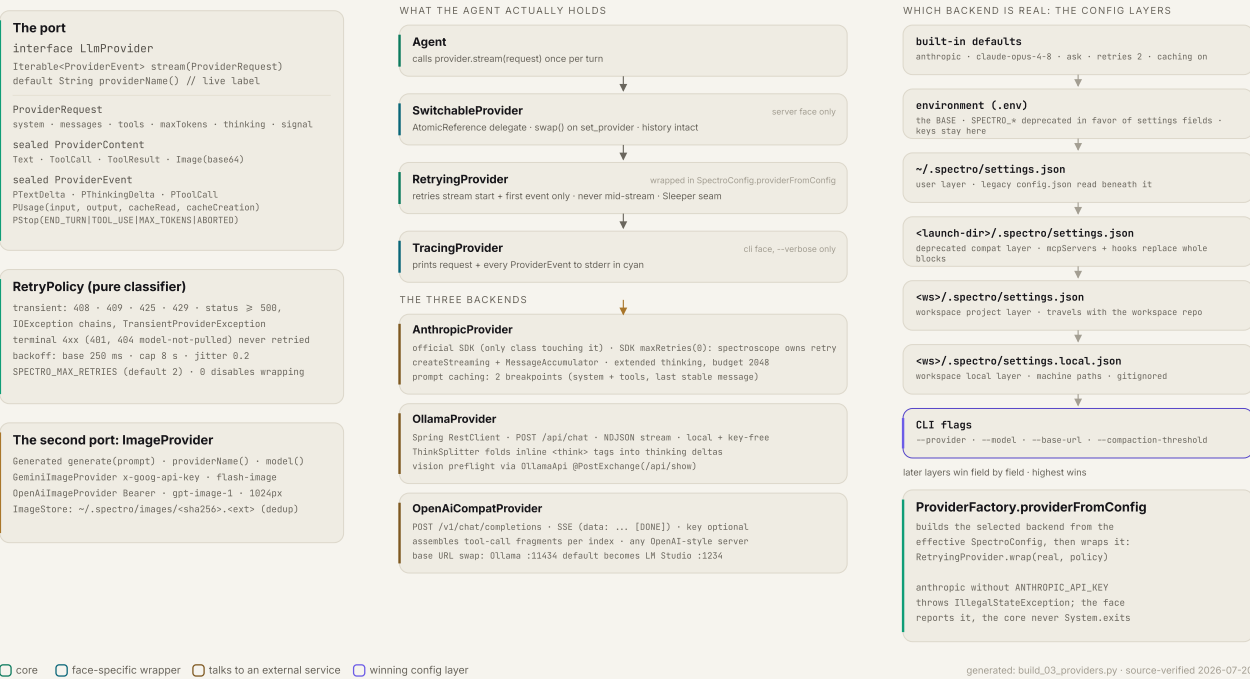
Everything the loop needs from a model sits behind one narrow port:

`Iterable<ProviderEvent> stream(request)`. Swap the implementation and the loop never notices, which is why spectroscope can switch between a cloud API and a local model mid-session.

SPECTROSCOPE · ARCHITECTURE DOSSIER · 03

The provider port.

Everything the loop needs from a model sits behind LlmProvider. Wrappers add runtime switching and retries without the loop noticing; the config layers decide which backend is real.



generated: build_03_providers.py · source-verified 2026-07-20

One port, many backends: the loop only ever sees `stream(request)`. Anthropic, Ollama and the OpenAI-compatible family each implement it; the retry decorator wraps whichever one you picked.

The backends

Five provider ids resolve to three implementations. The three OpenAI-compatible ids (`openai`, `lmstudio`, `openrouter`) share one wire protocol and differ only in which host they talk to and whether a key is attached.

	ANTHROPIC	OLLAMA	OPENAI-COMPATIBLE
provider id	<code>anthropic</code>	<code>ollama</code>	<code>openai</code> · <code>lmstudio</code> · <code>openrouter</code>
implementation	<code>AnthropicProvider</code>	<code>OllamaProvider</code>	<code>OpenAiCompatProvider</code>
transport	official SDK, SSE	NDJSON over <code>POST /api/chat</code>	SSE over <code>POST /v1/chat/completions</code>

endpoint	api.anthropic.com (fixed)	localhost:11434 (default)	openai → api.openai.com ; lmstudio → localhost:1234 ; openrouter → openrouter.ai/api (a set baseUrl overrides the preset)
key	ANTHROPIC_API_KEY	none	OPENAI_API_KEY (openai) · none (lmstudio) · OPENROUTER_API_KEY (openrouter)
default model	claude-opus-4-8	qwen3	name one for the cloud ids; a local server just serves the model it has loaded
thinking	extended thinking (2048- token budget)	native message.thinking + inline <think> tag splitting	reasoning_content deltas split out (LM Studio, vLLM)
quirks	usage + parsed tool inputs exist only after the stream ends	mints its own call ids; vision fail-fast for non- vision models	assembles fragmented tool-call deltas per index

One wire, three hosts, no guessing. spectroscope used to redirect a keyless openai to a local LM Studio endpoint on your behalf. That guess is gone: the provider id names the endpoint, the key only authenticates. openai now always means the cloud; for a local OpenAI-compatible server (LM Studio, llama.cpp, vLLM) pick lmstudio or set an explicit baseUrl. Same wire either way.

First run — choosing a backend

Out of the box the provider is anthropic with claude-opus-4-8, which needs a key. On its first boot every face (spectro run, the web chat, spectro node, spectro doctor, the server) checks whether a ~/.spectro/settings.json exists yet; when none does, it writes one from your current environment base (whatever you exported, or put in ./.env), so day one your settings file mirrors exactly what you already had and nothing changes behind your back. With nothing set at all you land on the Anthropic default, and a run without a key stops with a readable “no key” message instead of a mystery failure.

No Anthropic key on hand? Two of the backends are keyless and local:

- **ollama** — install it, pull a model, and it serves localhost:11434. Set provider=ollama; no key.
- **lmstudio** — run LM Studio in server mode with a model loaded, and it serves localhost:1234. Set provider=lmstudio; no key.

The two cloud OpenAI-compatible backends do need a key: openai (OPENAI_API_KEY) and openrouter (OPENROUTER_API_KEY). Set the provider in ~/.spectro/settings.json, through SPECTRO_PROVIDER, with a --provider flag, or from the header picker. The CLI tour is the guided path: spectro tour → Settings [s] picks a provider, takes the key hidden, and can save it to ./.env for you.

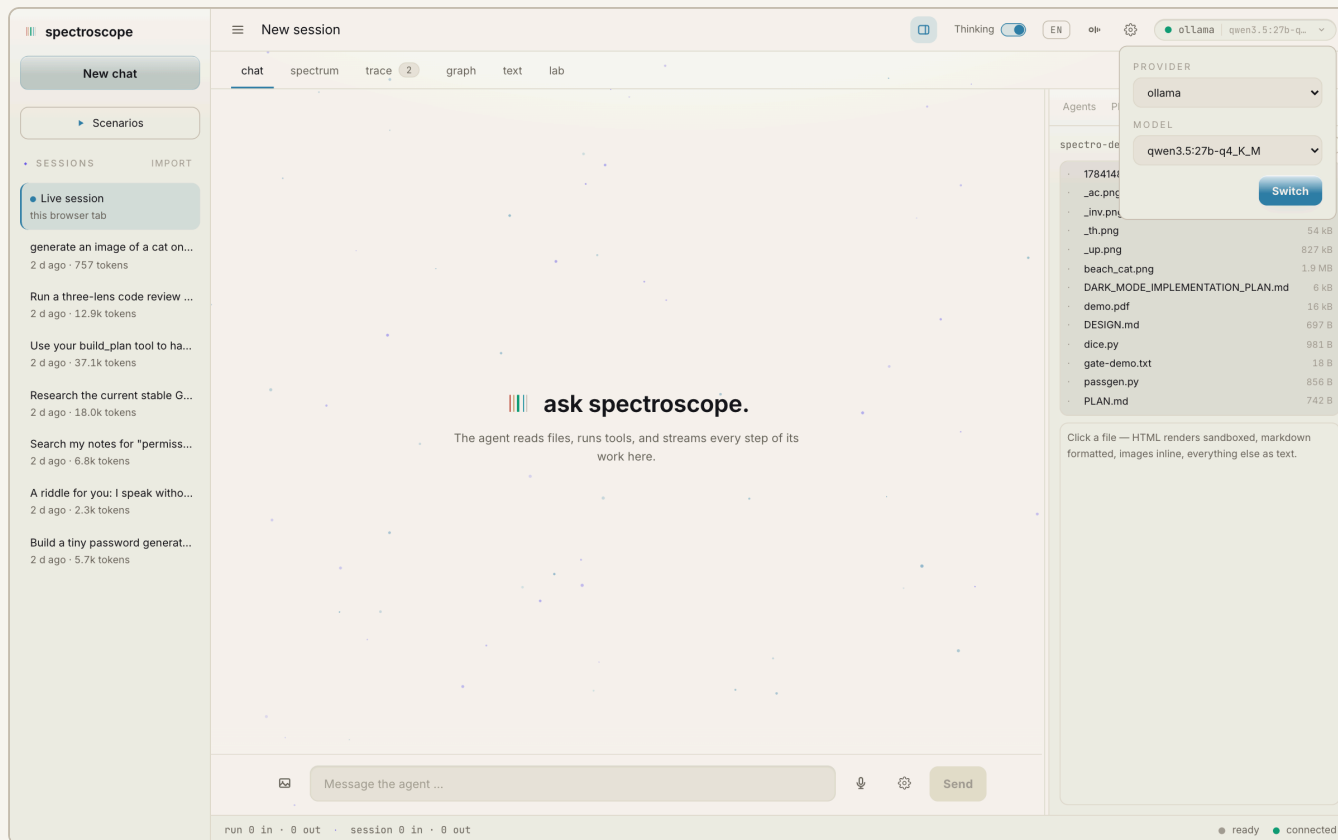
Putting a key in the .env

The simplest home for a secret is a ./.env file in the repo root: it is gitignored, and spectroscope reads it as the environment base at startup. One KEY=value per line, no quotes:

```
# ./.env - gitignored; the environment base layer
ANTHROPIC_API_KEY=sk-ant-...      # anthropic
OPENAI_API_KEY=sk-...             # openai (the cloud); also the image backend's key
OPENROUTER_API_KEY=sk-or-...     # openrouter
# ollama and lmstudio need no key
```

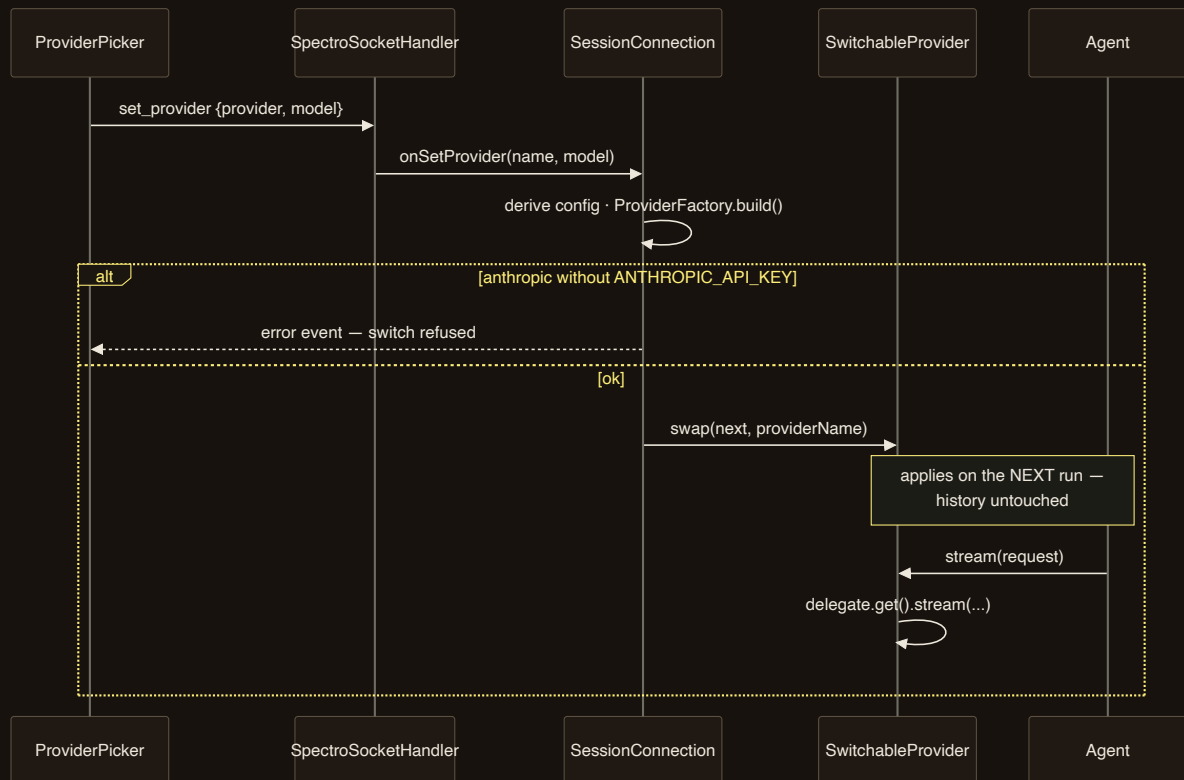
The `.env` is only the *base* layer: a value in `~/.spectro/settings.json` (or a flag) wins over it, and `spectro doctor` names any `SPECTRO_*` variable that is set *and* shadowed, so a value that isn't taking effect never fools you. Keys are secrets: they never enter a session file, the event stream, or the first-run settings seed. In the tour, `[w]` “save key to `.env`” writes the file for you (gitignored, mode 600). The image-generation keys (`GEMINI_API_KEY` , `OPENAI_API_KEY`) live in the same file — chapter 17.

Switching mid-session



The header picker: provider select plus a real model dropdown — Ollama's list is live from the server, cloud lists are curated, and a custom model is always typeable.

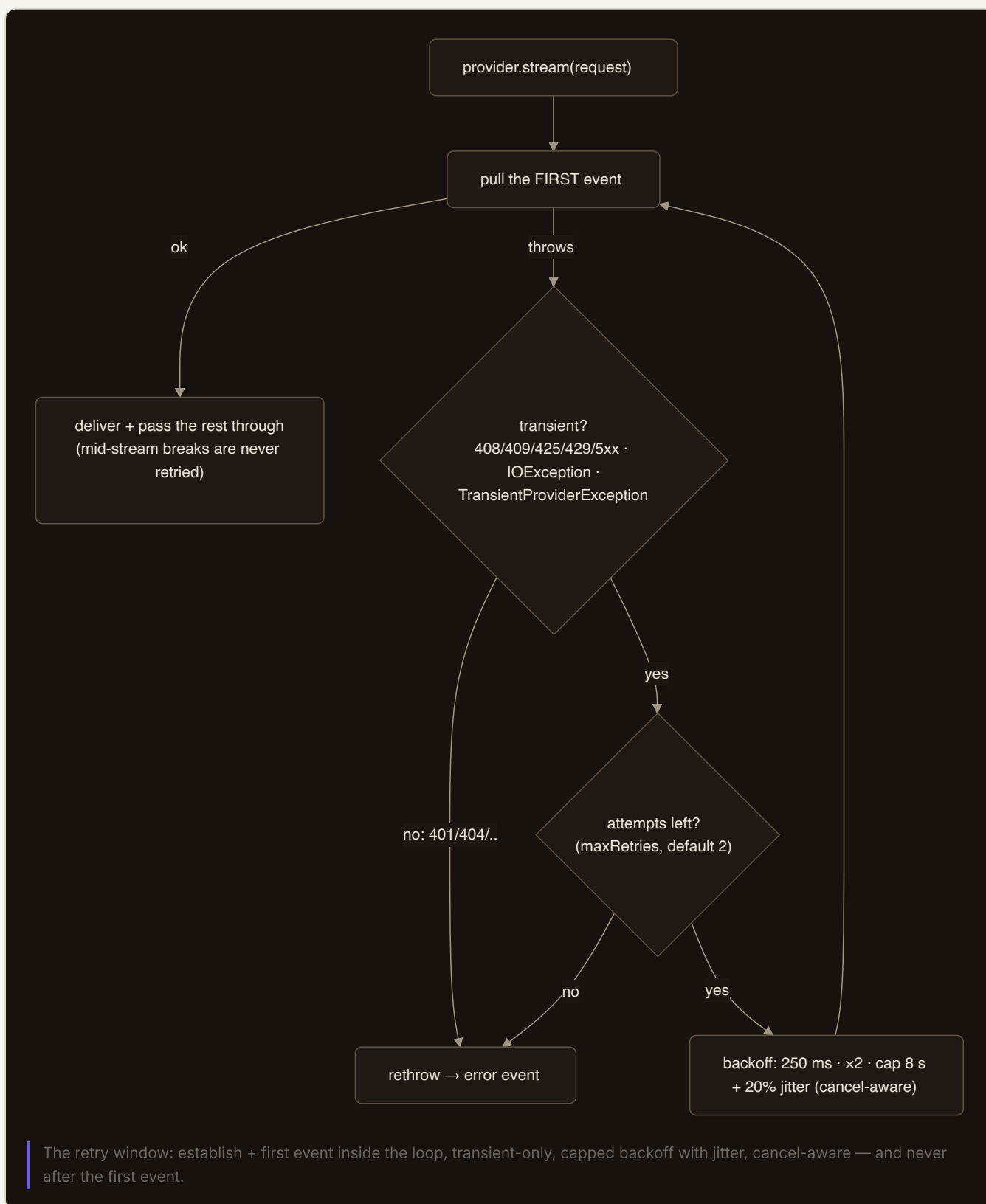
The header chip opens the picker. It switches the headline backends (anthropic, ollama, openai) and the model; the two extra OpenAI-compatible ids (lmstudio, openrouter) are chosen in settings (`SPECTRO_PROVIDER` or `~/.spectro/settings.json`) or with a `--provider` flag. The model dropdown is honest: for Ollama it lists what is actually installed (live from `/api/tags`), for Anthropic what your key can use (its Models API), for an OpenAI-compatible endpoint whatever `/v1/models` returns, each with a curated fallback and “Custom model ...” always available; an empty list degrades to free text. Apply sends `set_provider` , and because the agent is built once per connection (it carries your history), the switch swaps a delegate *inside* the provider port instead of rebuilding anything. It takes effect on the next prompt, history intact. Selecting Anthropic without a key is refused with a readable error, and the map in the Lab replaces the LLM node inside or beyond the network boundary to match.



set_provider: validate, build the new backend (key check included), swap the delegate. The next run_start reports the new provider — no new event type needed.

Retry — a loop that doesn't tip over

Transient failures (HTTP 408/409/425/429, any 5xx, dropped connections) are retried with exponential backoff and jitter — but **only before the first event** of a response. A stream that breaks mid-answer is never retried (that would duplicate already-emitted text into the transcript); it surfaces as an honest error. Terminal errors — 404 model-not-pulled, 401 bad key — never retry. spectroscope owns retry exclusively: the Anthropic SDK's own retry is disabled so nothing double-retries. Knobs: `maxRetries` (default 2, `SPECTRO_MAX_RETRIES` ; 0 disables).



Prompt caching (Anthropic)

With `promptCaching` on (default), spectroscope places two `cache_control` breakpoints per request: one covering the stable prefix (system prompt + tool list), one on the last *stable* message — the one just before the current turn. Costs and latency drop on every consecutive turn. The wire stays honest: the `usage` event reports the provider's raw token counts, while the cache-read/-creation counts are folded only into the *compaction trigger* — cached tokens still occupy the context window, so the threshold must see them (chapter 28).

The wrapper chain

```
Agent loop
└─ TracingProvider      (CLI --verbose only: the wire on stderr)
   └─ SwitchableProvider (web face only: the mid-session swap point)
      └─ RetryingProvider (all faces, unless maxRetries=0)
         └─ AnthropicProvider | OllamaProvider | OpenAiCompatProvider
```

All wrapping happens at one chokepoint (`providerFromConfig()`), so the CLI, the server, headless runs and the scheduler behave identically. The one `OpenAiCompatProvider` serves all three OpenAI-compatible ids — only its endpoint and key differ.

Seeing, hearing, speaking, painting

Four bonus-stage senses, all built on the same additive principles: binary data never enters the event stream, external engines sit behind injectable seams, and everything degrades readably when a piece is missing.

Vision — image input

- **Web:** attach images in the composer (picker or drag-and-drop) — they are downscaled client-side, stored content-addressed in the session's blob folder, and restored *before* the prompt text in the provider history (the position matters to the model).
- **CLI:** `spectro run -p "..." --image photo.png` (repeatable).
- **Local models fail fast:** Ollama models without the vision capability are refused with an actionable message ("use qwen3-vl or llava") before any tokens are wasted; cloud models handle images natively.
- Events carry only references (`blobPath` , `sha256`) — session files stay small and `jq`-able; a deleted blob degrades to a placeholder, never a crash.

Voice input — local push-to-talk

One-time setup: `bash scripts/setup-stt.sh` — installs whisper.cpp and ffmpeg (Homebrew on macOS) and downloads the ggml-small model (~460 MB) with a **pinned SHA-256**: a mismatch deletes the file and aborts. Recording is 16 kHz mono WAV (whisper's required format); transcription runs with automatic language detection, so German dictation is not forced through English.

- **REPL** `/voice` : record until Enter, transcribe, then an *editable confirmation* — Enter sends, typing replaces, blank discards. An STT error never reaches the agent unreviewed. The turn is audited by a `voice_input` event that never enters the provider history.
- **Web mic button:** records in the browser, transcribes via `POST /api/transcribe` (the server reuses the same Transcriber), and drops the text into the composer draft. STT not installed → the endpoint answers 503 and the button disables itself with the setup hint.

Voice output — spoken answers

Setup: `bash scripts/setup-tts.sh` (piper + the `en_US-lessac-medium` voice). Enable per run with `--speak` , at runtime with `/speak on|off` , or permanently via `"tts": { "enabled": true }` in `~/.spectro/settings.json` (the legacy `config.json` name is still read beneath it for one release — chapter 20). The renderer buffers streamed text to sentence boundaries and speaks *while the rest still streams* (synthesis runs up to three sentences ahead of strictly-ordered playback). Code blocks are never read aloud — you hear "Code block skipped." Abort stops mid-sentence with no ghost audio. Ollama + piper is a fully offline agent that talks.

Image generation

The `generate_image` tool sits behind its own provider port with two backends — Gemini (`gemini-2.5-flash-image` , `GEMINI_API_KEY`) and OpenAI (`gpt-image-1` , `OPENAI_API_KEY`) — selected by `imageProvider` / `SPECTRO_IMAGE_PROVIDER` or live from the gallery's dropdown. Results land content-addressed under `~/.spectro/images/` (same bytes = same file, shared across sessions); the additive `image_generated` event feeds the gallery panel; the model itself only sees a short confirmation string. Gated, because it is a paid cloud call. A missing key returns a readable `ERROR:` the model can relay — never a crash.

MCP — external tool servers

spectroscope is an MCP client: plug in any Model-Context-Protocol server and its tools become first-class spectroscope tools — gated, traced, replayable. No new event type was needed: MCP calls are ordinary `tool_call` / `tool_result` lines, so graph, trace and archive render them for free.

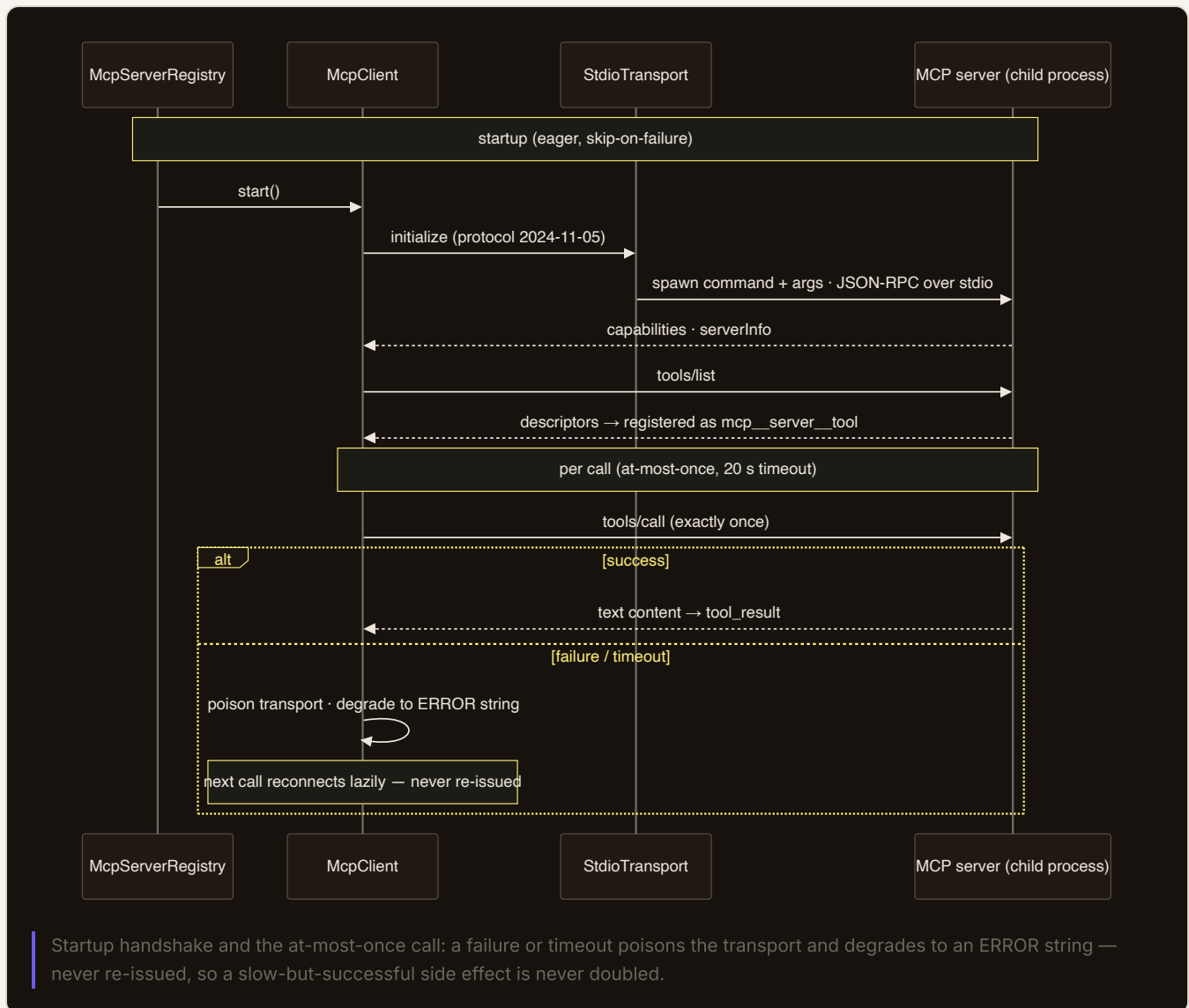
Configuration

The `mcpServers` block has the same shape Claude Desktop and Claude Code use — a config written for those drops straight in:

```
{ "mcpServers": {  
  "notes": {                                // stdio: command (+ args, env)  
    "command": "/path/to/spectro-mcp-notes/bin/spectro-mcp-notes",  
    "args": ["/path/to/notes-directory"]  
  },  
  "search": { "url": "http://localhost:9000/mcp", "type": "sse" } // or HTTP/SSE  
}
```

Servers connect eagerly at startup; one that fails to start is logged and **skipped** — a broken entry never takes the harness down. Each advertised tool registers as `mcp__<server>__<tool>`, always permission-gated: its inputs are model output and its effects are external. `/mcp` lists servers and tools in the REPL; `spectro doctor` probes reachability; the System-context panel names the servers.

Call semantics: at-most-once



Every call is issued *exactly once* with a 20-second budget. If the transport died between calls, it is re-established once, lazily. But a call that fails or times out on an established transport is **not retried** — a slow-but-successful `add_note` must not be doubled. Instead the transport is poisoned (the next call reconnects) and the model receives a readable `ERROR: MCP tool '...' on server '...' failed: ...`. A dead or slow server degrades; it never crashes the harness.

The example server: spectro-mcp-notes

The repository ships a complete, runnable MCP server as its own module — a stdio JSON-RPC 2.0 program whose only dependency is Jackson (“no heavy MCP SDK, no Lucene”). Build it with `./spectro mcp-notes`. It serves two tools over a directory of plain text files (default `~/.spectro/notes`, six seed notes included):

- `search_notes` — hand-rolled ranked full-text search (term frequency + substring bonus) returning snippets with their source files;
- `add_note` — appends a note as a new file, with collision-safe naming (two identical texts land in two files, never an overwrite).

Try (with the server configured): "Search my notes for 'permissions' and summarize what you find." — the call passes the permission gate like any other tool, and the Lab's map lights the whole MCP chain: client → network stack → boundary → server.

Scheduling

The scheduler turns spectroscope into an unattended worker: jobs in a JSON file, 5-field cron expressions, honest state tracking and desktop notifications.

```
# ~/.spectro/jobs.json - an array of jobs, validated loudly on load
[ { "id": "morning-report",
  "cron": "0 8 * * *",
  "prompt": "Summarize yesterday's session files under ~/.spectro/sessions.",
  "cwd": "/Users/you/projects/demo",
  "permissions": "readonly" } ]      // readonly (default) | auto
```

COMMAND	BEHAVIOUR
spectroscope cron	the foreground daemon — arms every job, reschedules after each firing, Ctrl+C stops
spectroscope cron list	each job with its expression, policy and next run time (explicit zone)
spectroscope cron status	list + last outcome per job: time, ok/failed/skipped, stop reason, session id, result preview
spectroscope cron --once <id>	run one job now; exit 0 only on ok

- **Overlap guard:** a job still running when its next slot fires records a `skipped (overlap)` state instead of stacking.
- **Headless policy:** jobs default to `readonly` ; every run writes a normal session file (full audit) and its outcome into `~/.spectro/jobs-state.json` .
- **Notifications:** macOS notification per finished job (terminal bell elsewhere); the desktop shell (chapter 10) adds native click-to-open notifications.
- **Container-free:** cron-utils computes the times, a single ScheduledExecutorService thread fires them — no Spring scheduling in the core.



PART IV

The file system

Everything spectroscope writes and reads lives in two places: a per-user home under `~/.spectro` and an optional per-project `.spectro/` folder. This part maps every file — and opens one up.

Where everything lives

spectroscope keeps no database, no hidden caches, no binary state. Everything is plain files you can read, version, back up and delete — most of it line-oriented JSON that `jq` understands directly.

The user home: `~/.spectro`

```
~/.spectro/
├─ settings.json      user settings layer (optional) – provider, model, autoApprove,
                      mcpServers, hooks, tts ... every key in chapter 30.
                      (config.json, the legacy name, is still read beneath it
                      for one release – `spectro doctor --migrate` renames it)
├─ sessions/         the session archive – the heart of it all
│   └─ 20260716-091500-3fa4b2c1.jsonl one session = one append-only event file
│       └─ 20260716-091500-3fa4b2c1/ only when images were attached:
│           └─ blobs/
│               └─ ca96adb5...f90157 attachment bytes, file name = sha256
├─ images/           generated images, shared across sessions
│   └─ 3f7a99c2...81d4.png content-addressed: <sha256>.<png|jpg|webp>
├─ skills/           user-scope skills (chapter 15)
│   └─ my-skill/SKILL.md
├─ models/           local speech models (voice setup scripts)
│   └─ ggml-small.bin whisper.cpp STT model (~460 MB, SHA-pinned)
│   └─ piper/piper TTS binary
│   └─ en_US-lessac-medium.onnx TTS voice
├─ notes/            the example MCP server's note store (one .txt per note)
├─ jobs.json          cron jobs – an array, hand-edited (chapter 19)
└─ jobs-state.json    last outcome per job id – written by the scheduler
```

Directories appear on demand — a fresh install has none of them until the first session, the first generated image, the first setup script. On this machine, today: `sessions/` (11 files, 228 kB) and `models/` (465 MB — the whisper model dominates).

The launch-dir layer: `<launch dir>/spectroscope`

```
<launch dir>/spectro/
├─ settings.json      the launch-dir settings layer – the directory spectroscope was
                      STARTED in. Still read (deprecated compat layer, one
                      release); its role as THE project file moved to the
                      workspace's own .spectro/ pair below. The “Persist”
                      checkbox appends here only when the session has no
                      real workspace (throwaway temp desk).
└─ skills/            project-scope skills – win over user scope by name
    └─ brainstorming/SKILL.md
    └─ test-driven-development/SKILL.md
    └─ verification/SKILL.md
    └─ writing-plans/SKILL.md
```

Two more files matter at the launch-dir root: the gitignored `./.env` (**secrets only** since the precedence flip — API keys, loaded by Gradle and the launcher, never committed; deprecated `SPECTRO_*` lines still work as the env base, but `.env.example` now documents just the keys) and an optional `SPECTRO.md`, whose content is appended to the system prompt as project context — the agent reads it before your first message.

Inside a session file

The session id doubles as the file name: `yyyyMMdd-HHmss-<uuid8>` — sortable by start time, collision-free. One event per line, compact JSON, UTF-8, append-only: the file is never rewritten, not even by compaction. This is a real session from this machine (a thinking model answering “What is 2+2? Think first.” — 43 lines, abbreviated in the middle):

```
{
  "type": "run_start",
  "runId": "e93e24cf-f415-4873-a21c-cbd73fec673e",
  "agentId": "main",
  "prompt": "What is 2+2? Think first.",
  "provider": "ollama",
  "ts": 1783086147207
}
{"type": "turn_start", "agentId": "main", "turn": 1, "ts": 1783086147208}
{"type": "thinking_delta", "agentId": "main", "text": "The", "ts": 1783086147561}
{"type": "thinking_delta", "agentId": "main", "text": " user", "ts": 1783086147561}
... 30 more thinking deltas — the model reasons token by token ...
{"type": "text_delta", "agentId": "main", "text": "2", "ts": 1783086148244}
{"type": "text_delta", "agentId": "main", "text": " +", "ts": 1783086148263}
{"type": "text_delta", "agentId": "main", "text": " 2", "ts": 1783086148282}
{"type": "text_delta", "agentId": "main", "text": " =", "ts": 1783086148301}
{"type": "text_delta", "agentId": "main", "text": " 4", "ts": 1783086148339}
{"type": "text_delta", "agentId": "main", "text": ".", "ts": 1783086148358}
{"type": "usage", "agentId": "main", "inputTokens": 297, "outputTokens": 49, "ts": 1783086148379}
{"type": "run_end", "runId": "e93e24cf-...", "stopReason": "end_turn", "ts": 1783086148379}
```

Everything you saw in Part II — chat, graph, trace, Lab, plan panel — is a fold over lines like these. A REPL session appends multiple runs into one file; resume appends to the same file again. Chapter 22 documents all 18 line types field by field.

Blobs and generated images

- **Attachments** (what you upload) live next to their session: `sessions/<id>/blobs/<sha256>`. Content-addressed — attaching the same image twice stores one file. Events reference `blobPath` + `sha256` only; a deleted blob degrades to a placeholder on replay, never a crash. Accepted types: jpeg, png, webp, gif.
- **Generated images** (what the agent paints) are shared across sessions in one global store: `~/.spectro/images/<sha256>.<ext>`. The web UI loads them via `GET /api/images/<file>`, where only `64-hex.png|jpg|webp` names are even considered — the content address is the traversal guard.

The workspace: where the agent actually works

The session files above are spectroscope's *records*; the **workspace** is the agent's *desk* — the directory the file tools sandbox against, glob/grep search and `run_command` runs in. Unless a `workspace` is configured (chapter 30), every session gets its own folder under the OS temp dir:

```
<tmpdir>/spectroscope-ws/
├── 20260716-204301-db5f3af7/  one folder per session, keyed by the SESSION id
│   └── add.py                 ← what “write me a script” produces lands HERE,
└── 20260716-204419-39b41a29/  not in the repo you started spectroscope in
```

The key is the session id, so a **resume finds its files again**; a New chat means a fresh desk. The web server announces the folder over the socket (`workspace_info` — a UI-only frame, never in the JSONL) and the Files tab follows it (chapter 6). The launch dir stays anchored: skills, MCP servers, SPECTRO.md and its own (deprecated) settings layer still load from the directory spectroscope was started in — while the workspace contributes its own settings pair on top (below).

Three ways to place the desk, highest wins: **pick it per session** in the web UI (the Files tab's "Choose folder ..." button opens the native macOS dialog on the spectroscope machine; only before the first run, pinned in server memory for the session), **configure it** (`workspace` key / `SPECTRO_WORKSPACE` / `--workspace` — chapter 30), or **let spectroscope mint** the per-session temp folder above.

A *real* workspace (picked or configured — not the throwaway temp desk) carries its own settings pair, the top file layers of the hierarchy (chapter 30) and the files the composer gear and the permission dialog's "Persist" checkbox write:

```
<workspace>/ .spectro/
├─ settings.json           the workspace PROJECT file — travels with that repo.
│                           Team conventions become checked-in settings:
│                           permissionMode, autoApprove, hooks, mcpServers ...
│                           Persisted "always allow" rules land here.
├─ settings.local.json     machine-local overrides for THIS workspace — absolute
│                           paths, personal rules. Never committed:
└─ .gitignore              spectroscope writes it on the first local save; it lists
                           settings.local.json so the local file stays out of git
```

This pair is not just read, it is **executed**: `mcpServers` spawns processes, `hooks` runs shell commands around every tool call, and a permissive `autoApprove` / `permissionMode` can auto-allow gates that would otherwise ask (chapter 30). Pinning a cloned or otherwise foreign folder as your workspace means reviewing its `.spectro/` first — the same way you would review its build scripts before running them.

Lifecycle: create, resume, delete

- **Created** on the first prompt of a connection (web) or at REPL start; headless and cron runs write the same files.
- **Resumed** by `spectroscope --resume <id>` or the web Resume button — the history is reconstructed from the file, new events append.
- **Deleted** only by the guarded two-step (chapter 8): `SessionStore.deleteSession` removes the JSONL and the blob folder — and only for ids resolving to a direct child of the sessions directory. Nothing else in spectroscope ever deletes or rewrites a session line.
- **Crash-safe by construction**: one `write` per event, no open handle, no save step. A torn last line after a crash is silently discarded on read; everything that streamed is on disk.

The jq cookbook

Because the format keeps its rules (one object per line, no binary, camelCase), the shell is a first-class session browser:

```
# watch a session live
tail -f ~/.spectro/sessions/<id>.jsonl | jq -c '{type, agentId, name}'

# tool timings
jq -r 'select(.type="tool_result") | "\(.durationMs)ms\t\(.callId)"' <id>.jsonl

# token totals
jq -s '[] | select(.type="usage")
      | {in: map(.inputTokens)|add, out: map(.outputTokens)|add}' <id>.jsonl

# the agent tree
jq -r 'select(.type="agent_spawn") | "\(.parentId) → \(.agentId): \(.task)"' <id>.jsonl
```



PART V

Under the hood

The complete technical reference: the generated architecture dossier, every event on the wire, the provider port, the WebSocket and REST protocols, the loop internals, context management, MCP internals, every configuration key, and the build itself.

- | | |
|---|--|
| 21 The architecture dossier | 28 Context: compaction, caching, introspection |
| 22 The RunEvent protocol | 29 MCP internals |
| 23 Streams & cancellation | 30 Configuration reference |
| 24 The provider wire | 31 Build & test inventory |
| 25 The WebSocket protocol | A Troubleshooting |
| 26 The REST API | B Reproducing this guide |
| 27 The agent loop, hop by hop | |

The architecture dossier

Fourteen generated diagrams cover the full build — every class name, wire string, endpoint and count in them was read from this repository, and each comes from a rerunnable Python generator (docs/diagrams/build_NN_*.py): change the system, rerun the script, the diagram is current again. The shared visual language: **ocean** outlines the core, **lilac** the faces, **salmon** the model side and external services, **moss** the disk; sand carries the event wire and stats; coral appears only where a human decides, a run is cancelled or the network boundary is crossed.

The big picture

Chapter 1 already showed diagram 00 (one core, five faces). The build view below adds the honest module structure: five JVM modules, two JS toolchains, and the one deliberate cross-dependency — the server reuses the CLI's Transcriber for `/api/transcribe`.

SPECTROSCOPE · ARCHITECTURE DOSSIER · 01

The build.

settings.gradle.kts includes exactly four JVM modules; spectro-web and spectro-desktop live beside the Gradle build with their own toolchains. Java 21 everywhere, tests run against build/test-home.

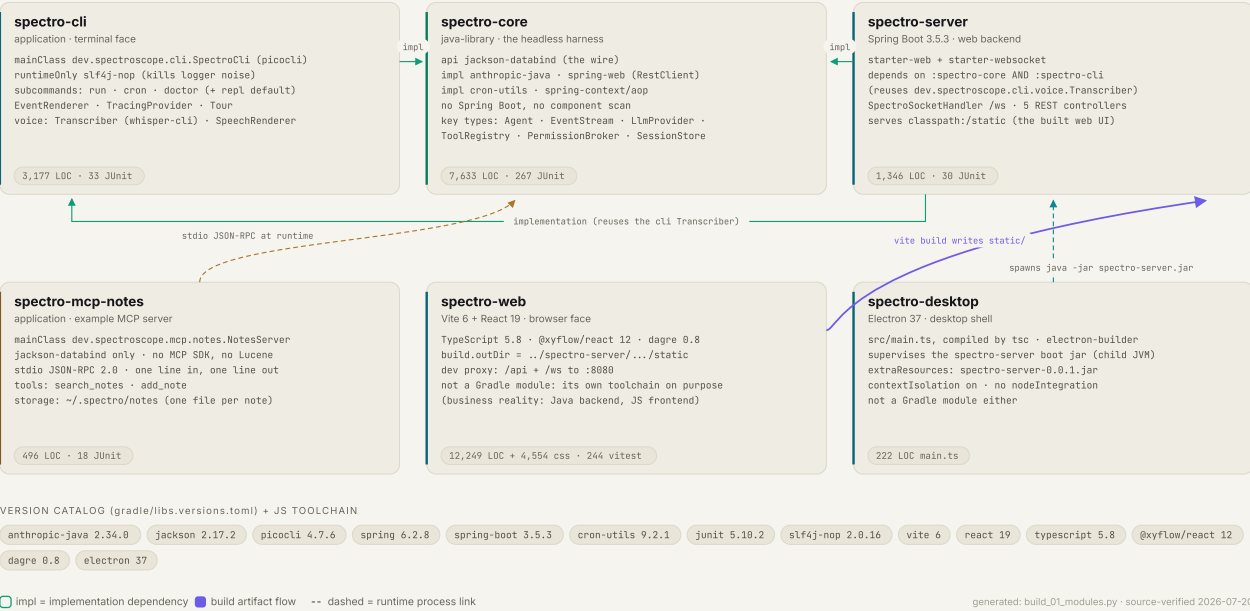
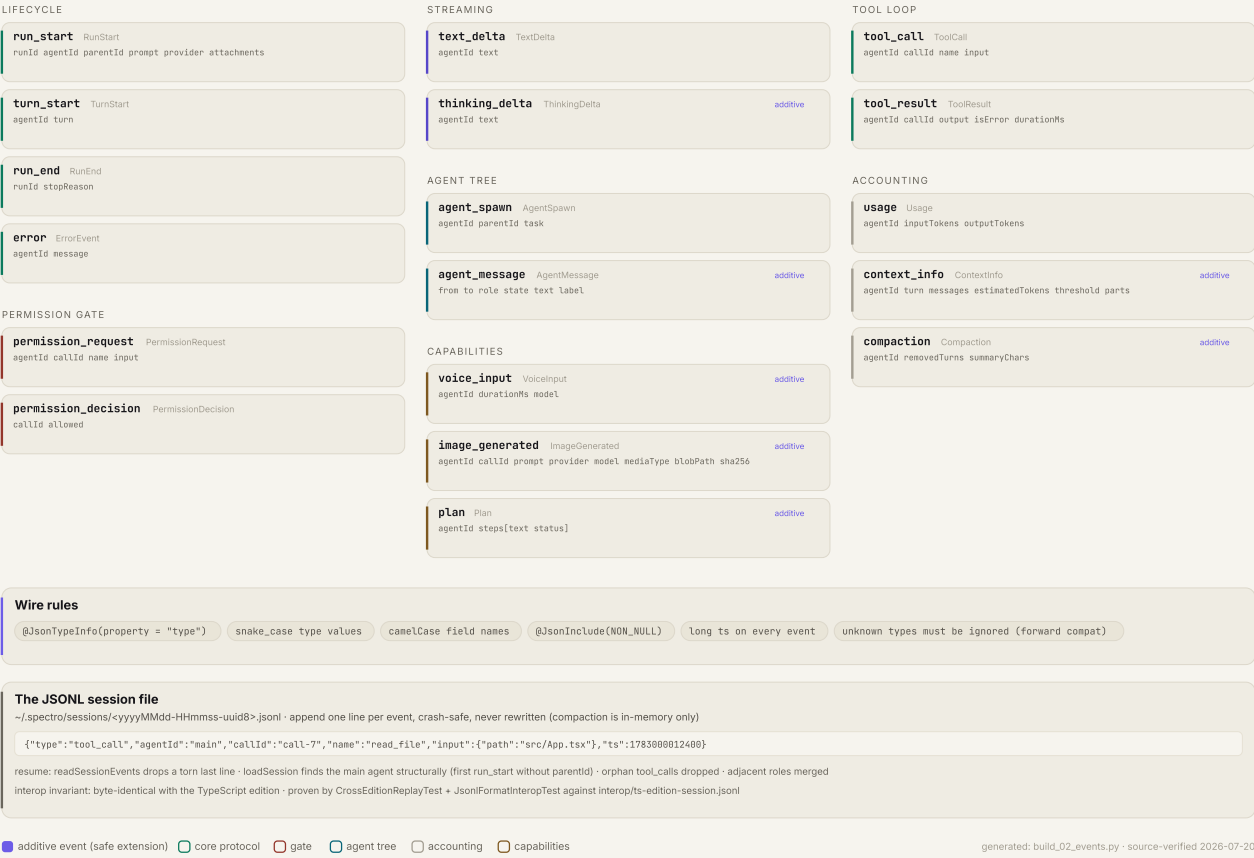


Diagram 01 — the Gradle build: module dependency edges, version catalog, and the two npm toolchains that live next to the JVM graph on purpose.

The protocol layer

The event protocol.

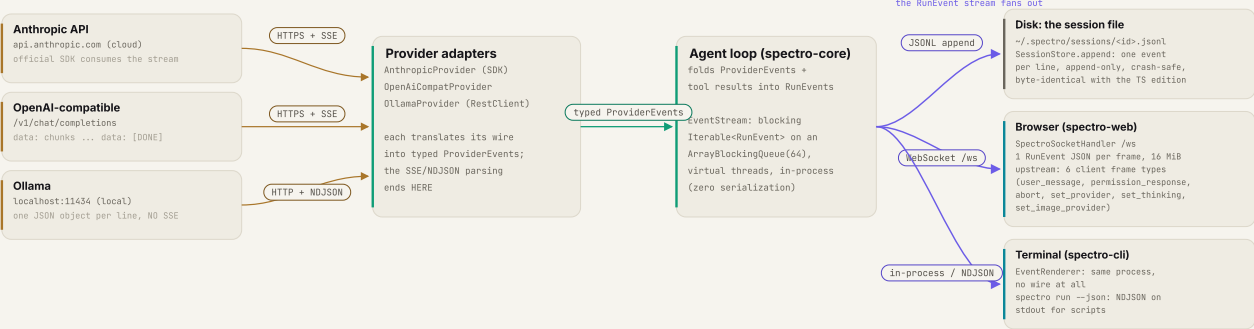
One sealed interface, 18 records, Jackson-typed by a snake_case "type" property. The same line is the socket frame, the JSONL storage row and the graph input.



The protocols.

Five wire shapes carry one event model. SSE lives only between the cloud LLM APIs and the provider adapters; inside the harness everything is an in-process stream, the browser gets WebSocket frames, the disk gets JSONL, and agents talk to each other through plain events.

THE HOP MAP: EVERY TRANSPORT, END TO END



MCP side channel (tools, not events)
McpClient ↔ server: JSON-RPC 2.0 over stdio, one message per line (protocol 2024-11-05)
optional HttpSseTransport: JSON-RPC over HTTP POST, the RESPONSE arrives as an SSE body
results surface as ordinary tool_call / tool_result events: nothing new on the wire

Where SSE actually lives
1 cloud LLM APIs stream SSE: Anthropic (inside the official SDK) and OpenAI-style endpoints
2 optional: an MCP server over HTTP answers SSE
x Ollama does NOT: it streams NDJSON
x the agent NEVER sees SSE: adapters translate to typed events before anything leaves the provider
x the browser does NOT get SSE: it gets WebSocket

Where JSON lines live (same framing, three documents)
1 the session file: one RunEvent per line (the storage format, shared with the TS edition)
2 MCP stdio framing: one JSON-RPC message per line
3 spectro run --json: NDJSON RunEvents on stdout

do not confuse the framing (JSON per line) with the document type: three different vocabularies



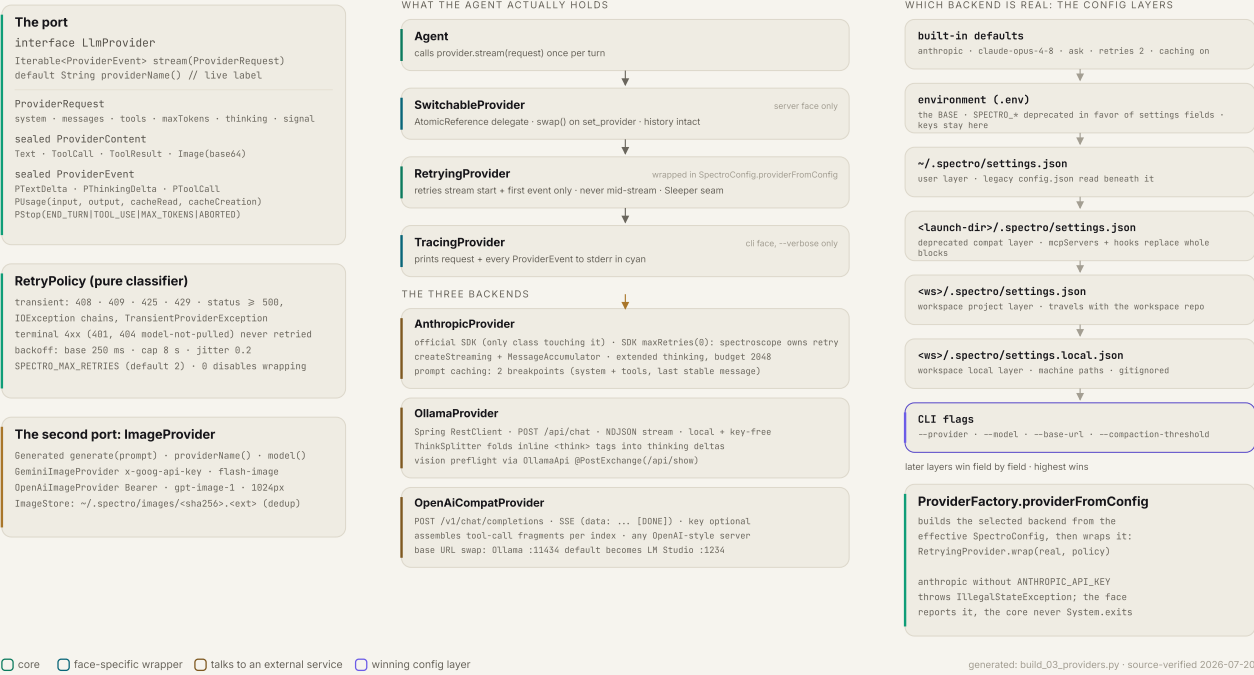
Diagram 13 — protocol breakdown, hop by hop: SSE lives only between the cloud APIs and their adapters (and the optional MCP-HTTP transport); Ollama streams NDJSON; in-process it is the blocking EventStream; WebSocket to the browser; JSONL on disk; JSON-RPC/stdio to MCP servers; A2A between agents is events on the merged stream — no network.

The model side

SPECTROSCOPE · ARCHITECTURE DOSSIER · 03

The provider port.

Everything the loop needs from a model sits behind LlmProvider. Wrappers add runtime switching and retries without the loop noticing; the config layers decide which backend is real.



core face-specific wrapper talks to an external service winning config layer

generated: build_03_providers.py · source-verified 2026-07-20

Diagram 03 — the LlmProvider port: the sealed request/event vocabulary, the wrapper chain (Switchable → Retrying → backend), the three backends with their transports, the retry policy table, the ImageProvider port and the config precedence.

The loop and the belt

SPECTROSCOPE · ARCHITECTURE DOSSIER · 04

One turn through the loop.

The agent streams a model turn, guards every tool call through hooks, allowlist and the human gate, feeds the result back, and repeats until the model stops calling tools.

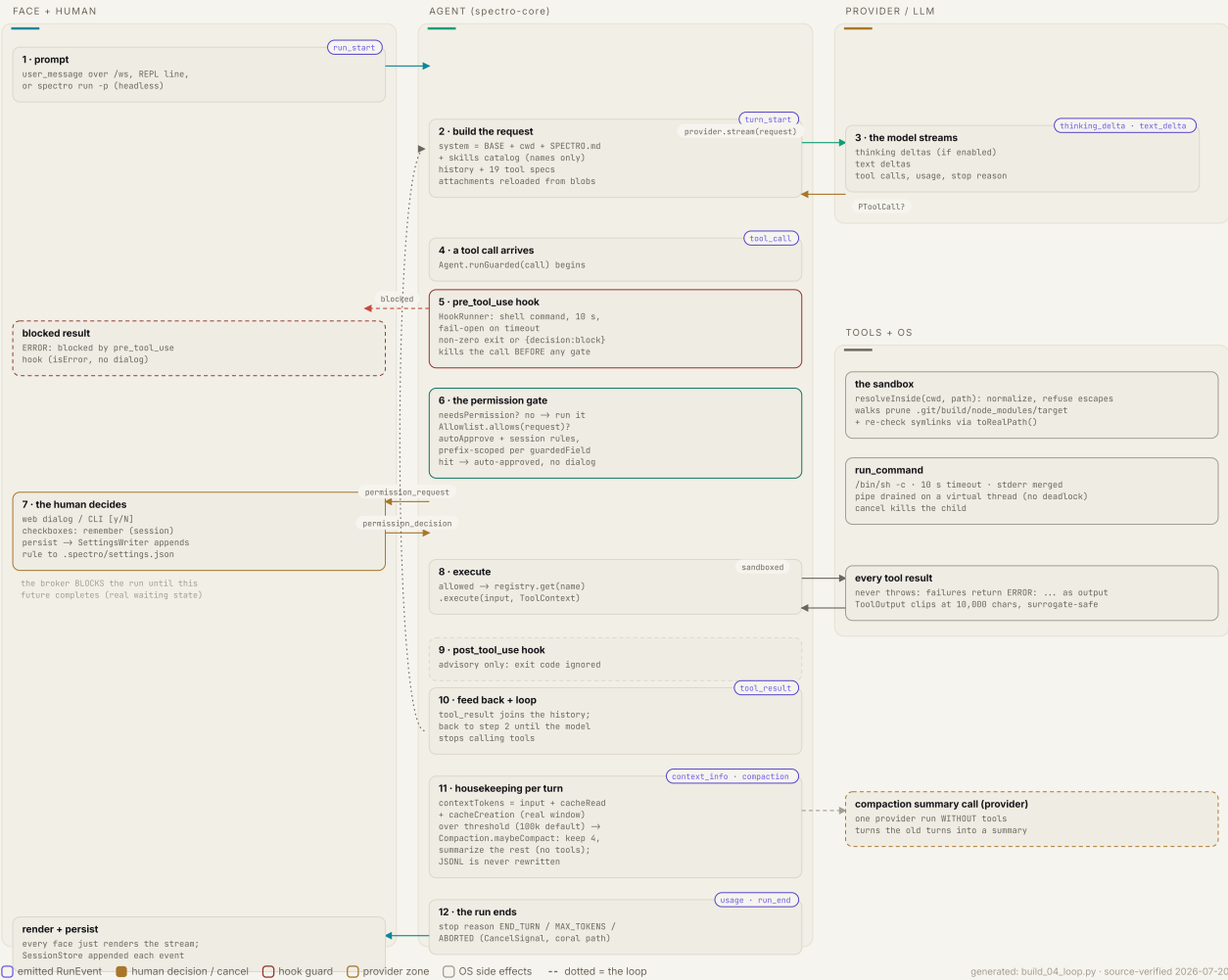
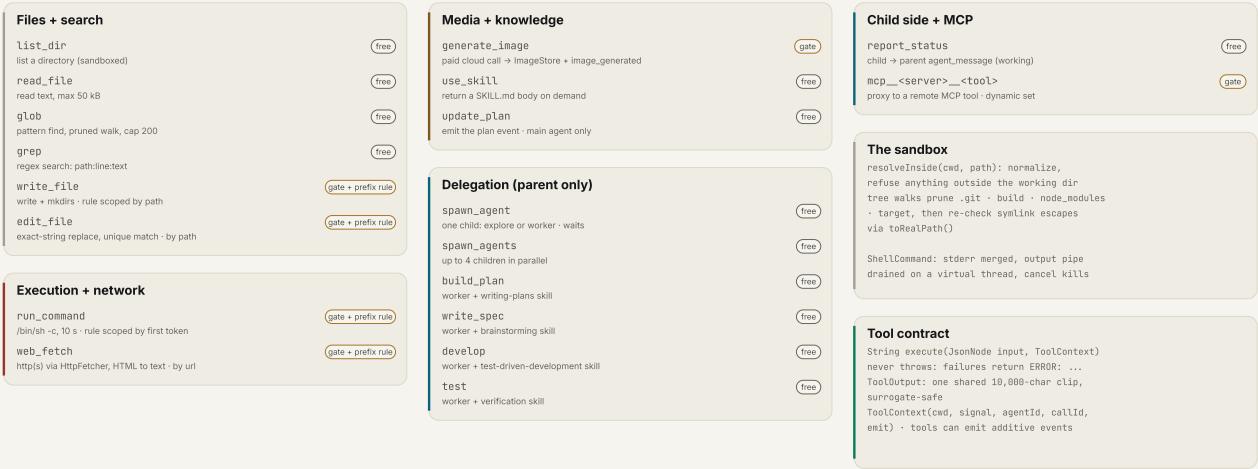


Diagram 04 — one turn as a swimlane: request build, streaming, the pre_tool_use hook, allowlist and human gate, sandboxed execution, the feedback loop into the next turn, compaction and run end.

The tool belt.

19 tools behind one registry and one gate. Read-only tools run free; anything that writes, executes, spends money or leaves the machine asks first. Course maxim: tool inputs are model output, and therefore untrusted.



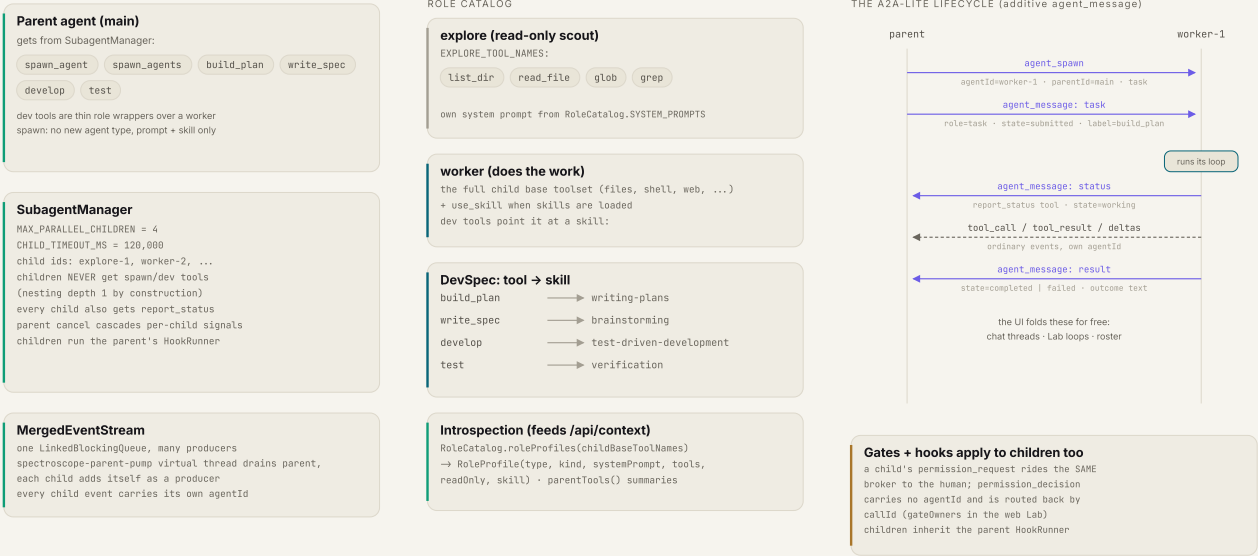
☐ free (read-only or harmless) ☐ gated by the permission broker ☐ delegation ☐ external side effects generated: build_05_tools.py · source-verified 2026-07-20

Diagram 05 — the tool belt: every tool grouped by family with its gate class (free / gated / gated + prefix rule), the sandbox rules, the tool contract, and the allowlist's guardedField mapping.

Delegation

Delegation: agents spawning agents.

The parent agent spawns typed children through ordinary tools. Every child is its own loop with its own toolset and CancelSignal; an additive agent_message lifecycle makes the protocol between them visible.



The web face

SPECTROSCOPE · ARCHITECTURE DOSSIER · 07

The web backend.

Spring Boot on 127.0.0.1:8080. One TextWebSocketHandler speaks the RunEvent wire; a handful of read-only REST endpoints serve replay, context, models and the workspace. One jar serves the built UI.



Runtime and integration

SPECTROSCOPE • ARCHITECTURE DOSSIER • 09

Start it: launcher, desktop, doctor.

One executable per checkout fixes the two host problems (JDK version, .env) before any command runs. The desktop face is a supervisor: Electron manages the Spring Boot JVM as a child process.

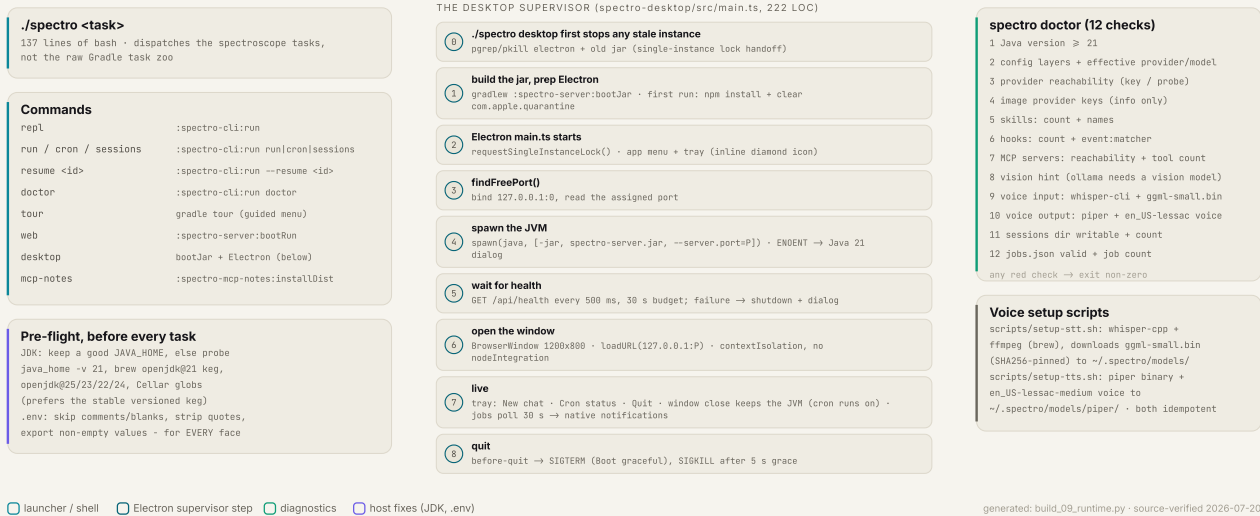


Diagram 09 — runtime: the `./spectro` command table and its pre-flight (JDK resolution, `.env`), the nine-step Electron supervision sequence, the twelve doctor checks and the voice setup scripts.

SPECTROSCOPE • ARCHITECTURE DOSSIER • 10

MCP: external tools, no new events.

A container-free MCP client wraps every remote tool as an ordinary spectroscope Tool. Calls surface as plain `tool_call` / `tool_result`, so graph, trace and JSONL show them for free and the wire stays byte-identical.

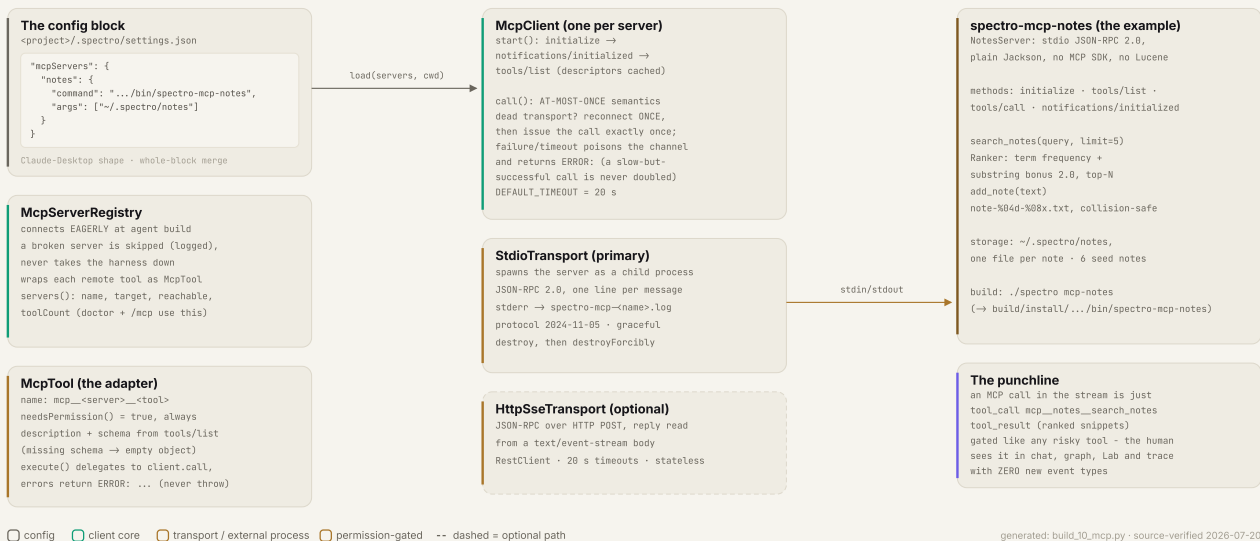


Diagram 10 — MCP: the config block, the registry's eager-connect/skip-on-failure policy, the at-most-once client, both transports, the McpTool adapter and the spectro-mcp-notes example server.

The codebase itself

SPECTROSCOPE • ARCHITECTURE DOSSIER • 11

Code inventory.

Area proportional to lines of source (main + tests), counted per module and package on 2026-07-16.

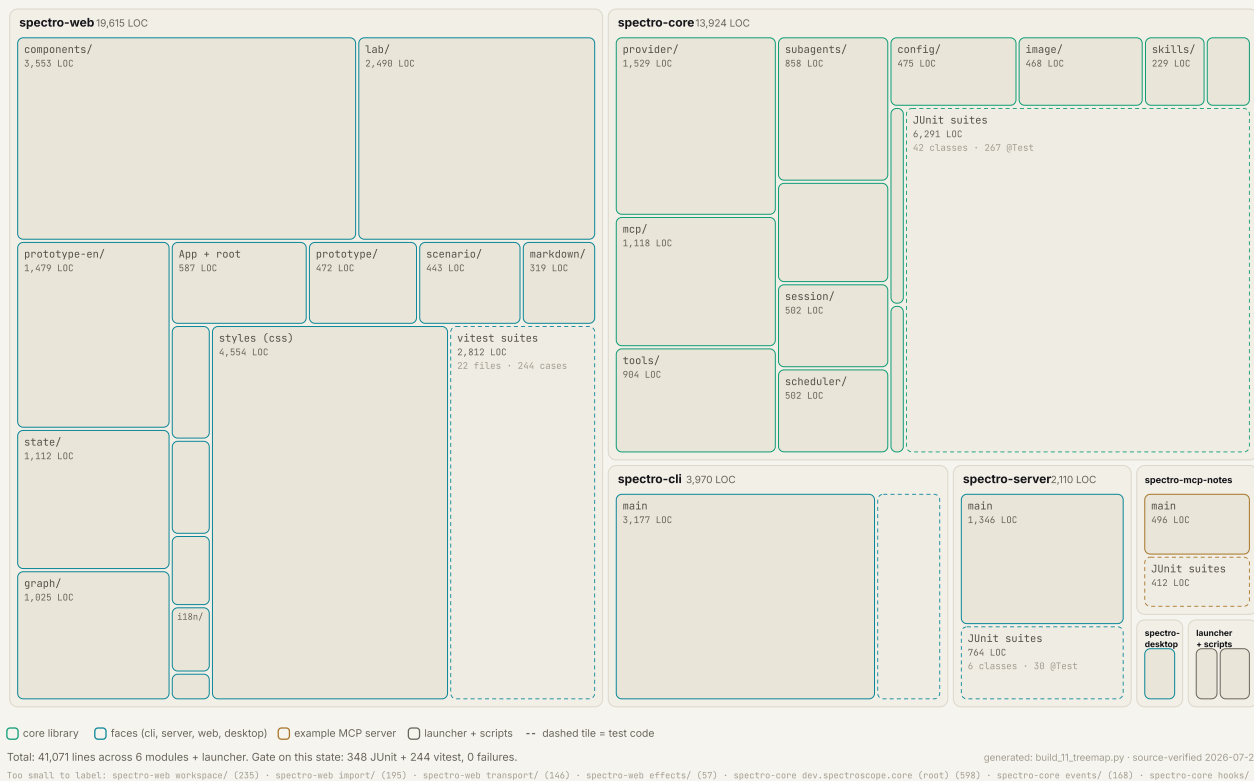


Diagram 11 — the code inventory as a treemap: area proportional to lines of source per module and package, test code as dashed tiles, measured counts in the footer.

SPECTROSCOPE • ARCHITECTURE DOSSIER • 12 • THE WALL

spectroscope, the whole machine.

Agent harness, five faces, one event wire. Everything on this wall exists in the repo; the dashed line is the real network boundary

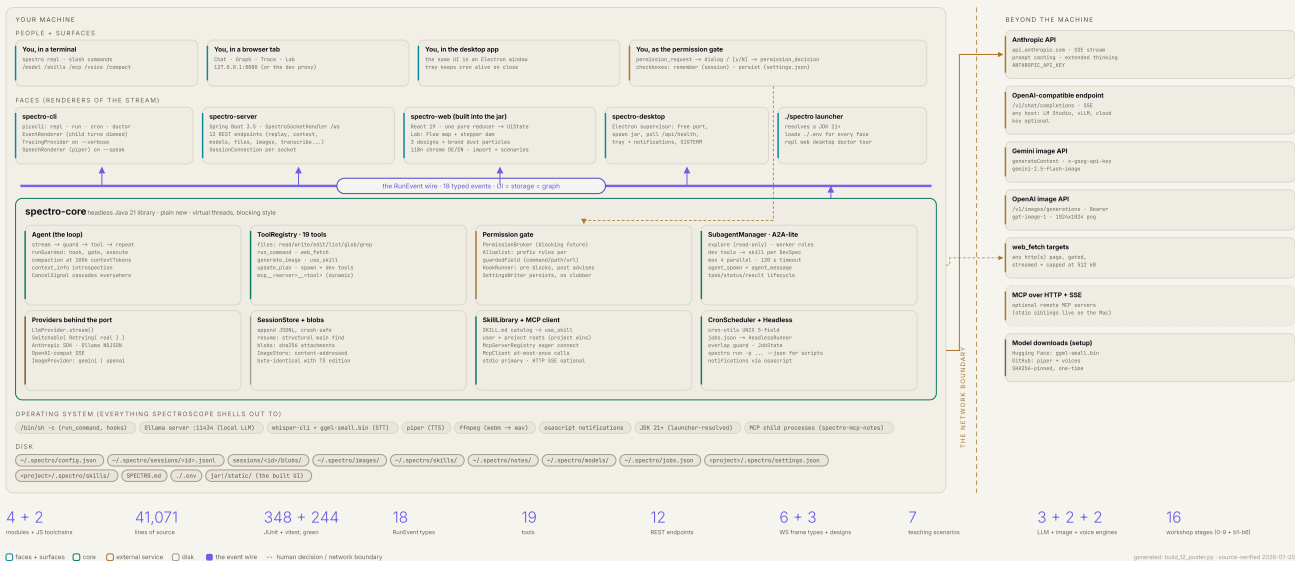


Diagram 12 — the wall poster: people, faces, core, operating system, disk, and everything beyond the network boundary in one picture.

The RunEvent protocol

Eighteen event types, one sealed Java interface, one JSON line each. This chapter is the complete wire reference — every field, every optionality, every rule. The format is binding and shared byte-for-byte with the TypeScript edition (docs/concept/JSONL-FORMAT.md is the contract document).

The wire rules

- 1. **snake_case types, camelCase fields.** "type":"tool_call" carries callId, never call_id.
- 2. **One line = one event**, compact JSON, UTF-8, \n-terminated. No arrays, no comments, no blank lines.
- 3. **Append-only.** A written line is never modified, reordered or deleted — compaction included.
- 4. **Additive evolution only.** New types and new optional fields may appear; renaming, removing or reinterpreting is forbidden. Old files replay forever.
- 5. **Tolerant consumers.** Unknown types and fields are skipped, never errors; an unparseable (torn) trailing line is silently discarded.
- 6. **Null optionals are absent.** Optional fields are omitted from the JSON entirely (@JsonInclude(NON_NULL)), matching the TS edition.
- 7. **Structured tool input.** input travels as a JSON object, never as a serialized string — consumers parse trees, never string-match JSON.
- 8. **Only data.** No binary, no object references, no cycles; every line round-trips losslessly.

IDS

sessionId = file name (yyyyMMdd-HHmss<uuid8>) · runId = UUID per run · agentId = "main" or explore-1 / worker-2 ... · callId links tool_call ↔ permission_request/decision ↔ tool_result.

TIMESTAMPS

ts = epoch milliseconds, monotone within a file (write order).

The catalog

run_start			since stage 3
FIELD	TYPE		
runId	string	UUID per run	
agentId	string	"main" or child id	
parentId	string?	only on child runs — the structural marker of a subagent	
prompt	string	the user/task text	
provider	string? additive	kept live-accurate across mid-session switches	
attachments	Attachment[]? additive	image references: {kind, mediaType, blobPath, sha256} — bytes never enter the event	

turn_start

since stage 3

`agentId`, `turn` (1-based; the loop caps at 15). On resume a turn boundary flushes the reconstruction buffer.

text_delta

since stage 3

`agentId`, `text` — one streamed chunk of the answer. Consecutive deltas concatenate into the assistant text on resume.

thinking_delta

additive · stage 7+

`agentId`, `text` — one chunk of the model's reasoning. A sibling of `text_delta` but a separate stream: rendered apart, and it **never re-enters the provider history** (the resume fold ignores it).

tool_call

since stage 3

`agentId`, `callId`, `name`, `input` (JSON object). MCP calls (`mcp__server__tool`) and skill uses are ordinary tool calls — no dedicated types exist.

permission_request

since stage 3

`agentId`, `callId`, `name`, `input` — emitted when a gated tool is about to run in ask mode; same `callId` as the pending call.

permission_decision

since stage 3

`callId`, `allowed` (boolean). One of only two types **without an `agentId`** — consumers route it back via `callId`. Emitted for every decision, allowlist auto-approvals included (the audit trail).

tool_result

since stage 3

`agentId`, `callId`, `output` (always a string, clipped surrogate-safely), `isError` (true iff the output starts `ERROR:`), `durationMs`.

agent_spawn

additive · stage 5

`agentId` (the *child*), `parentId`, `task`. The tree edge — the whole agent hierarchy lives in these two id fields.

compaction

additive · stage 4

`agentId`, `removedTurns`, `summaryChars`. An audit line only: compaction replaces the *in-memory* history — the file is never rewritten.

voice_input

additive · bonus 2

`agentId`, `durationMs`, `model` (e.g. "ggml-small"). Written *before* the `run_start` of a spoken turn; pure provenance — a resumed voice turn is byte-identical to a typed one.

usage

since stage 3

`agentId`, `inputTokens`, `outputTokens` — per agent, raw provider counts. The wire never includes cache arithmetic (that lives only in the compaction trigger) — the token truth.

run_end

since stage 3

`runId`, `stopReason` — no `agentId`. Stop reasons: `end_turn` (regular), `max_tokens`, `tool_use` (edge case), `aborted` (cancel/Ctrl+C), `max_turns` (the 15-turn brake), `error` (preceded by an `error` event).

error

since stage 3

`agentId?`, `message` — the failure channel, followed by `run_end {stopReason:"error"}`. Also the web server's own error path (a refused provider switch, a too-large attachment) — always a first-class event, never a separate frame type.

image_generated

additive · bonus 4

`agentId`, `callId`, `prompt`, `provider`, `model`, `mediaType`, `blobPath` (relative to `~/spectro`, e.g. `images/3f7a...png`), `sha256`. The event carries the reference; the bytes live in the content-addressed store.

context_info

additive · full build, opt-in

`agentId`, `turn`, `messages`, `estimatedTokens`, `threshold`, `parts[]` (`{label, chars, estTokens}` — system prompt / tool schemas / conversation). Emitted once per turn when introspection is on; sizes are chars/4 *estimates* — `usage` stays the truth. Drives the context ring.

agent_message

additive · full build (A2A-lite)

`from`, `to`, `role`, `state`, `text`, `label?` — note the addressing is from/to, not `agentId`. Roles: `task` (parent→child, state `submitted`), `status` (child→parent, `working`, fed by `report_status`), `result` (child→parent, `completed` / `failed`). `label` names the dev tool (`build_plan` ...) and is absent on plain spawns.

plan

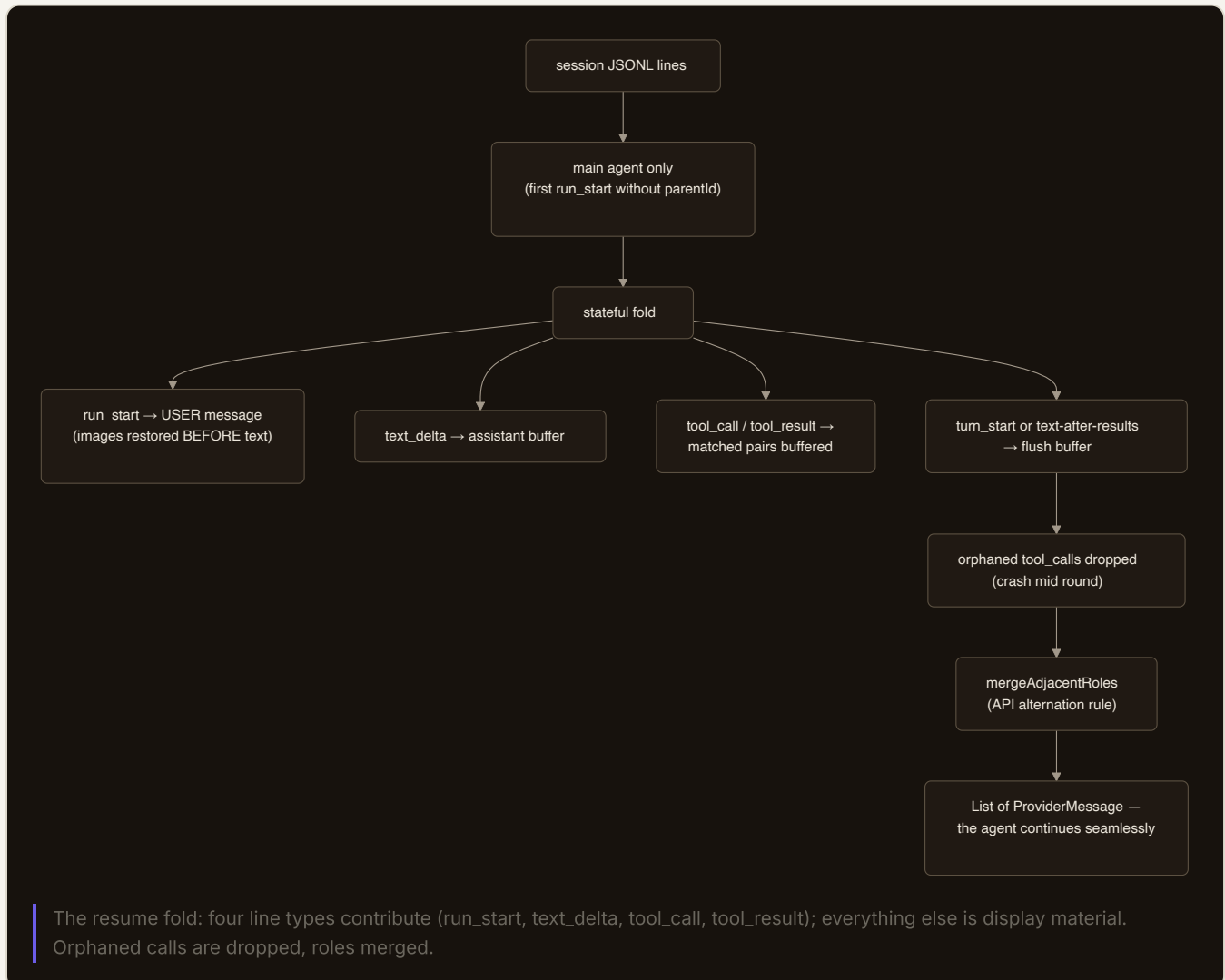
additive · full build

`agentId`, `steps[]` (`{text, status}`). Latest-wins full replacement from `update_plan`; statuses are the canonical English enum `pending` / `in_progress` / `completed`, enforced at the write boundary.

The cross-edition proof

The invariant is tested, not asserted: `CrossEditionReplayTest` replays the TypeScript edition's canonical example session verbatim — every line parses into a typed event, **re-serialization reproduces the other edition's bytes line for line**, and the resume fold reconstructs a usable history even though that file uses compact ids (`a0`) instead of `"main"` . Two interop hardenings came out of that work and are now permanent behaviour: the main agent is found *structurally* (first `run_start` without `parentId`), and text arriving after buffered tool results flushes the reconstruction even when a foreign file omits `turn_start` .

Resume: lines become a conversation



Only four line types rebuild the provider history — everything else (`usage` , permissions, `thinking_delta` , `compaction` , `voice_input` ...) is display and audit material. Child events never leak into the history: their results are already inside the parent's `tool_result` . Orphaned tool calls (a crash mid-round) are dropped because the provider API would reject the history; attachment images are restored *before* their prompt text, at their original position.

Streams & cancellation

The Java-21 answer to the TypeScript edition's async generators: a blocking `Iterable` over a bounded queue, produced by a virtual thread, plus a cooperative cancel signal. No reactive framework anywhere.

EventStream

```
public interface EventStream extends Iterable<RunEvent>, AutoCloseable {
    void cancel();
    @Override void close(); // idempotent; also cancels
    static EventStream start(CancelSignal signal, Consumer<Consumer<RunEvent>> body) { ... }
}
```

- **Backpressure for free:** a bounded queue (64) — a slow renderer blocks the producer instead of ballooning a buffer.
- **The producer** runs on a virtual thread named `spectroscope-agent`; a `finally` always enqueues the end sentinel, so a consumer's for-each can never hang, even when the body throws.
- **The sentinel** is a private instance recognized by reference identity — never handed out, never serialized.
- `close()` also interrupts a producer parked on a full queue — try-with-resources cleans up even when a consumer bails early.

MergedEventStream

Subagents write into a shared merge: one unbounded queue, many producers (the parent pump plus one forwarder per child), exactly one consumer. Unbounded on purpose — child producers must never block behind a slow renderer. This is why A2A needs no network: the “protocol between agents” is just events interleaving on the one stream that every renderer and the session store already consume.

CancelSignal

```
public final class CancelSignal {
    public synchronized void cancel(); // idempotent, runs listeners
    public boolean isCancelled();
    public synchronized Runnable onCancel(Runnable l); // returns a deregistration handle
}
```

The loop checks it at safe points; providers close their HTTP streams on it; running shell commands are force-killed by a per-call listener that deregisters itself afterwards (the handle return — a run's listener list must not grow per tool call). `onCancel` fires immediately if already cancelled, closing the classic registration race at spawn time. Children get their own signal, cascaded from the parent's — one Ctrl+C ends the whole tree with `run_end {stopReason:"aborted"}`, not a stack trace.

The provider wire

Between the agent loop and any model sits one sealed vocabulary. The loop consumes five event kinds and produces one request record — everything provider-specific stays inside the adapters.

The port

```
public interface LlmProvider {
    Iterable<ProviderEvent> stream(ProviderRequest request); // blocking, lazy
    default String providerName() { return null; }           // live label for run_start
}

record ProviderRequest(String system, List<ProviderMessage> messages,
                        List<ToolSpec> tools, int maxTokens, boolean thinking,
                        CancelSignal signal)

record ProviderMessage(Role role /* USER | ASSISTANT */, List<ProviderContent> content)

sealed interface ProviderContent
    permits TextContent, ToolCallContent, ToolResultContent, ImageContent

sealed interface ProviderEvent permits PTextDelta, PThinkingDelta, PToolCall, PUsage, PStop
record PUsage(int inputTokens, int outputTokens,
              int cacheReadTokens, int cacheCreationTokens)
record PStop(StopReason reason /* END_TURN | TOOL_USE | MAX_TOKENS | ABORTED */)
```

Contract points: all three backends return *lazy* iterables (tokens reach the UI while the HTTP stream is still open); `PUsage.inputTokens` stays the provider's raw count (it feeds the wire `usage` event) while the two cache fields ride separately for the compaction trigger only; a cancel mid-stream surfaces as `PStop(ABORTED)`, never as an exception.

Backend mappings

Anthropic (SSE via the official SDK)

- SDK client built with `maxRetries(0)` — spectroscope owns retry.
- Deltas translate one-to-one; **usage and fully-parsed tool inputs exist only in the accumulated final message** — the classic SDK trap — so `PToolCall`s, `PUsage` and `PStop` emit after the stream is exhausted.
- Extended thinking with a 2048-token budget (clamped below `maxTokens`); thinking requests carry no sampling parameters.
- Vision: image blocks are reordered before text within a user message.
- Cancel closes the SDK stream; the resulting error maps to `ABORTED` iff the signal fired.

Ollama (NDJSON)

- `POST /api/chat` read line by line; typed wire records
(`ChatRequest(model, stream, messages, tools, options, think)`); tool results become `role:"tool"` messages; images ride as raw base64 arrays.
- No call ids from the server — the adapter mints `ollama-call-<nanos>`.
- Token counts arrive only in the final `done:true` chunk.
- **Thinking, two ways:** the native `message.thinking` field, plus a streaming state machine that splits inline `<think>` tags even when a tag is torn across chunk boundaries.
- **Vision fail-fast:** before sending images, `POST /api/show` checks the model's capabilities — a text-only model is refused with an actionable message instead of a confusing 400.
- Error classification at the source: retryable HTTP → `TransientProviderException` ; terminal (404 model-not-pulled, 401) → `IllegalStateException` — never retried.

OpenAI-compatible (SSE)

- `data:` lines until `data: [DONE]` ; keep-alive comments skipped; `stream_options.include_usage` puts usage in the final chunk.
- **Fragmented tool calls** accumulate per index until the turn closes; arguments arrive as a JSON *string* and are re-parsed (unparseable → `{"raw": ...}`).
- Bearer auth only when a key is present — LM Studio and friends run keyless.
- Endpoint by provider name, no key-based swap: `openai` → `https://api.openai.com` , `lmstudio` → `http://localhost:1234` , `openrouter` → `https://openrouter.ai/api` ; an explicit `baseUrl` always wins. One wire protocol, three hosts.

The WebSocket protocol

One endpoint (/ws), one connection per browser tab, one agent per connection. Server→client traffic is *nothing but serialized RunEvents* — there is no separate frame vocabulary in that direction. Client→server is six small frames.

Client → server

FRAME	SHAPE	SEMANTICS
user_message	{type, text, attachments?: [{mediaType, dataBase64}]}	starts a run (refused with an error event while one is active). Attachments are decoded and stored <i>before</i> the run starts; 5 MB cap per attachment; supported: jpeg/png/webp/gif
permission_response	{type, callId, allowed, remember?, persist?}	answers a parked permission future. remember adds a session rule, persist appends it to .spectro/settings.json
abort	{type}	fires the current run's CancelSignal
set_provider	{type, provider, model?}	mid-session backend switch; validated; refused without a key; applies on the next run
set_image_provider	{type, provider}	gemin openai ; effective on the next generation
set_thinking	{type, enabled}	reasoning visibility; applies on the next run

Nuances the names hide: the client frame is permission_response — permission_decision is the RunEvent that flows back and into the file. Cancel is spelled abort on the wire. Resume is not a frame: it is the ?resume=<id> query parameter at connect time (kept across auto-reconnects); archive replay is pure REST.

Server → client

Every event of a run goes to the socket and the session file — the same object. Server-side errors (invalid frame, refused switch, oversized attachment) arrive as first-class error RunEvents. One synchronized writer per connection; a dead socket swallows sends — “a dead socket is not a run failure; the JSONL already has it.”

Threading model

- The Tomcat thread only parses frames; every run executes on a virtual thread (spectroscope-run).
- Permission waits park the agent's virtual thread on a CompletableFuture keyed by callId — cheap, and the browser dialog answers with the same id. On disconnect every pending future completes with deny, so no thread ever hangs.
- Two tabs = two connections = two agents, two session files, two independent remembered-rule lists. Only the persisted settings file is shared — its writer is serialized.

- Text frames up to 16 MB (base64 images need the headroom; the Spring default of 8 kB would close the socket with status 1009).

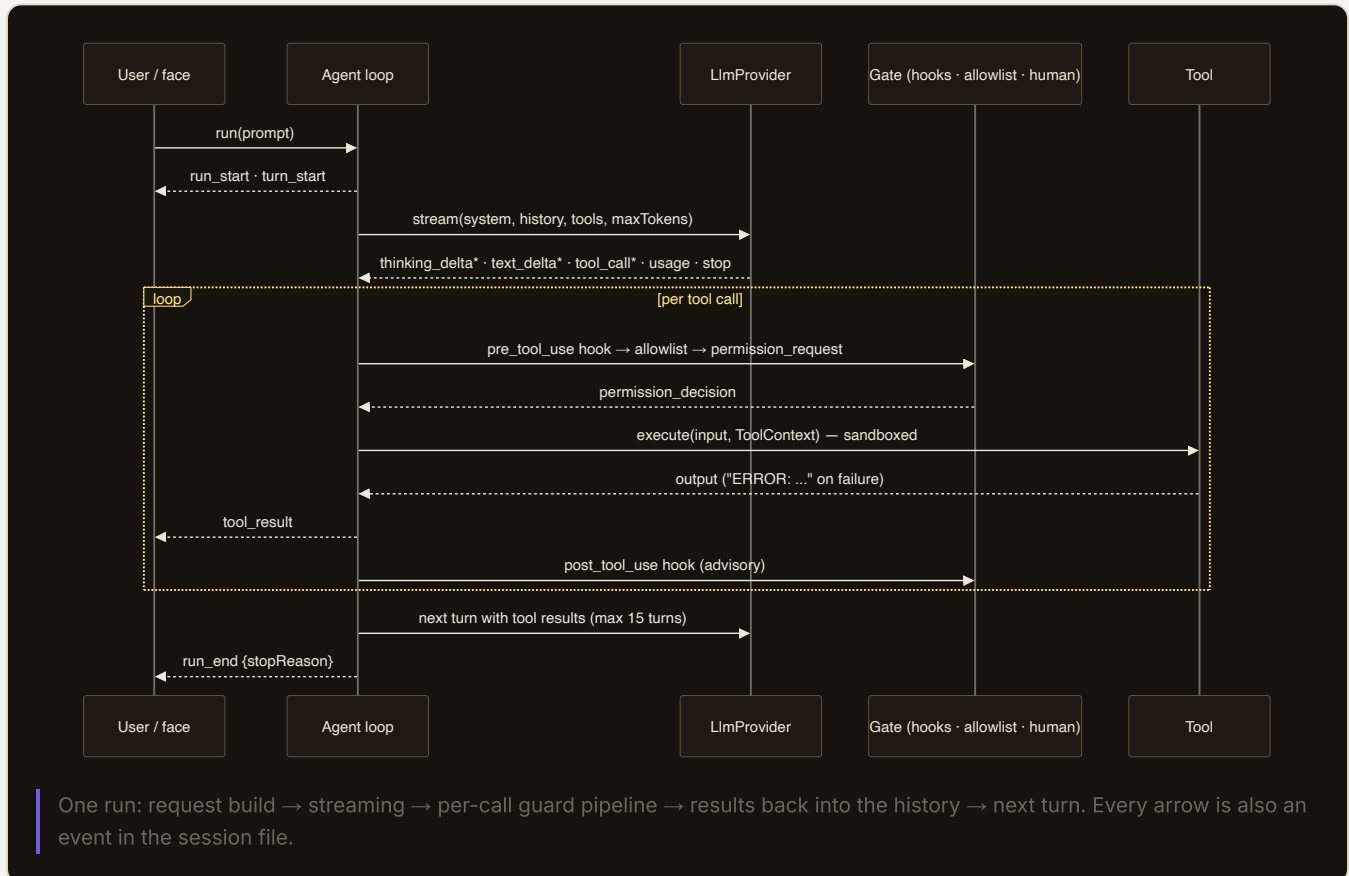
The REST API

The session stream is read-only over REST — the socket carries run mutations — but a handful of control writes exist: the transcription POST, the guarded session DELETE, the three settings PUTs, the workspace picker and the image copy-to-workspace POST (plus the fleet control POSTs). Bound to 127.0.0.1; auth and TLS are a deliberate course boundary.

ENDPOINT	RETURNS	NOTES & GUARDS
GET /api/health	<code>{"status": "ok"}</code>	the desktop shell's startup probe
GET /api/sessions	<code>[{id, startedAt, firstPrompt, tokens, provider}]</code>	fold over the sessions dir; broken files skipped
GET /api/sessions/{id}/events	the session's RunEvents as a JSON array	replay + resume seeding; torn last line dropped
DELETE /api/sessions/{id}	204 · 400 · 404	the one destructive endpoint: id-shape guard (<code>[A-Za-z0-9][A-Za-z0-9-]*</code>) → 400; unknown → 404; the store additionally refuses ids that do not resolve to a direct child of the sessions dir. Removes JSONL + blob folder
GET /api/config	<code>{provider, model}</code>	boot truth; live switches are client-side overlay
GET /api/context	system prompt, tools, skills, MCP servers, thinking, provider/model, subagent role profiles	stateless — assembled from the same constants the live agent uses; no agent built, MCP not connected
GET /api/models?provider=	<code>[string]</code>	ollama: live from <code>/api/tags</code> with finite 1.5 s/2.5 s timeouts (a stalled Ollama must not pin a worker); cloud: curated; errors → <code>[]</code>
GET /api/images/{file}	image bytes	name must match <code>[0-9a-f]{64}\.(png jpg webp)</code> — 400 <i>before</i> any filesystem access; the content address is the traversal guard
GET /api/jobs/state	<code>{jobId: JobState}</code>	cron outcomes; corrupt file → <code>{}</code> ; polled by the desktop shell
GET /api/files	the sandboxed workspace tree	hidden + ignored dirs skipped; depth ≤ 8; ≤ 2000 entries with an honest <code>truncated</code> flag
GET /api/file?path=	one file for preview	<code>resolveInside</code> + per-segment hidden/ignored re-check (the <code>.env</code> answers 404 even by direct URL); every document served with <code>Content-Security-Policy: sandbox allow-scripts</code> ; 413 oversized, 415 binary
POST /api/transcribe	<code>{text}</code>	browser audio → ffmpeg 16 kHz mono → whisper-cli; 503 with a setup hint when STT is not installed; the transcript goes to the composer, never to the agent
GET /api/claude/transcripts	<code>[{path, project, file, size, modifiedAt}]</code>	lists <code>~/.claude/projects/**/*.jsonl</code> (newest first, ≤ 300) — the folder is Finder-invisible, hence this endpoint
GET /api/claude/transcripts/content?path=	raw JSONL	must end <code>.jsonl</code> ; canonical real-path must stay inside the real base (no traversal, no symlink escape); 64 MB cap

The agent loop, hop by hop

The whole harness reduces to one loop in one class: build a request, stream the response, execute the tool calls behind the guards, feed the results back, repeat — at most fifteen times. Everything else in this book hangs off one of these hops.



The turn, precisely

1. **Run start.** Fresh UUID, `run_start` with the *live* provider label (a switched backend reports itself truthfully). Attachments convert to image content *before* the prompt text in the first user message.
2. **Per turn (1...15):**
 - a. `turn_start`; with introspection on, a `context_info` estimate of what the next call will carry;
 - b. the compaction check (chapter 28) — a no-op below the threshold;
 - c. the request: system prompt, full history copy, tool specs, maxTokens (default 32 000), thinking flag, the cancel signal;
 - d. streaming: text deltas emit immediately; thinking deltas emit but never join the history; tool calls collect; usage updates the context measure; the stop reason records;
 - e. abort check — a cancelled run ends here with `aborted`;
 - f. history append: the assistant message (text + tool-call blocks) goes in *before* any results — the API's ordering rule;

g. no tool calls → `run_end` with the mapped stop reason; otherwise:

h. the tool round: every call passes the guard pipeline below, in order; all results of the round return to the model in *one* user message — denials and errors included, as data the model can read and react to.

3. **Turn 15 exceeded** → `run_end {stopReason:"max_turns"}` — the runaway brake.

4. **Any exception** → `error` + `run_end {stopReason:"error"}` (or `aborted` if the signal fired). The stream terminates on every path.

The guard pipeline (per tool call)

Chapter 13 showed it from the user's side; this is the implementation order inside `runGuarded`, and it is load-bearing:

1. **pre_tool_use hooks** — before anything else, so a policy veto costs no dialog. First matching block wins; timeout fails open.
2. **The gate** — only for `needsPermission()` tools: `permission_request` → the broker blocks the virtual thread (terminal read, or a parked future in the web) → `permission_decision` — always emitted, allowlist answers included. Denial returns the literal `ERROR: the user denied the execution.`
3. **Execution** — `tool.execute(input, context)`; the context carries the sandbox root, the cancel signal, the caller's agent id, the call id and an `emit` sink through which artifact tools publish their additive events (`plan`, `image_generated`, `agent_message`).
4. **post_tool_use hooks** — after the fact, advisory, never rewriting.

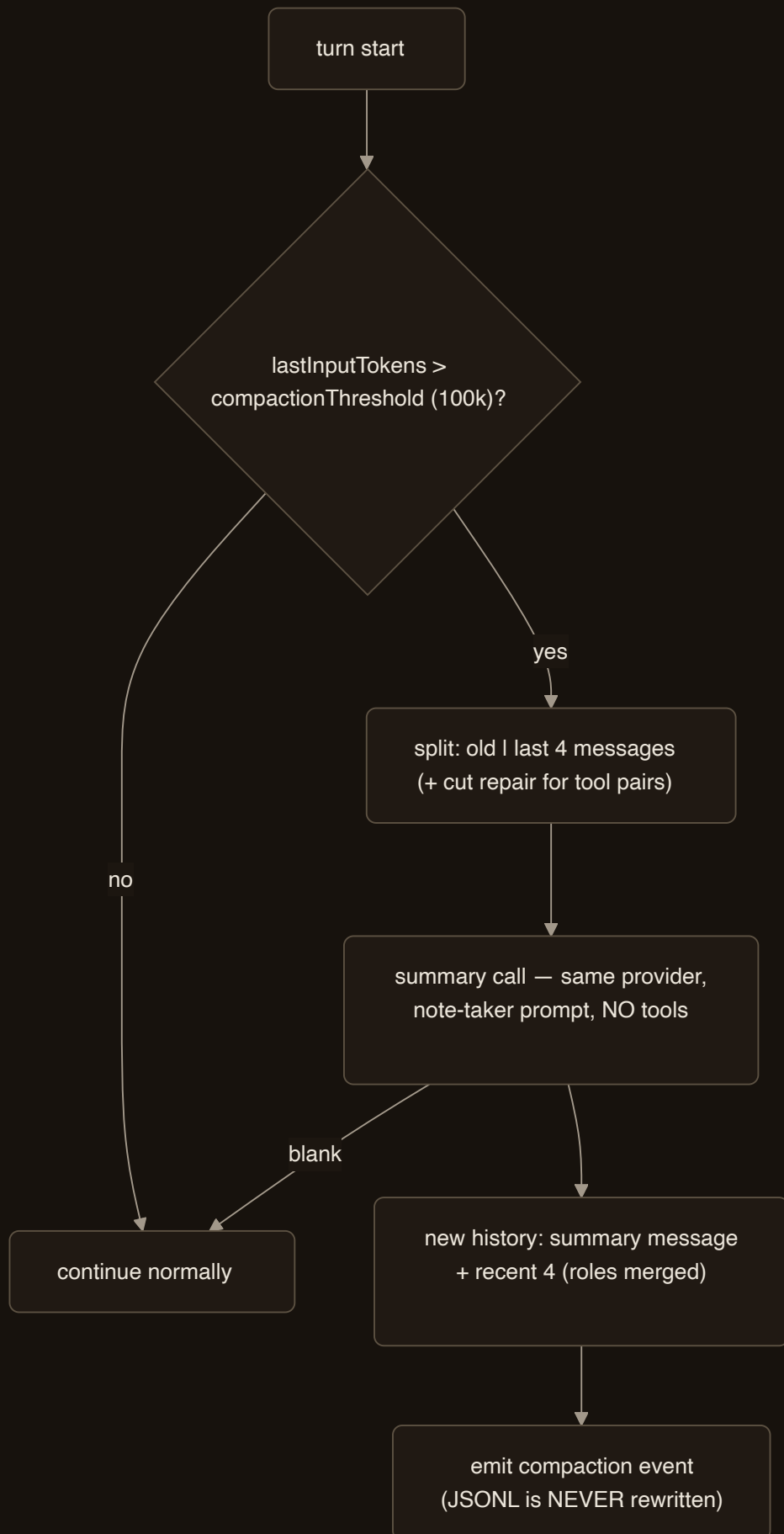
The headless variant

`spectro run` and cron jobs share a runner with a fixed system prompt for unattended operation ("there is no human at the terminal: do not ask questions ... if a tool is denied, do not retry it"), a constant-policy broker (readonly denies, auto allows), standard tools only (never the spawn tools), an external turn brake, and a result record (final text, stop reason, session id, exit-ok). Every headless run writes a normal session file — automation is fully auditable after the fact.

Context: compaction, caching, introspection

Three mechanisms manage the finite context window: compaction shrinks the history when it grows past the threshold, prompt caching makes the stable prefix cheap, and introspection shows you both at work.

Compaction



Above the threshold: split the history, summarize the old part with the same model (no tools), swap in "summary + last 4 messages". In memory only — the file grows, never shrinks.

- **Trigger:** the last turn's context size vs. `compactionThreshold` (default 100 000 input tokens) — checked at the start of each turn. The measure is honest under caching: `inputTokens + cacheRead + cacheCreation`, because cached tokens still occupy the window even though the provider bills them separately.
- **Mechanism:** everything except the last four messages is summarized by the same provider (note-taker system prompt, summary-only instruction, no tools); the new history is one summary message plus the recent four, with a cut repair so no tool result loses its call. A failed summary is an `error` event and an unchanged history — never a broken run.
- **The file never shrinks.** A `compaction` audit line is appended; a later resume reconstructs the *full* history from the file and compacts again if needed — the file is always the unabridged truth.
- **Force it:** `/compact` in the REPL calls the same mechanism regardless of the threshold.

Prompt caching

Chapter 16 covered the two breakpoints (system + tools; the last stable message). The reference detail: only text and tool-result blocks can carry a breakpoint (others pass through, best effort); the index is recomputed fresh every call, so a history reshaped by compaction simply gets a new breakpoint; and the wire `usage` event never includes the fold — a cache hit must not change the bytes an old consumer sees.

Introspection

With introspection on (both interactive faces enable it), every turn emits `context_info`: message count, an estimated token total against the threshold, and labeled parts — system prompt, tool schemas, conversation — each measured in chars and estimated at chars/4. The web header's **context ring** renders it live (green < 70 %, amber ≤ 90 %, red above); its popover shows the parts table with the honest caveat that the estimate is an estimate — the `usage` line is the truth. The "Context window" scenario (chapter 7) demonstrates the whole arc: fill → warn → compact → continue.

MCP internals

Chapter 18 explained MCP for users; this is the wire level. The `dev.spectroscope.core.mcp` package is container-free: hand-rolled JSON-RPC 2.0 records, a transport seam, a client with strict delivery semantics, and a registry that treats broken servers as skippable.

JSON-RPC framing

- Protocol version pinned: `2024-11-05`; client info `spectroscope / 1.0`.
- **stdio transport** (primary): one JSON object per line; stdout is protocol, stderr goes to a per-server log file under the system temp dir. Handshake: `initialize` → `notifications/initialized` → `tools/list`. Reads are bounded (20 s) by a watcher thread; **a timed-out channel is poisoned** — the uninterruptible reader is unblocked by closing the streams, and the channel refuses further requests until reconnect.
- **HTTP/SSE transport** (optional): each JSON-RPC frame POSTs to the configured URL; the reply's `data:` lines are concatenated per the SSE spec (a plain JSON body is accepted too); 20 s connect/read timeouts; stateless.
- Tool results: the `text` of all `content[]` blocks joined by newlines; unexpected shapes return the raw JSON rather than failing.

Client semantics

- **At-most-once:** `tools/call` is issued exactly once per tool call. Dead transport *before* the call → one lazy reconnect. Failure or timeout *during* the call → poison + readable `ERROR:`, never a re-issue (a slow-but-successful `add_note` must not run twice).
- **Registry policy:** eager connect at startup, log-and-skip on failure, descriptors cached, config order preserved; handles (name, target, reachable, tool count) feed `spectro doctor` and `/mcp`. Closed on shutdown.
- **Adapter hygiene:** a server-supplied schema that is missing or not an object is replaced by `{"type":"object"}` — untrusted server output must not crash the provider advertisement.
- **Scope:** MCP tools bind to the session at connect time, independent of LLM-provider switches; children never see them.

The reference server

`spectro-mcp-notes` doubles as executable documentation of the server side: a single-threaded stdio loop answering `initialize`, `tools/list`, `tools/call`; JSON-RPC errors `-32601` / `-32603` where they belong; tool-level failures as MCP-style `isError` results (still JSON-RPC successes). Its store is a directory of text files with collision-safe creation; its ranker a few dozen lines of term frequency + substring bonus — search as a small program, not a search engine. Eighteen tests include one that spawns the real child JVM and speaks the protocol end to end.

Configuration reference

Seven layers, seventeen keys (three of them whole-block), plus a small handful of true environment-only secrets. Missing files are fine at every layer; malformed JSON fails loudly — a broken config is a programming error, not a fallback case.

The layers

```
built-in defaults
< environment SPECTRO_*           (usually injected from ./env —
                                   now the BASE, directly above defaults)
< ~/.spectro/settings.json        (user; config.json read beneath it,
                                   for one release)
< <launch-dir>/spectro/settings.json (deprecated compat layer)
< <workspace>/spectro/settings.json (project — travels with the workspace)
< <workspace>/spectro/settings.local.json (machine-local, gitignored)
< CLI flags    --provider --model --base-url --compaction-threshold --workspace
```

The flip (owner decision 2026-07-18, "settings-productization"): env used to sit just below the flags, outranking every settings file — editing `./env` was the loudest way to configure spectroscope, and a stale line nobody remembered could silently shadow a deliberate choice made in the Settings page. Now env is the BASE, one step above the built-in defaults: it still seeds the starting configuration (a good fit for bootstrap, CI, or a one-shot experiment), but ANY settings file — user, project or local — outranks it. *"Die env ist die Basis, die Settings geben den Ton an."* Later layers win field by field — except `autoApprove`, `mcpServers` and `hooks`, which merge as **whole blocks**: a layer that defines the block replaces it entirely (no per-entry merge). Unknown JSON keys are ignored everywhere, so additive fields never break older builds.

Two resolution moments. Config resolves at two different times, and not every field is meaningful at both. The *process* moment (server boot, `spectro doctor`, CLI startup, a stateless REST call) has no session yet, so only `defaults < env < user < launch-dir < flags` applies — this is where `workspace` itself resolves, and the one-per-process `LogLevel`. The *session* moment (the agent build, the composer gear, a per-connection re-apply) joins the session's own resolved workspace pair — project, then local — directly below launch-dir: two concurrent sessions with different workspaces can legitimately run different providers or permission modes. A workspace scope may not set `workspace` itself (a folder must not repoint the agent elsewhere) or `LogLevel` (one log file per process) — both fail loudly if you try.

Workspace-supplied config is executed, not just read. A workspace's own `.spectro/settings.json` can set `mcpServers`, `hooks`, `autoApprove` and `permissionMode` — and every one of those takes effect the moment the workspace resolves: an `mcpServers` entry spawns its process, a `hooks` entry runs a shell command around every tool call, and a permissive `autoApprove` / `permissionMode` can auto-allow gates that would otherwise ask. Pinning a folder as your workspace therefore runs whatever its checked-in settings declare — the course maxim *"tool inputs are model output and therefore untrusted"* extends one layer further to *folder-supplied config is untrusted too*. Review a cloned or otherwise foreign repository's `.spectro/` before pointing spectroscope at it, the same way you would before running its build scripts.

The settings API. `GET /api/settings[?session=]` returns the resolved configuration alongside per-field **provenance** (which layer won, which lower layers were shadowed) and the raw, non-empty layers — without `?session=` it is the process-moment view, with one the session-relative view. `PUT /api/settings/{user|project|local}` writes a schema-validated partial patch to exactly one scope; a `null` value removes that key. Secret-shaped keys (`*_API_KEY` , `*_TOKEN`) are always rejected — a settings file is never a place to paste a key.

Two gears, two scopes. The header gear (the Settings page) edits the GLOBAL, user-scope fields only: session defaults (provider/model/thinking/image backend), the default workspace (origin badge + a "reset" that falls back to the layer below), `logLevel` , and the machine-tool paths (`chromeBinary` / `imageModel` / `sttModel`). A second, compact gear sits in the composer row and edits THIS session's workspace: permission mode as a keyboard-navigable text list (live immediately via `set_permission_mode` , persisted to the workspace project file once one is pinned), always-allow rules (add and delete), machine-local overrides, and raw-JSON editors for `mcpServers` / `hooks` . An unpinned session — still in its throwaway temp folder — shows the composer gear's project sections disabled: there is no durable folder to write to yet.

Doctor never lets a shadowed variable go unnoticed. Every run, `spectro doctor` walks the same provenance and prints one line for every `SPECTRO_*` variable that IS set but is no longer the effective source, naming both the variable and the settings layer that now wins:

```
env SPECTRO_MODEL is set but shadowed by user settings (effective model comes from user)
```

On first boot after the flip, if no user settings file exists yet, spectroscope materializes one FROM the current env base (secrets excluded, only the fields the env actually sets) — so day one changes nothing functionally, and from then on that file is the truth the doctor line, the Settings page and every face agree on. `spectro doctor --migrate` renames the old `~/.spectro/config.json` to its new name, `settings.json` , whenever the new name is not yet present.

Deprecation table

Every `SPECTRO_*` variable below still works exactly as before — as the env BASE — but is deprecated in favor of the settings field it feeds:

ENV VARIABLE (DEPRECATED)	SETTINGS FIELD
<code>SPECTRO_PROVIDER</code>	<code>provider</code>
<code>SPECTRO_MODEL</code>	<code>model</code>
<code>SPECTRO_BASE_URL</code>	<code>baseUrL</code>
<code>SPECTRO_WORKSPACE</code>	<code>workspace</code>
<code>SPECTRO_THINKING</code>	<code>thinking</code>
<code>SPECTRO_IMAGE_PROVIDER</code>	<code>imageProvider</code>
<code>SPECTRO_IMAGE_MODEL</code>	<code>imageModel</code>
<code>SPECTRO_STT_MODEL</code>	<code>sttModel</code>
<code>SPECTRO_CHROME</code>	<code>chromeBinary</code>
<code>SPECTRO_MAX_RETRIES</code>	<code>maxRetries</code>
<code>SPECTRO_PROMPT_CACHING</code>	<code>promptCaching</code>
<code>SPECTRO_LOG_LEVEL</code>	<code>logLevel</code>

Not deprecated — these never had a settings-field counterpart: `compactionThreshold` and `permissionMode` (aside from its live socket message) are flag/UI-only, no env form ever existed for them; the API keys in "Environment-only variables" below are true secrets and never graduate into a settings file.

Every key

KEY	TYPE · DEFAULT	ENV BASE (DEPRECATED)	MEANING
<code>provider</code>	<code>anthropic</code> <code>ollama</code> <code>openai</code> <code>lmstudio</code> <code>openrouter</code> · <code>anthropic</code>	<code>SPECTRO_PROVIDER</code>	the LLM backend; unknown values fail loudly
<code>model</code>	<code>string</code> · <code>claude-opus-4-8</code>	<code>SPECTRO_MODEL</code>	if provider is ollama/openai and no layer set a model, the default flips to <code>qwen3</code> / <code>local-model</code>
<code>baseUrl</code>	<code>URL</code> · <code>http://localhost:11434</code>	<code>SPECTRO_BASE_URL</code>	ignored by anthropic; the openai-compatible providers use a preset endpoint by name (openai → <code>api.openai.com</code> , lmstudio → <code>:1234</code> , openrouter → <code>openrouter.ai/api</code>) — no key-based swap; an explicit URL always wins
<code>compactionThreshold</code>	<code>int</code> · <code>100000</code>	— (flag only)	compaction trigger in input tokens
<code>permissionMode</code>	<code>ask</code> <code>auto</code> <code>readonly</code> · <code>ask</code>	—	chapter 13; headless <code>run</code> has its own <code>--permissions</code> ; live via the composer gear's <code>set_permission_mode</code>
<code>workspace</code>	<code>path</code> · <code>unset</code>	<code>SPECTRO_WORKSPACE</code> / <code>--workspace</code>	the agent's working directory (file tools, glob/grep, run_command); unset = a per-session folder <code><tmpdir>/spectroscope-ws/<session-id></code> — a resume finds its files again, and the project you started spectroscope in stays clean; process-global — a workspace scope may not set this (a folder must not reprint the agent)
<code>autoApprove</code>	<code>string[]</code> · <code>[]</code>	—	allowlist rules: <code>tool</code> or <code>tool:prefix*</code> — whole-block; lives in the workspace project file once one is pinned
<code>imageProvider</code>	<code>gemini</code> <code>openai</code> · <code>gemini</code>	<code>SPECTRO_IMAGE_PROVIDER</code>	image-generation backend
<code>thinking</code>	<code>bool</code> · <code>true</code>	<code>SPECTRO_THINKING</code>	reasoning-stream visibility (REPL <code>/think</code> , web header toggle)
<code>mcpServers</code>	<code>object</code> · <code>{}</code>	—	Claude-Desktop-shaped server map — whole-block (chapter 18)
<code>maxRetries</code>	<code>int</code> · <code>2</code>	<code>SPECTRO_MAX_RETRIES</code>	transient retry budget; 0 disables; garbage values fail readably
<code>promptCaching</code>	<code>bool</code> · <code>true</code>	<code>SPECTRO_PROMPT_CACHING</code>	Anthropic cache_control breakpoints; no-op elsewhere
<code>logLevel</code>	<code>error</code> <code>warn</code> <code>info</code> <code>debug</code> <code>trace</code> · <code>info</code>	<code>SPECTRO_LOG_LEVEL</code>	operator-log detail (chapter 31); process-global — one log file per JVM, refused in a workspace scope, user scope only
<code>imageModel</code>	<code>string</code> · <code>unset</code>	<code>SPECTRO_IMAGE_MODEL</code>	overrides the image backend's own default model; machine-tool path — belongs in user or workspace-local scope
<code>sttModel</code>	<code>path</code> · <code>unset</code>	<code>SPECTRO_STT_MODEL</code>	absolute path to the whisper.cpp model (default <code>~/.spectro/models/ggml-small.bin</code>); machine-tool path
<code>chromeBinary</code>	<code>path</code> · <code>unset</code>	<code>SPECTRO_CHROME</code>	override for the system-Chrome binary <code>browse_page</code> discovers automatically; machine-tool path
<code>hooks</code>	<code>array</code> · <code>[]</code>	—	pre/post tool hooks — whole-block (chapter 13)

tts	{enabled, voice} · off, en_US-lessac- medium	—	CLI-side voice output block in the same user settings file (not a SpectroConfig field)
-----	--	---	--

Every `SPECTRO_*` name in the "env base" column still works exactly as before, but only as the BASE layer — see the deprecation table above for the field each one now defers to, and run `spectro doctor` to see which of yours are currently shadowed. The three "machine-tool path" fields are session-scoped like any other field, but discouraged from a committed project file — the header gear and the composer gear's local-overrides section are where they belong.

Environment-only variables

Everything that used to live only in the environment has graduated to a settings field (deprecation table above). What is left is **secrets** — spectroscope never writes an API key to a settings file, and the `PUT` endpoints refuse to accept one outright (any `*_API_KEY` / `*_TOKEN` -shaped key is rejected):

VARIABLE	CONSUMED BY
ANTHROPIC_API_KEY	the Anthropic SDK; checked up front — a missing key refuses provider construction with a readable message
OPENAI_API_KEY	optional Bearer for the chat provider; required by the OpenAI image backend
OPENROUTER_API_KEY	Bearer for the <code>openrouter</code> openai-compatible provider
GEMINI_API_KEY	the Gemini image backend
TAVILY_API_KEY	the Tavily tier of <code>web_search</code> ; absent falls back to the keyless DuckDuckGo tier

Three `.env` injection paths, one parse rule. Gradle's run tasks (CLI and server) and the `./spectro` launcher all read the project root's gitignored `.env` identically: comments and blank lines skipped, first `=` splits, one quote layer stripped, **empty values excluded** — a `KEY=` line can never blank a variable you exported in your shell. The launcher's copy exists because the desktop face bypasses Gradle entirely. All JavaExec tasks also run from the project root, so the project settings layer is always found.

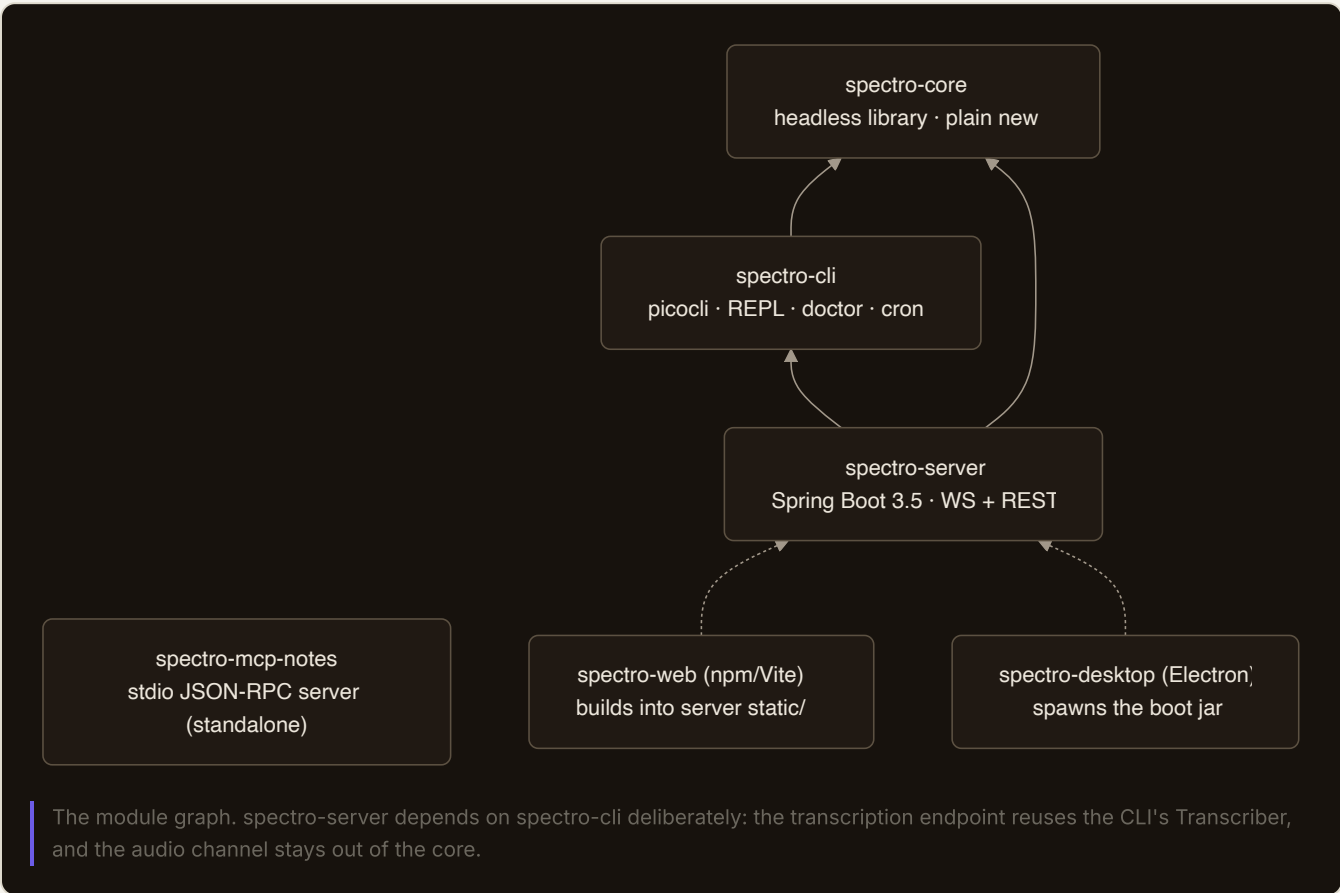
The single writer. Only `SettingsWriter` ever writes a settings file — two entry points, the same serialized read-modify-write discipline: `appendAutoApprove` (the permission dialog's "Persist"/"dauerhaft" checkbox) appends one allowlist rule to the workspace project file; `patch` (behind every `PUT /api/settings/*` call, so behind both gears) applies a schema-validated partial patch to any of the three files, rejecting secrets and process-globals in the wrong scope before anything touches disk. Both preserve every other key already in the file, and a `local` write also ensures the workspace's `.spectro/.gitignore` lists `settings.local.json`. Everything else only reads.

SPECTRO.md

A `SPECTRO.md` in the working directory is appended to the system prompt as a "Project context" section — provider-neutral, visible in the System-context panel. The counterpart of a CLAUDE.md, for spectroscope.

Build & test inventory

Five JVM modules, two npm toolchains, one version catalog — and a test suite that never needs an API key.



Modules

MODULE	ROLE	KEY FACTS
spectro-core	the headless library	java-library; Jackson is <code>api</code> (<code>JsonNode</code> appears in public contracts); the Anthropic SDK is <code>implementation</code> — no other class may import it; Spring <i>Framework</i> as a library (<code>RestClient</code> + HTTP interfaces); <code>cron-utils</code>
spectro-cli	terminal face	application; <code>picocli</code> ; <code>slf4j-nop</code> keeps output clean; registers the <code>tour</code> task; <code>stdin</code> wired for the REPL
spectro-server	web backend	Spring Boot 3.5.3 (only here); starters <code>web</code> + <code>websocket</code> ; depends on <code>core</code> <i>and</i> <code>cli</code> ; serves the built UI from <code>static/</code>
spectro-mcp-notes	example MCP server	standalone; Jackson only; its test spawns the real child JVM
spectro-orchestrator	the fleet facade	java-library; <code>Spectro.panel()</code> lanes as core agents on a bus; POM pins <code>spectro-core</code> at the same version; Maven Central library

spectro-web	browser UI	npm/Vite; React 19, React Flow 12, dagre; npm run build writes straight into the server's resources
spectro-desktop	desktop shell	npm/Electron 37; TypeScript strict; zero runtime dependencies (health polling via built-in fetch)

Version pins

LIBRARY	VERSION	
com.anthropic:anthropic-java	2.34.0	owns SSE; client built with maxRetries(0)
jackson-databind	2.17.2	the one api dependency
info.picocli:picocli	4.7.6	CLI framework
org.springframework:spring-web	6.2.8	Framework as library — no Boot in the core
com.cronutils:cron-utils	9.2.1	computes slots; a ScheduledExecutorService fires them
org.junit.jupiter	5.10.2	
Gradle wrapper	9.6.1	checked in
Spring Boot	3.5.3	spectro-server only
react / @xyflow/react / dagre	19.1 / 12.11 / 0.8.5	spectro-web
electron / electron-builder	37.2 / 26.0	spectro-desktop

The test suite

MODULE	TESTS	WHAT THEY PIN
spectro-core	270	event round-trips and wire shapes, the loop against fake providers, sessions/resume/delete, compaction, subagents (parallelism, timeout), scheduler, all three provider mappings against scripted local servers, MCP client/transports, hooks, allowlist, retry
spectro-cli	33	renderer, overrides plumbing, allowlist, voice seams
spectro-server	33	REST guards (incl. the delete endpoint), a full WebSocket round-trip against an Ollama mock, workspace sandbox, transcripts controller
spectro-mcp-notes	18	store, ranker, stdio protocol in-JVM and as a real child process
JUnit total	354	plus 252 vitest for the web UI (reducer, threads, stepper, scene models, scenario compiler, importer, markdown parser, layout stores)
spectro-web (vitest)	252	

Every test runs key-free and network-model-free: fake providers, scripted HTTP servers, injectable process seams, and a redirected user.home so no test can ever touch your real ~/.spectro. Gate: ./gradlew build + cd spectro-web && npm test.

Releases & working with the source

Two libraries go to Maven Central, one download per app module goes to the GitHub release, and the whole tree is MIT and reproducible from a script. Rule of thumb: every module ships *something*. This chapter is the map of what that something is, and how to work with the source that builds it.

What a release ships

The seven modules split cleanly into reusable Java (published to Maven Central so you can depend on it) and runnable faces (attached to the GitHub release, one download each). One module is the exception, and it is called out below.

MODULE	BUILD	SHIPS AS
spectro-core	Gradle · java-library	Maven Central library
spectro-orchestrator	Gradle · java-library	Maven Central library (POM pins core at the same version)
spectro-cli	Gradle · application	GitHub asset spectro-<v>.zip (the terminal face, bin/spectro)
spectro-server	Gradle · Spring Boot	GitHub asset spectro-server-<v>.jar (executable; embeds the built web UI)
spectro-mcp-notes	Gradle · application	GitHub asset spectro-mcp-notes-<v>.zip (the sample MCP server)
spectro-desktop	npm · Electron	GitHub asset spectroscope-<v>-<arch>.dmg (the desktop run kit; bundles server + JRE)
spectro-web	npm · Vite	not a standalone download — built into the server jar (and the desktop kit)

The build.

settings.gradle.kts includes exactly four JVM modules; spectro-web and spectro-desktop live beside the Gradle build with their own toolchains. Java 21 everywhere, tests run against build/test-home.

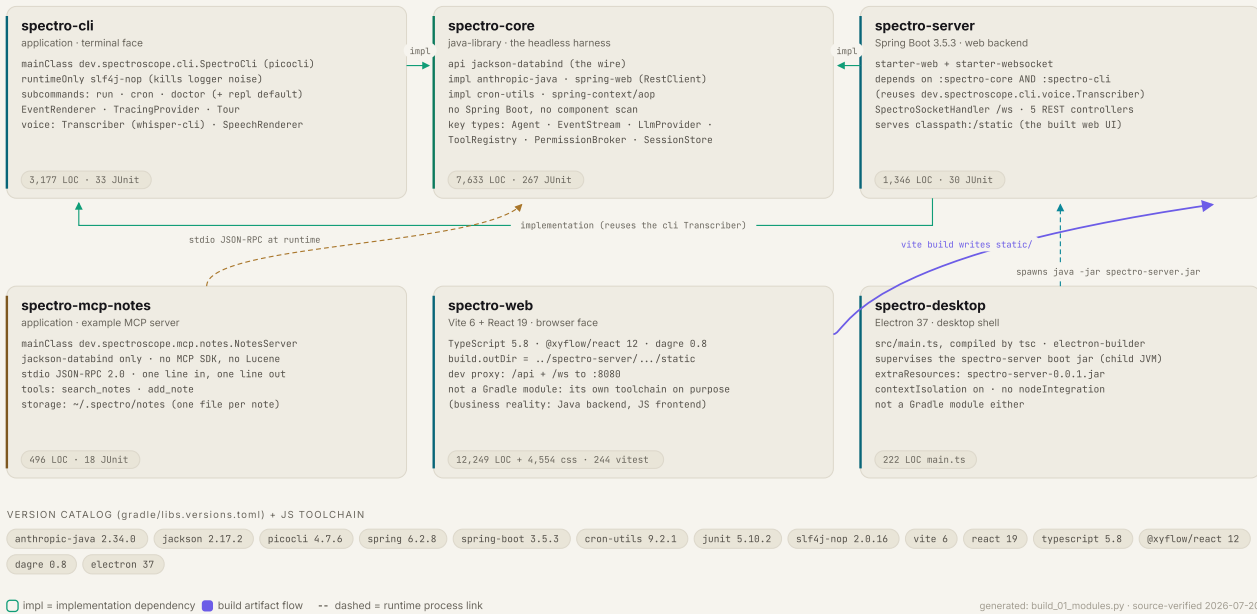


Diagram 01 — the Gradle build: five JVM modules with their dependency edges and the version catalog, plus the two npm toolchains (spectro-web, spectro-desktop) that live next to the JVM graph on purpose. What ships from each is the subject of this chapter.

Why the web has no download. `spectro-web` is a single-page app; it needs the server's REST and WebSocket API on the same origin, so a bundle handed out on its own could not run. The release build compiles the UI *first*, writing its output straight into `spectro-server`'s static resources, so the server jar below already carries the current UI. The desktop kit carries that same server jar, so the web face is present in three assets and standalone in none.

`scripts/build-release-assets.sh` produces every GitHub asset in one pass into `build/release-assets/`; the two libraries are the exception and go to Maven Central instead. On a machine without a full JDK/Node for the Electron step, `SKIP_DESKTOP=1` builds everything except the `.dmg`.

The libraries on Maven Central

The two headless modules are published under the **verified namespace** `dev.spectroscope`. Depend on the orchestrator and you get the core transitively:

```
dependencies {
    implementation("dev.spectroscope:spectro-core:0.1.0")
    implementation("dev.spectroscope:spectro-orchestrator:0.1.0") // POM pins spectro-core at 0.1.0
}
```

`v0.1.0` is the first published version. **Maven Central is append-only**: a published version cannot be pulled or overwritten, only superseded, so the next release is always a bump (`0.1.0` → `0.1.1` → ...), never a re-publish. That is why the release ritual gates hard before the single irreversible step.

Only these two ship to Maven. `spectro-server` is a Spring Boot application: its plain `jar` is a thin, non-runnable shell, so it ships as an executable `bootJar` on the GitHub release, never to Maven. `spectro-web` and `spectro-desktop` are npm/frontend modules and cannot go to Maven Central at all.

The desktop run kit

The `.dmg` is a **self-contained** app. The Electron shell (`spectro-desktop`) spawns and supervises the server, and the build bundles both the server jar *and* a `jlink`'d JRE into the app, so the target machine needs no Java installed: double-click, the server starts with it, the cockpit opens. When the window closes, the shell reaps its child JVM. The script that builds it is `scripts/build-desktop-runkit.sh`.

Two honest limits, and the release notes state them:

- **Per-platform.** Electron and the bundled JRE are OS/arch specific, and the script builds the *host* target only. Windows, Linux and Intel each need building on and for themselves.
- **Ad-hoc signed, not notarized.** On first download macOS reports an *"unidentified developer"*; open it once with right-click → Open (or clear the quarantine flag with `xattr -cr`) and it launches normally from then on. A local build has no quarantine flag and starts directly.

For a zero-warning, plain double-click app you need a paid **Apple Developer Program** membership, a Developer ID Application certificate, and notarization. The one spectroscopy-specific twist is signing the bundled JRE under the hardened runtime; the full end-to-end procedure is `docs/DESKTOP-SIGNING.md`. Sign only with the ad-hoc identity (`-`) otherwise, never a corporate certificate the machine may happen to list.

Working with the source

The layout is the module table above, read as folders. `settings.gradle.kts` includes the five JVM modules (`spectro-core`, `-cli`, `-server`, `-mcp-notes`, `-orchestrator`); the two npm modules (`spectro-web`, `spectro-desktop`) sit next to the Gradle build with their own toolchains, on purpose — a Java backend and JS frontends are separate worlds and the build does not pretend otherwise. The gate is two commands:

```
./gradlew build                # the whole JVM suite + all tests
( cd spectro-web && npm test )  # the web UI suite (vitest)
```

Neither needs an API key: the tests run against fake providers and scripted local servers, and the one live contract check self-skips unless `ANTHROPIC_API_KEY` is set (chapter 31 has the full inventory). The whole tree is **MIT licensed** (`LICENSE`, © 2026 Christopher Ezell) — use it, fork it, ship it, keep the notice.

The release ritual

Cutting a release is a written runbook, `docs/RELEASE-PLAYBOOK.md` (`RELEASING.md` is the narrower "just Maven Central" companion; the two agree). The order exists to put every irreversible step behind a green gate:

1. **Preflight** — clean tree, on `main`, and a version strictly higher than the last published (Central is append-only).
2. **Bump versions together** — the two Maven libs (`spectro-core`, `spectro-orchestrator`) and the app versions that name the assets (`spectro-cli`, `-server`, `-mcp-notes`, and `spectro-desktop/package.json`).
3. **Full green gate** — `./gradlew test` plus the two `javadoc` tasks, then `vitest`. At `v0.1.0` the baseline was **678 JUnit** (one skipped: the live-Opus check) and **383 vitest**.

4. **Dry-run the publish** — `publishToMavenLocal` for both libs confirms GPG signing and POM generation (every artifact gets a `.asc`; the orchestrator POM depends on `spectro-core` at the same version) without touching the portal.
5. **Commit + tag** `v<v>`, push both.
6. **Publish the libraries** — `./gradlew publishAndReleaseToMavenCentral`. This is the one irreversible step; wait for the coordinates to resolve on `repo1.maven.org` before trusting them.
7. **Build the assets** — `scripts/build-release-assets.sh`, then `gh release create v<v>` with everything in `build/release-assets/`.
8. **Flip the install snippets** — the landing page and the dev portal go from "planned" to the real coordinates; push is deploy, so verify the live copy last.

One-time owner prerequisites (a Central Portal token and GPG key in `~/.gradle/gradle.properties`, a full JDK for `jlink`, Node for the web bundle and Electron, and `gh` authenticated) are listed at the top of the playbook. The gitignored build artifacts (`jre/`, `build/`) never get committed.

The brand mark

The mark is a spectrum. Five vertical lines of different weights and colours, read left to right — the same figure the event stream makes when it fans into its lines. It sits on the cover of this guide. Colour lives only on the lines (a coral edge, amber, a broad teal, cyan, a brown edge); the wordmark is always **lowercase**, one word, set flat with no shadow and no glow. That restraint is deliberate and matches the interface, where colour is reserved for the spectral lines and nothing else carries it.

Saving a key, safely

You can paste an API key into the cockpit and have spectroscope save it for you. That is a web page writing a secret to disk, so it only ships fenced. This is the honest account of the fences: what the feature does, why it takes effect without a restart, and exactly which requests it refuses.

Where the key goes

The key lands in one place: `~/.spectro/.env`, written at `0600` (owner read and write only). It is **never** put in `settings.json` — those files are meant to be shared and checked into a project, so a secret there is a leak waiting to happen. It never enters a session file, a log line, or any GET response: the API reports a key's *presence* (`true` / `false`), never its value. There is one write path, the POST below; the value leaves your browser once, as that request body, and is never sent back.

Why it works without a restart

A running JVM cannot change its own environment: `System.getenv` is frozen for the life of the process. So a key merely written to disk would be inert until the next launch. spectroscope closes that with a read seam. Every provider resolves its key as *environment first, then* `~/.spectro/.env` (`SpectroConfig.resolveApiKey`). Providers are built per chat, so the moment you save a key the next new chat picks it up: no restart, and it works the same from the jar and the desktop app, not only the launcher's `./.env`. A real environment variable or a launcher-loaded `./.env` still wins, so nothing you set on the command line is ever overridden.

The two fences on the write

The endpoint is `POST /api/onboarding/key`. It is bound to `127.0.0.1` like the rest of the server, and it passes only when **both** of these hold:

FENCE	WHAT IT CHECKS	WHAT IT STOPS
local origin	the TCP peer is loopback <i>and</i> the <code>Host</code> header is <code>localhost</code> / <code>127.0.0.1</code>	a remote caller, and a DNS-rebinding attack that aims a public name at your loopback
same origin	the <code>Origin</code> header is absent (a non-browser client) <i>or</i> points at loopback	CSRF: a page on <code>evil.com</code> firing a <code>fetch</code> at your local server. Its request is still loopback, but its Origin is its own domain, so it is refused

The second fence matters because the read side of this controller deliberately allows cross-origin requests, so the Vite dev server on another port can read `/api/config`. Reads are harmless; a *write* is not, so the write checks the Origin itself instead of trusting the CORS policy. A refused request answers `404`, not `403`: an attacker gets no signal that the endpoint is even there.

What an attacker can and cannot do

Plainly, for the sceptics:

- A website you visit **cannot** write a key here. The Origin fence answers it 404 (verified: an `evil.com` Origin is rejected, a loopback Origin is accepted).
- A remote machine **cannot** reach the endpoint. The server binds loopback, and the Host fence 404s a rebinding attempt.
- Nobody can **read** a saved key back through the API. Every surface is presence-only; the value sits on disk at `0600` and in the process, nowhere else.
- The honest residual: a malicious program *already running on your machine* could POST a key (its Origin is loopback) or just read the file directly. But a hostile local process can do far worse than this anyway, so it is outside the line this feature draws. That line is external websites and remote hosts, and both are closed.

You never have to use it

The dialog is a convenience, not the only door. The same key is read from a real environment variable, from a `./.env` next to the launcher (`./spectro-serve --env-file`), or typed hidden on the command line with `./spectro-serve set-key` . All four land in the same resolver, so use whichever you trust. The cockpit field is there for the person who would rather not open a terminal.

Troubleshooting

SYMPTOM	CAUSE & FIX
<code>zsh: operation timed out: ./spectro</code>	The folder arrived as a download — every file carries the macOS quarantine flag, and Gatekeeper hangs in an online notarization check on exec. Fix: <code>xattr -cr <path>/spectroscope</code> (read-only <code>.git/objects</code> complaints are harmless).
<code>ollama unreachable</code> in doctor	The Ollama server is not running — open the app or <code>ollama serve</code> . Probe: <code>curl -s http://localhost:11434/api/tags</code> .
Gradle: “release version 21 not supported”	Your default JDK is older than 21. The launcher normally solves this; without it, install <code>brew install openjdk@21</code> or set <code>JAVA_HOME</code> .
git hangs / <code>Operation timed out</code> on <code>git diff</code>	On a sync filesystem (OneDrive/iCloud) the fsmonitor daemon can wedge. Fix: <code>git config core.fsmonitor false</code> .
Model without vision error	You attached an image while a text-only local model is selected. <code>ollama pull qwen3-vl</code> (or llava) and switch, or use a cloud provider.
Mic button disabled with a tooltip	STT is not installed — <code>bash scripts/setup-stt.sh</code> ; or the browser denied microphone access.
Voice output disabled: piper, voice or audio player missing	<code>bash scripts/setup-tts.sh</code> (note: its checksums are pin-on-first-run — the script prints the sums it computed and asks you to verify and paste them).
Anthropic switch refused in the picker	<code>ANTHROPIC_API_KEY</code> is not set for the <i>server</i> process — put it in <code>spectroscope/.env</code> and restart <code>./spectro web</code> .
Desktop window blank after a rebuild	An old instance held Electron's single-instance lock. <code>./spectro desktop</code> stops stale instances first — use it instead of a bare <code>npm start</code> .
MCP server UNREACHABLE in doctor	The configured <code>command</code> path does not exist on this machine (the settings file carries machine-specific absolute paths) or the dist was never built: <code>./spectro mcp-notes</code> .
<code>SPECTRO_MAX_RETRIES</code> must be an integer	Exactly what it says — the env var refuses garbage loudly instead of silently defaulting.

Reproducing this guide

This document is generated — never hand-edited — and every asset in it is reproducible from the repository.

SCREENSHOTS	<code>docs/guide-assets/capture_screens.mjs</code> — Playwright on the system Chrome against a running <code>./spectro web</code> . Deterministic: EN chrome seeded before load, scenario replays for all feature states, two live local-model runs for the plan tab and the permission dialog (the delete shot only <i>arms</i> the button and lets it disarm). Output: <code>docs/guide-assets/shots/*.png</code> .
MERMAID DIAGRAMS	<code>docs/guide-assets/mermaid-src/*.mmd</code> rendered to SVG by <code>render_mermaid.mjs</code> (mermaid 11, dark theme on the spectroscope tokens) — the guide inlines the SVGs so the PDF needs no JavaScript.
ARCHITECTURE SVGS	generated by <code>docs/diagrams/build_NN_*.py</code> ; rerun with <code>for g in build_*.py; do python3 \$g; done</code> .
TERMINAL CAPTURES	real output of <code>./spectro</code> and <code>./spectro doctor</code> on 2026-07-16, ANSI stripped at build time.
ASSEMBLY	<code>python3 docs/guide-assets/build_user_guide.py</code> — inlines the fonts (from the server's static assets), all images as data URIs, and the content parts from <code>docs/guide-assets/parts/</code> into the self-contained <code>docs/USER-GUIDE.html</code> .
PDF	headless Chrome: <code>--headless --no-pdf-header-footer --print-to-pdf=docs/USER-GUIDE.pdf docs/USER-GUIDE.html</code> . Dark pages incl. margins come from the thead/tfoot page-spacer technique with <code>@page margin 0</code> .
SOURCE OF TRUTH	all facts verified against the v0.1.0 source tree; gate: 678 JUnit + 383 vitest, 0 failures.

Credits and colophon

spectroscope is designed and built by **Christopher Ezell**, with Claude (Anthropic) doing much of the heavy lifting across code, design and copy. It began as the reference harness of a build-an-agent-harness workshop and grew into its own product; the brand, the orchestrator and this guide came after.

The guide is generated, never hand-written. The type is Inter and JetBrains Mono, both embedded so the pages read the same on any machine. The architecture plates are drawn by Python generators; the sequence and flow figures are Mermaid; the screenshots are real captures of spectro web. Everything assembles into one self-contained HTML file and a PDF, in an espresso-dark and a paper-light edition. The wire format is shared byte-for-byte with the TypeScript edition, so a session recorded by one is replayable by the other.

Fonts: Inter and JetBrains Mono, both SIL Open Font License 1.1. Diagrams: Mermaid (MIT). Rendering and capture: headless Chrome. spectroscope itself is MIT licensed and pre-release. Contact: chris (at) spectroscope.ai

spectroscope — the agent orchestrator you can watch. This guide ships in espresso dark and paper light, like the product. The wire format is shared byte-identically with the TypeScript edition, so sessions are forever.